

Métodos numéricos para ingenieros

QUINTA EDICIÓN

Mc
Graw
Hill

Steven C. Chapra
Raymond P. Canale

CAPÍTULO 2

Programación y software

En el capítulo anterior, desarrollamos un modelo matemático a partir de la fuerza total para predecir la velocidad de caída de un paracaidista. Este modelo tenía la forma de una ecuación diferencial,

$$\frac{dv}{dt} = g - \frac{c}{m} v$$

También vimos que se obtenía una solución de esta ecuación utilizando un método numérico simple, llamado método de Euler,

$$v_{i+1} = v_i + \frac{dv_i}{dt} \Delta t$$

Dada una condición inicial, se emplea esta ecuación repetidamente para calcular la velocidad como una función del tiempo. Sin embargo, para obtener una buena precisión sería necesario desarrollar muchos pasos pequeños. Hacerlo a mano sería muy laborioso y tomaría mucho tiempo; pero, con la ayuda de las computadoras tales cálculos pueden realizarse fácilmente.

Por ende, nuestro siguiente objetivo consiste en observar cómo se hace esto. En el presente capítulo daremos una introducción al uso de la computadora como una herramienta para obtener soluciones de este tipo.

2.1 PAQUETES Y PROGRAMACIÓN

En la actualidad existen dos tipos de usuarios de software. Por un lado están aquellos que toman lo que se les da. Es decir, quienes se limitan a las capacidades que encuentran en el modo estándar de operación del software existente. Por ejemplo, resulta muy sencillo resolver un sistema de ecuaciones lineales o generar una gráfica con valores x - y con Excel o con MATLAB. Como este modo de operación por lo común requiere un mínimo esfuerzo, muchos de los usuarios adoptan este modo de operación. Además, como los diseñadores de estos paquetes se anticipan a la mayoría de las necesidades típicas de los usuarios, muchos de los problemas pueden resolverse de esta manera.

Pero, ¿qué pasa cuando se presentan problemas que están más allá de las capacidades estándar de dichas herramientas? Por desgracia, decir “Lo siento jefe, pero no lo sé hacer” no es algo aceptado en la mayoría de los círculos de la ingeniería. En tales casos usted tiene dos alternativas.

La primera sería buscar otro paquete y ver si sirve para resolver el problema. Ésta es una de las razones por las que quisimos usar tanto Excel como MATLAB en este libro. Como veremos, ninguno de los dos abarca todo y cada uno tiene sus ventajas.

Sabiendo usar ambos, se amplía de forma notable el rango de problemas que pueden resolverse.

La segunda sería que es posible volverse un “potente usuario” si se aprende a escribir macros en Excel VBA¹ o archivos M (M-files) en MATLAB. ¿Y qué son tales cuestiones? No son más que programas computacionales que permiten ampliar la capacidad de estas herramientas. Como los ingenieros nunca se sentirán satisfechos al verse limitados por las herramientas, harán todo lo que sea necesario para resolver sus problemas. Una buena manera de lograrlo consiste en aprender a escribir programas en los ambientes de Excel y MATLAB. Además, las habilidades necesarias para crear macros o archivos M (M-files) son las mismas que se necesitan para desarrollar efectivamente programas en lenguajes como Fortran 90 o C.

El objetivo principal del capítulo es enseñarle cómo se hace esto. Sin embargo, supondremos que usted ya ha tenido contacto con los rudimentos de la programación y, por tal razón, destacaremos las facetas de la programación que afectan directamente su uso en la solución de problemas en ingeniería.

2.1.1 Programas computacionales

Los *programas computacionales* son únicamente conjuntos de instrucciones que dirigen a la computadora para realizar una cierta tarea. Hay mucha gente que escribe programas para un amplio rango de aplicaciones en los lenguajes de alto nivel, como Fortran 90 o C, porque tienen una gran variedad de capacidades. Aunque habrá algunos ingenieros que usarán toda la amplia gama de capacidades, la mayoría sólo necesitará realizar los cálculos numéricos orientados a la ingeniería.

Visto desde esta perspectiva, reducimos toda esa complejidad a unos cuantos tópicos de programación, que son:

- Representación de información sencilla (declaración de constantes, variables y tipos)
- Representación de información más compleja (estructuras de datos, arreglos y registros)
- Fórmulas matemáticas (asignación, reglas de prioridad y funciones intrínsecas)
- Entrada/Salida
- Representación lógica (secuencia, selección y repetición)
- Programación modular (funciones y subrutinas)

Como suponemos que el lector ya ha tenido algún contacto con la programación, no dedicaremos mucho tiempo en las cuatro primeras áreas. En lugar de ello, las presentamos como una lista para que el lector verifique lo que necesitará saber para desarrollar los programas que siguen.

No obstante, sí dedicaremos algún tiempo a los dos últimos tópicos. Destacaremos la representación lógica porque es el área que más influye en la coherencia y la comprensión de un algoritmo. Trataremos la programación modular porque también contribuye de manera importante en la organización de un programa. Además, los módulos son un medio para almacenar algoritmos utilizados frecuentemente en un formato adecuado para aplicaciones subsecuentes.

¹ VBA son las siglas de Visual Basic for Applications.

2.2 PROGRAMACIÓN ESTRUCTURADA

En los comienzos de la computación, los programadores no daban mucha importancia a que sus programas fueran claros y fáciles de entender. Sin embargo, hoy se reconoce que escribir programas organizados y bien estructurados tiene muchas ventajas. Además de las ventajas obvias de tener un software más accesible para compartirlo, también ayuda a generar programas mucho más eficientes. Es decir, algoritmos bien estructurados, que son invariablemente mucho más fáciles de depurar y de probar, lo que resulta en programas que toman menos tiempo desarrollar, probar y actualizar.

Los científicos de la computación han estudiado sistemáticamente los factores y los procedimientos necesarios para desarrollar software de alta calidad de este tipo. En esencia la *programación estructurada* es un conjunto de reglas que desarrollan en el programador los hábitos para lograr un buen estilo. Aunque la programación estructurada es bastante flexible para permitir considerable creatividad y expresión personal, sus reglas imponen suficientes restricciones para hacer que los programas resultantes sean muy superiores a sus versiones no estructuradas. En particular, el producto terminado es mucho más elegante y fácil de entender.

La idea clave detrás de la programación estructurada es que cualquier algoritmo numérico requiere tan sólo de tres estructuras de control fundamentales: secuencia, selección y repetición. Limitándonos a dichas estructuras el programa resultante será claro y fácil de seguir.

En los párrafos siguientes describiremos cada una de estas estructuras. Para mantener esta descripción de una manera general usaremos diagramas de flujo y pseudocódigo. Un *diagrama de flujo* es una representación visual o gráfica de un algoritmo. Un diagrama de flujo emplea una serie de cajas o bloques y flechas, cada una de las cuales representa un determinado paso u operación del algoritmo (figura 2.1). Las flechas representan el orden en el que se realizarán las operaciones.

No todas las personas relacionadas con la computación están de acuerdo en que los diagramas de flujo sean una buena opción. Incluso, algunos programadores experimentados no usan los diagramas de flujo. Sin embargo, nosotros pensamos que existen tres buenas razones para estudiarlos. La primera es que sirven para expresar y comunicar algoritmos. La segunda es que aunque no se empleen de manera rutinaria, algunas veces resultarán útiles para planear, aclarar o comunicar la lógica del propio programa o del de otra persona. Por último, que es lo más importante para nuestros objetivos, son excelentes herramientas didácticas. Desde el punto de vista de la enseñanza, son los medios ideales para visualizar algunas de las estructuras de control fundamentales que se emplean en la programación.

Otra manera de expresar algoritmos, y que constituye un puente de unión entre los diagramas de flujo y el código de la computadora, es el *seudocódigo*. En esta técnica se utilizan expresiones semejantes a las del código, en lugar de los símbolos gráficos del diagrama de flujo. En esta obra, para el pseudocódigo hemos adoptado algunas convenciones de estilo. Escribiremos con mayúsculas las palabras clave como IF, DO, INPUT, etc., mientras que las condiciones, pasos del proceso y tareas irán en minúsculas. Además, los pasos del proceso se escribirán en forma indentada. De esta manera las palabras clave forman un “sandwich” alrededor de los pasos para definir visualmente lo que abarca cada estructura de control.

SÍMBOLO	NOMBRE	FUNCIÓN
	Terminal	Representa el inicio o el final de un programa.
	Líneas de flujo	Representan el flujo de la lógica. Los arcos en la flecha horizontal indican que ésta pasa sobre las líneas de flujo verticales y no se conecta con ellas.
	Proceso	Representa cálculos o manipulación de datos.
	Entrada/Salida	Representa entrada o salida de datos e información.
	Decisión	Representa una comparación, una pregunta o una decisión que determina los caminos alternativos a seguir.
	Unión	Representa la confluencia de líneas de flujo.
	Conexión de fin de página	Representa una interrupción que continúa en otra página.
	Ciclo de cuenta controlada	Se usa para <i>ciclos</i> que repiten un número predeterminado de iteraciones.

FIGURA 2.1

Símbolos usados en los diagramas de flujo.

Una ventaja del pseudocódigo es que con él resulta más fácil desarrollar un programa que con el diagrama de flujo. El pseudocódigo es también más fácil de modificar y de compartir con los demás. No obstante, los diagramas de flujo, debido a su forma gráfica, resultan a veces más adecuados para visualizar algoritmos complejos. Nosotros emplearemos diagramas de flujo con fines didácticos, y el pseudocódigo será el principal medio que usaremos para comunicar algoritmos relacionados con métodos numéricos.

2.2.1 Representación lógica

Secuencia. La estructura secuencial expresa la trivial idea de que, a menos que se indique otra cosa, el código debe realizarse instrucción por instrucción. Como en la figura 2.2, la estructura se puede expresar de manera general como un diagrama de flujo o como un pseudocódigo.

Selección. En contraste con el paso por paso de la estructura secuencial, la selección nos ofrece un medio de dividir el flujo del programa en ramas considerando el resultado de una condición lógica. La figura 2.3 muestra las dos principales maneras de hacer esto.

La decisión ante una sola alternativa, o estructura *IF/THEN* (figura 2.3a), nos permite una desviación en el flujo del programa si una condición lógica es verdadera. Si esta condición es falsa no ocurre nada y el programa continúa con la indicación que se encuentra después del *ENDIF*. La decisión ante dos alternativas, o estructura *IF/THEN/ELSE* (figura 2.3b), se comporta de la misma manera si la condición es verdadera; sin embargo, si la condición es falsa, el programa realiza las instrucciones entre el *ELSE* y el *ENDIF*.

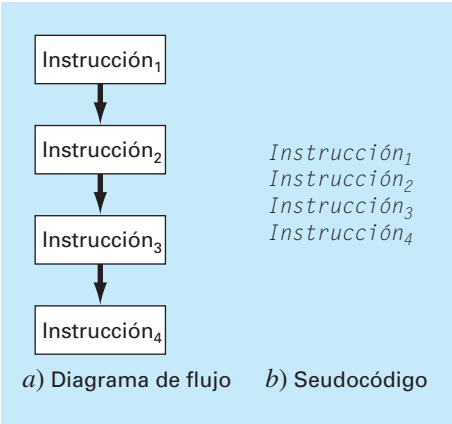
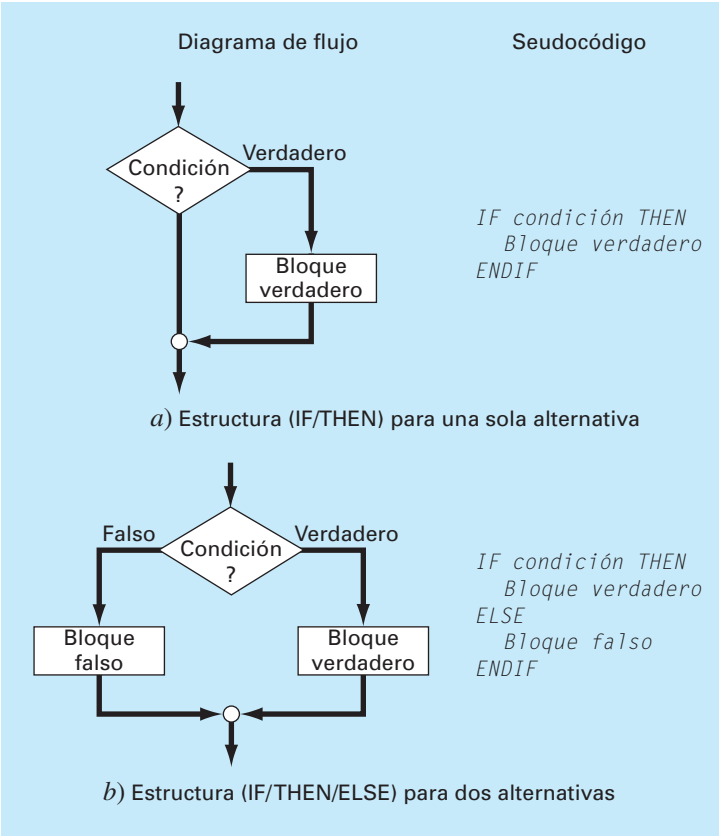


FIGURA 2.2
a) Diagrama de flujo y
b) pseudocódigo para la
estructura secuencial.

Aunque las estructuras IF/THEN e IF/THEN/ELSE son suficientes para construir cualquier algoritmo numérico, por lo común también se usan otras dos variantes. Suponga que el ELSE de un IF/THEN/ELSE contiene otro IF/THEN. En tales casos el ELSE y el IF se pueden combinar en la estructura IF/THEN/ELSEIF que se muestra en la figura 2.4a.

FIGURA 2.3
Diagrama de flujo y pseudo-
código para estructuras de
selección simple.
a) Selección con una alter-
nativa (IF/THEN) y b) se-
lección con dos alternativas
(IF/THEN/ELSE).



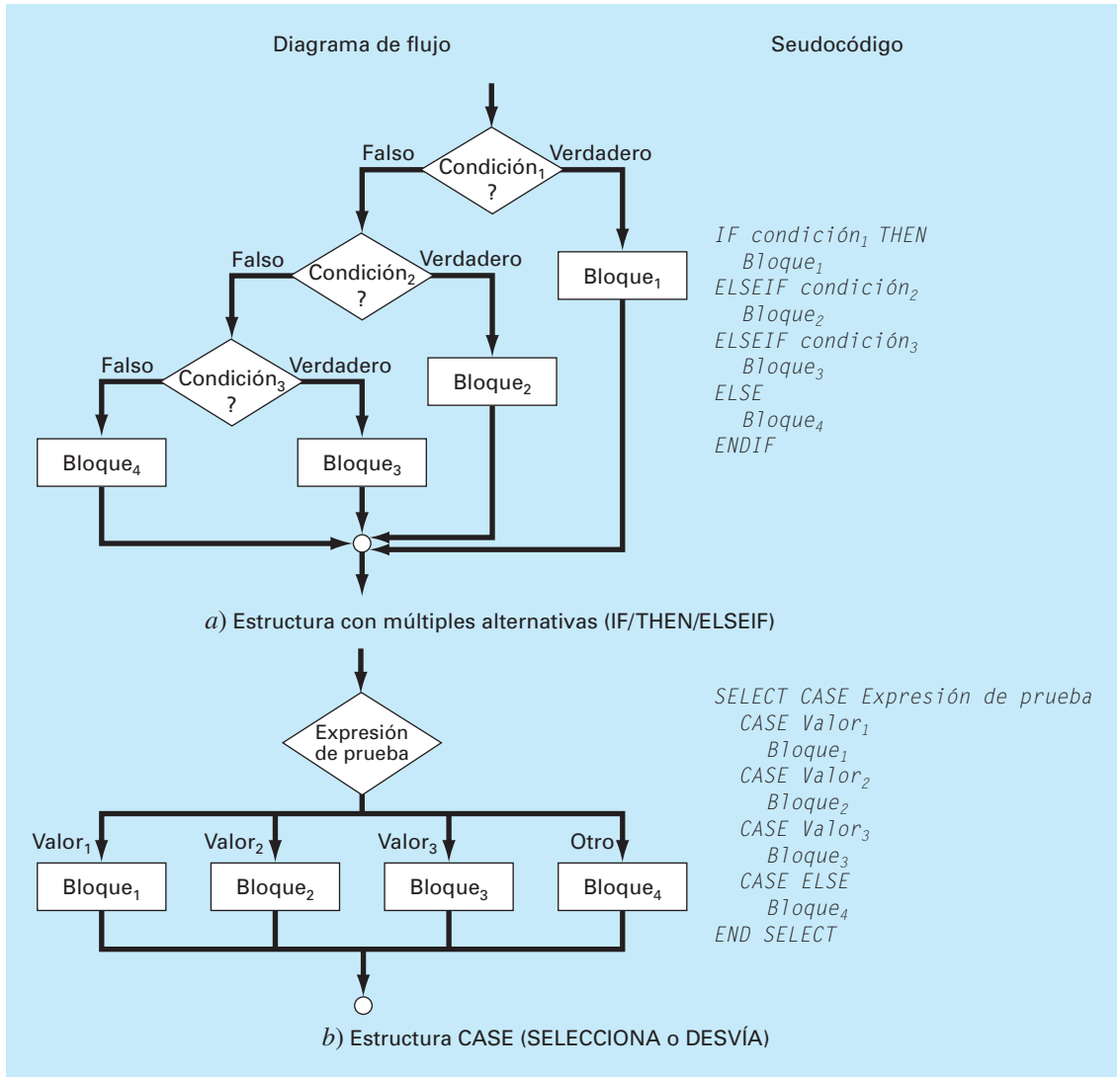
**FIGURA 2.4**

Diagrama de flujo y pseudocódigo para construcciones de selección o ramificación.
a) Selección de múltiples alternativas (IF/THEN/ELSEIF) y b) Construcción CASE.

Observe que en la figura 2.4a hay una cadena o “cascada” de decisiones. La primera es una instrucción IF y cada una de las decisiones sucesivas es un ELSEIF. Siguiendo la cadena hacia abajo, la primera condición que resulte verdadera ocasionará una desviación a su correspondiente bloque de código, seguida por la salida de la estructura. Al final de la cadena de condiciones, si todas las condiciones resultaron falsas, se puede adicionar un bloque ELSE opcional.

La estructura CASE es una variante de este tipo de toma de decisiones (figura 2.4b). En lugar de probar condiciones individuales, las ramificaciones dependen del valor de una sola *expresión de prueba*. Según sea su valor, se presentarán diferentes bloques de código. Además, si la expresión no toma ninguno de los valores previstos, se puede proponer un bloque opcional (CASE ELSE).

Repetición. La repetición nos proporciona una manera de llevar a cabo instrucciones repetidamente. Las estructuras resultantes, llamadas *loops* o ciclos, se presentan en dos formas distintas que se diferencian por la manera en que terminan.

El primer tipo, y el fundamental, es el llamado *loop de decisión* debido a que termina basándose en el resultado de una condición lógica. La figura 2.5 muestra el tipo general de *loop* de decisión, la *construcción DOEXIT*, también llamada *loop de interrupción* (*break loop*). Esta estructura realiza repeticiones hasta que una condición lógica resulte verdadera.

En esta estructura no es necesario tener dos bloques. Cuando se omite el primer bloque, a la estructura se le suele llamar *loop de preprueba* porque la prueba lógica se realiza antes de que ocurra algo. Si se omite el segundo bloque, se le llama *loop posprueba*. Al caso general, en el que se incluyen los dos bloques, se le llama *loop de prueba intermedia* (*midtest*).

Hay que hacer notar que el *loop* DOEXIT fue introducido en Fortran 90 para tratar de simplificar los loops de decisión. Esta estructura de control es parte estándar del lenguaje VBA de macros en Excel; pero no forma parte estándar de C o de MATLAB, que usan la estructura llamada WHILE. Como nosotros consideramos superior a la estructura DOEXIT, la hemos adoptado en este libro como la estructura de loop de decisión. Para que nuestros algoritmos se realicen tanto en MATLAB como en Excel, mostraremos más adelante, en este capítulo (véase la sección 2.5), cómo simular el *loop* de interrupción usando la estructura WHILE.

FIGURA 2.5

Loop DOEXIT
o de interrupción.

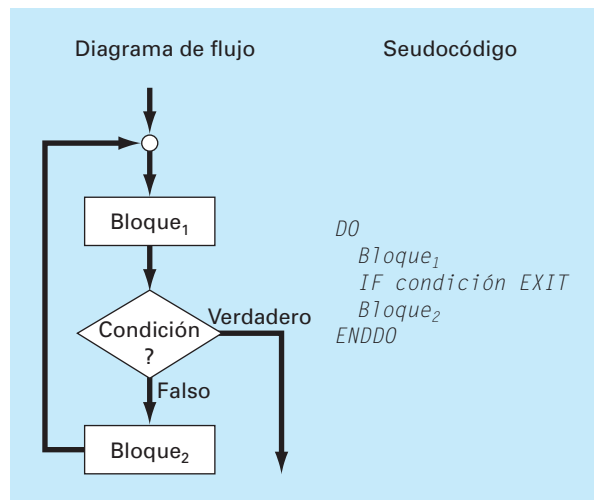
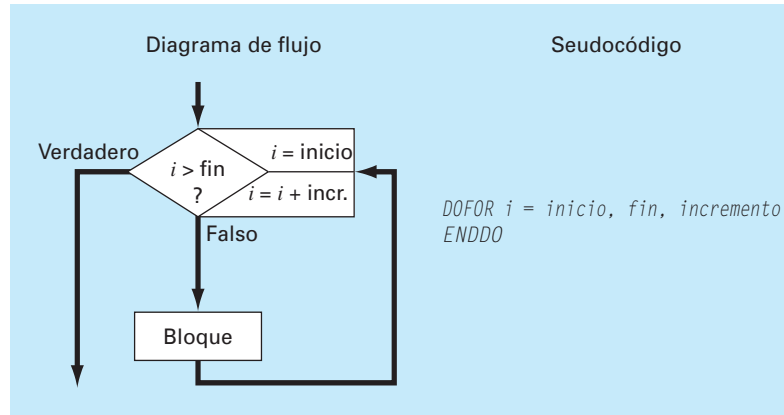


FIGURA 2.6
Construcción controlada
por conteo o construcción
DOFOR.



Al *loop* de interrupción que se presenta en la figura 2.5 se le llama *loop* lógico porque termina a causa de una condición lógica. Por otro lado, se tiene el *loop controlado por contador* o *loop DOFOR* (figura 2.6) que realiza un número determinado de repeticiones o iteraciones.

El *loop* controlado por contador funciona como sigue. El índice (representado por i en la figura 2.6) es una variable a la que se le da un valor *inicial*. El programa prueba si el *índice* es menor o igual al valor final, *fin*. Si es así, entonces ejecuta el cuerpo del *loop* y vuelve al DO. Cada vez que encuentra el ENDDO el *índice* se incrementa automáticamente con el valor definido por el incremento. De manera que el índice actúa como un contador. Cuando el *índice* es mayor que el valor final (*fin*), la computadora sale automáticamente del *loop* y transfiere el control a la línea que sigue después del ENDDO. Observe que casi en todos los lenguajes de programación, incluyendo Excel y MATLAB, si se omite el *incremento*, la computadora supone que éste es igual a 1.²

Los algoritmos numéricos que se describen en las páginas siguientes se desarrollarán usando únicamente las estructuras presentadas en las figuras 2.2 a 2.6. El ejemplo siguiente presenta el método básico para desarrollar un algoritmo que determine las raíces de la ecuación cuadrática.

EJEMPLO 2.1 Algoritmo para las raíces de la ecuación cuadrática

Planteamiento del problema. Las raíces de una ecuación cuadrática

$$ax^2 + bx + c = 0$$

se determinan mediante la fórmula cuadrática,

$$\begin{aligned} x_1 &= \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \\ x_2 & \end{aligned} \quad (2.1)$$

² Se puede usar incremento (decremento) negativo, en cuyo caso el *loop* termina cuando el índice es menor que el valor final.

Desarrolle un algoritmo que haga lo siguiente:

Paso 1: Pida al usuario los coeficientes a , b y c .

Paso 2: Realice las operaciones de la fórmula cuadrática previendo todas las eventualidades (como, por ejemplo, evitar la división entre cero y permitir raíces complejas).

Paso 3: Dé la solución, es decir, los valores de x .

Paso 4: Dé al usuario la opción de volver al paso 1 y repetir el proceso.

Solución. Para desarrollar el algoritmo usaremos un método que va de lo general a lo particular (método *top-down*). Esto es, iremos refinando cada vez más el algoritmo en lugar de detallar todo a la primera vez.

Para esto, supongamos, por lo pronto, que ya probamos que están bien los valores de los coeficientes de la fórmula cuadrática (claro que esto no es cierto, pero por lo pronto así lo consideraremos). Un algoritmo estructurado para realizar la tarea es

```
DO
    INPUT a, b, c
    r1 = (-b + SQRT (b2 - 4ac)) / (2a)
    r2 = (-b - SQRT (b2 - 4ac)) / (2a)
    DISPLAY r1, r2
    DISPLAY '¿Repetir? Conteste sí o no'
    INPUT respuesta
    IF respuesta = 'no' EXIT
ENDDO
```

La construcción DOEXIT se utiliza para repetir el cálculo de la ecuación cuadrática siempre que la condición sea falsa. La condición depende del valor de la variable de tipo carácter *respuesta*. Si *respuesta* es igual a 'sí' entonces se llevan a cabo los cálculos. Si no es así, si *respuesta* es igual a 'no', el *loop* termina. De esta manera, el usuario controla la terminación mediante el valor de *respuesta*.

Ahora bien, aunque el algoritmo anterior funcionará bien en ciertos casos, todavía no está completo. El algoritmo quizá no funcione para algunos valores de las variables. Esto es:

- Si $a = 0$ se presentará inmediatamente un problema debido a la división entre cero. Si inspeccionamos cuidadosamente la ecuación (2.1) veremos que aquí se pueden presentar dos casos:
 - Si $b \neq 0$, la ecuación se reduce a una ecuación lineal con una raíz real, $-c/b$
 - Si $b = 0$, entonces no hay solución. Es decir, el problema es trivial.
- Si $a \neq 0$, entonces, según sea el valor del discriminante, $d = b^2 - 4ac$, se pueden presentar también dos casos,
 - Si $d \geq 0$, habrá dos raíces reales.*
 - Si $d < 0$, habrá dos raíces complejas.

Observe cómo hemos dejado una sangría adicional para hacer resaltar la estructura de decisión que subyace a las matemáticas. Esta estructura se traduce, después, en un conjunto de estructuras IF/THEN/ELSE acopladas que se pueden insertar en la parte con los comandos sombreados en el código anterior, obteniéndose finalmente el algoritmo:

* En realidad si $d = 0$ las dos raíces reales tienen el mismo valor $x = -b/2a$.

```

DO
  INPUT a, b, c
  r1 = 0: r2 = 0: i1 = 0: i2 = 0
  IF a = 0 THEN
    IF b ≠ 0 THEN
      r1 = -c/b
    ELSE
      DISPLAY "Solución trivial"
    ENDIF
  ELSE
    discr = b² - 4 * a * c
    IF discr ≥ 0 THEN
      r1 = (-b + Sqrt(discr))/(2 * a)
      r2 = (-b - Sqrt(discr))/(2 * a)
    ELSE
      r1 = -b/(2 * a)
      r2 = r1
      i1 = Sqrt(Abs(discr))/(2 * a)
      i2 = -i1
    ENDIF
  ENDIF
  DISPLAY r1, r2, i1, i2
  DISPLAY '¿Repetir? Conteste sí o no'
  INPUT respuesta
  IF respuesta = 'no' EXIT
ENDDO

```

El método que se utilizó en el problema anterior puede emplearse para desarrollar un algoritmo para el problema del paracaidista. Recordemos que, dadas la condición inicial para tiempo y velocidad, el problema consistía en resolver de manera iterativa la fórmula

$$v_{i+1} = v_i + \frac{dv_i}{dt} \Delta t \quad (2.2)$$

Como sabemos, para lograr una buena precisión será necesario emplear incrementos pequeños. Por lo que será necesario emplear la fórmula repetidas veces, desde el tiempo inicial hasta el tiempo final. En consecuencia, un algoritmo para resolver este problema estará basado en el uso de un *loop*.

Supongamos, por ejemplo, que empezamos los cálculos en $t = 0$ y queremos predecir la velocidad en $t = 4$ s con incrementos de tiempo $\Delta t = 0.5$ s. Entonces tendremos que aplicar la ecuación (2.2) ocho veces, esto es,

$$n = \frac{4}{0.5} = 8$$

donde n es el número de iteraciones del *loop*. Como este número es exacto, es decir, esta división nos da un número entero, podemos usar como base del algoritmo un *loop* controlado por contador. A continuación damos un ejemplo de pseudocódigo.

```

g = 9.8
INPUT cd, m
INPUT ti, vi, tf, dt
t = ti
v = vi
n = (tf - ti) / dt
DOFOR i = 1 TO n
    dvdt = g - (cd / m) * v
    v = v + dvdt * dt
    t = t + dt
ENDDO
DISPLAY v

```

Aunque este esquema es fácil de programar, no está completo. Sólo funcionará si el intervalo es divisible exactamente entre el incremento.³ Para tomar en cuenta el otro caso, en el código anterior, en lugar del área sombreada se puede usar un *loop* de decisión. El resultado es:

```

g = 9.8
INPUT cd, m
INPUT ti, vi, tf, dt
t = ti
v = vi
h = dt
DO
    IF t + dt > tf THEN
        h = tf - t
    ENDF
    dvdt = g - (cd / m) * v
    v = v + dvdt * h
    t = t + h
    IF t ≥ tf EXIT
ENDDO
DISPLAY v

```

Al introducir el *loop*, usamos la estructura IF/THEN para probar si el valor $t + dt$ nos lleva más allá del final del intervalo. Si no es así, lo cual comúnmente será el caso al principio, no hacemos nada. De lo contrario, necesitaremos reducir el intervalo haciendo el tamaño de incremento h igual a $tf - t$. Así, garantizamos que el paso siguiente caiga precisamente en tf . Después de hacer este paso final, el *loop* terminará, debido a que $t \geq tf$ será verdadero.

Observe que antes de entrar en el *loop* hemos asignado el valor del incremento, dt , a otra variable, h . Creamos esta variable con el objeto de que nuestra rutina no cambie el valor de dt cuando tengamos que reducir el incremento. Hacemos esto anticipándonos a que tengamos que usar el valor original de dt en algún otro lado, en el caso de que este programa sea parte de otro programa mayor.

³ Este problema se combina con el hecho de que las computadoras usan internamente, para la representación de números, la base 2. En consecuencia, algunos números que aparentemente son divisibles no dan exactamente un entero cuando la división se hace en una computadora. De esto hablaremos en el capítulo 3.

Hay que destacar que este algoritmo aún no está terminado. Puede ser, por ejemplo, que el usuario dé por error un incremento que sea mayor que el intervalo, como por ejemplo, $t_f - t_i = 5$ y $dt = 20$. Entonces, habrá que poner, en el programa, trampas para detectar tales errores y que el usuario pueda corregirlos.

2.3 PROGRAMACIÓN MODULAR

Imaginemos qué difícil sería estudiar un libro que no tuviera capítulos, ni secciones, ni párrafos. Dividir una tarea o una materia complicada en partes más accesibles es una manera de hacerla más fácil. Siguiendo esta misma idea, los programas de computación se dividen en subprogramas más pequeños, o módulos que pueden desarrollarse y probarse por separado. A esta forma de trabajar se le llama *programación modular*.

La principal cualidad de los módulos es que son tan independientes y autosuficientes como sea posible. Además, en general, están diseñados para llevar a cabo una función específica y bien definida, y tienen un punto de entrada y un punto de salida. Los módulos a menudo son cortos (50 a 100 instrucciones) y están bien enfocados.

En los lenguajes estándar de alto nivel como Fortran 90 y C, el principal elemento de programación usado para representar módulos es el procedimiento. Un procedimiento es un conjunto de instrucciones para computadora que juntas realizan una tarea dada. Se emplean comúnmente dos tipos de procedimientos: *funciones* y *subrutinas*. Las primeras normalmente dan un solo resultado, mientras que las últimas dan varios.

Además, hay que mencionar que gran parte de la programación relacionada con paquetes de software como Excel y MATLAB implica el desarrollo de subprogramas. Así, los macros de Excel y las funciones de MATLAB están diseñadas para recibir información, llevar a cabo un cálculo y dar un resultado. De manera que el pensamiento modular también es consistente con la manera en que se programa en ambientes de paquetes.

La programación modular tiene diversas ventajas. El uso de unidades pequeñas e independientes hace que la lógica subyacente sea más fácil de seguir y de entender, tanto para el que desarrolla el módulo como para el usuario. Se facilita el desarrollo debido a que se puede perfeccionar cada módulo por separado. En proyectos grandes, varios programadores pueden trabajar por separado las diferentes partes individuales. En el diseño modular también la depuración y la prueba de un programa se simplifican debido a que los errores se pueden encontrar con facilidad. Por último, es más sencillo el mantenimiento y la modificación del programa. Esto se debe principalmente a que se pueden desarrollar nuevos módulos que desarrollen tareas adicionales e incorporararlos en el esquema coherente y organizado que ya se tiene.

Aunque todas esas ventajas son razones suficientes para usar módulos, la razón más importante, relacionada con la solución de problemas numéricos en ingeniería, es que permiten tener una biblioteca de módulos útiles para posteriores usos en otros programas. Ésta será la filosofía de la presente obra: todos los algoritmos serán presentados como módulos.

El procedimiento anterior se ilustra en la figura 2.7 que muestra una función desarrollada para usar el método de Euler. Observe que esa función y las versiones previas difieren en cómo manipulan la entrada y la salida (input/output). En las versiones anteriores directamente la entrada viene (mediante el INPUT) del usuario, y la salida va (mediante el DISPLAY) al usuario. En la función, se le da la entrada a ésta mediante su lista de argumentos FUNCTION

```

FUNCTION Euler(dt, ti, tf, yi)
  t = ti
  y = yi
  h = dt
  DO
    IF t + dt > tf THEN
      h = tf - t
    ENDIF
    dydt = dy(t, y)
    y = y + dydt * h
    t = t + h
    IF t ≥ tf EXIT
  ENDDO
  Euler = y
END

```

FIGURA 2.7

Seudocódigo para una función que resuelve una ecuación diferencial usando el método de Euler.

Function Euler(dt, ti, tf, yi)

y la salida es regresada mediante una asignación

y = Euler(dt, ti, tf, yi)

Observe, además, lo general que se ha vuelto esta rutina. No se hace para nada referencia al caso específico del paracaidista. Por ejemplo, dentro de la función, en lugar de llamar a la variable dependiente v , de velocidad, se le nombra y , de manera más general. Asimismo, note que la derivada no se calcula mediante una ecuación explícita dentro de la función. En lugar de ello se llama a otra función dy para calcularla, lo cual indica el hecho de que podemos usar esta función en muchos problemas distintos, además de encontrar la velocidad del paracaidista.

2.4 EXCEL

Excel es una hoja de cálculo producida por Microsoft Inc. Las hojas de cálculo son un tipo especial de software para matemáticas que permite al usuario ingresar y realizar cálculos en renglones y columnas de datos. Como tales, son una versión computarizada de una gran hoja de contabilidad en la que se lleva a cabo una gran cantidad de cálculos interrelacionados. Puesto que cuando se modifica un valor de la hoja, hay que actualizar todos los cálculos, las hojas de cálculo son ideales para hacer análisis del tipo “¿y qué pasa si...?”

Excel cuenta con varios recursos numéricos interconstruidos como resolución de ecuaciones, ajuste de curvas y optimización. Incluye también VBA como un lenguaje de macro que sirve para hacer cálculos numéricos. Por último, tiene varias herramientas para la visualización como diagramas y gráficas tridimensionales, que son un valioso complemento para el análisis numérico. En esta sección mostraremos cómo se utilizan estos recursos en la solución del problema del paracaidista.

Para ello, construimos primero una hoja de cálculo sencilla. Como se ve abajo, el primer paso consiste en colocar números y letras o palabras en las celdas de la hoja de cálculo.

	A	B	C	D
1	Problema del paracaidista			
2				
3	m	68.1	kg	
4	cd	12.5	kg/s	
5	dt	0.1	s	
6				
7	t	vnum (m/s)	vanal (m/s)	
8	0	0.000		
9	2			

Antes de escribir un programa de macro para calcular el valor numérico, podemos facilitar el trabajo consecuente dando nombres a los valores de los parámetros. Para esto, seleccione las celdas A3:B5 (la manera más fácil de hacerlo es mover el ratón hasta A3, mantener oprimido el botón izquierdo del ratón y arrastrarlo hasta B5). Después seleccione, del menú,

Insert Name Create Left column OK

Para verificar que todo haya funcionado correctamente, seleccione la celda B3 y verifique que aparezca la etiqueta “m” en la casilla del nombre (casilla que se encuentra en el lado izquierdo de la hoja, justo debajo de las barras del menú).

Muévase hasta la celda C8 e introduzca la solución analítica (ecuación 1.9),

$$=9.8*m/cd*(1-exp(-cd/m*A8))$$

Al introducir esta fórmula debe aparecer el valor 0 en la celda C8. Después copie la fórmula a la celda C9 para obtener 16.405 m/s.

Todo lo anterior es típico del uso estándar de Excel. Hecho esto, podría, por ejemplo, cambiar los valores de los parámetros y observar cómo se modifica la solución analítica.

Ahora mostraremos cómo se usan las macros de VBA para extender los recursos estándar. En la figura 2.8 se da una lista que contiene, para cada una de las estructuras de control dadas en la sección anterior (figuras 2.2 a 2.6), el seudocódigo junto con el código VBA de Excel. Observe que, aunque los detalles difieren, la estructura del seudocódigo y la del código VBA son idénticas.

Ahora podemos usar algunas de las construcciones dadas en la figura 2.8 para escribir una función de macro que calcule la velocidad. Para abrir VBA seleccione⁴

Tools Macro Visual Basic Editor

⁴ ¡La combinación de las teclas Alt-F11 es más rápida!

a) Seudocódigo	b) Excel VBA
IF/THEN: <i>IF condición THEN</i> <i>Bloque verdadero</i> <i>ENDIF</i>	<i>If b <> 0 Then</i> <i>r1 = -c / b</i> <i>End If</i>
IF/THEN/ELSE: <i>IF condición THEN</i> <i>Bloque verdadero</i> <i>ELSE</i> <i>Bloque falso</i> <i>ENDIF</i>	<i>If a < 0 Then</i> <i>b = Sqr(Abs(a))</i> <i>Else</i> <i>b = Sqr(a)</i> <i>End If</i>
IF/THEN/ELSEIF: <i>IF condición₁ THEN</i> <i>Bloque₁</i> <i>ELSEIF condición₂</i> <i>Bloque₂</i> <i>ELSEIF condición₃</i> <i>Bloque₃</i> <i>ELSE</i> <i>Bloque₄</i> <i>ENDIF</i>	<i>If class = 1 Then</i> <i>x = x + 8</i> <i>ElseIf class < 1 Then</i> <i>x = x - 8</i> <i>ElseIf class < 10 Then</i> <i>x = x - 32</i> <i>Else</i> <i>x = x - 64</i> <i>End If</i>
CASE: <i>SELECT CASE Expresión de prueba</i> <i>CASE Valor₁</i> <i>Bloque₁</i> <i>CASE Valor₂</i> <i>Bloque₂</i> <i>CASE Valor₃</i> <i>Bloque₃</i> <i>CASE ELSE</i> <i>Bloque₄</i> <i>END SELECT</i>	<i>Select Case a + b</i> <i>Case Is < -50</i> <i>x = -5</i> <i>Case Is < 0</i> <i>x = -5 - (a + b) / 10</i> <i>Case Is < 50</i> <i>x = (a + b) / 10</i> <i>Case Else</i> <i>x = 5</i> <i>End Select</i>
DOEXIT: <i>DO</i> <i>Bloque₁</i> <i>IF condición EXIT</i> <i>Bloque₂</i> <i>ENDIF</i>	<i>Do</i> <i>i = i + 1</i> <i>If i >= 10 Then Exit Do</i> <i>j = i*x</i> <i>Loop</i>
LOOP CONTROLADO POR CONTADOR: <i>DOFOR i = inicio, fin, incremento</i> <i>Bloque</i> <i>ENDDO</i>	<i>For i = 1 To 10 Step 2</i> <i>x = x + i</i> <i>Next i</i>

FIGURA 2.8
Estructuras de control funda-
mentales en a) seudo-
código y b) VBA de Excel.

Una vez dentro del *Visual Basic Editor* (VBE), seleccione

Insert Module

y se abrirá una nueva ventana para código. La siguiente función en VBA se puede obtener directamente del pseudocódigo de la figura 2.7. Escriba la función dentro de la nueva ventana.

```
Option Explicit
Function Euler(dt, ti, tf, yi, m, cd)
Dim h As Single, t As Single, y As Single, dydt As Single
t = ti
y = yi
h = dt
Do
    If t + dt > tf Then
        h = tf - t
    End If
    dydt = dy(t, y, m, cd)
    y = y + dydt * h
    t = t + h
    If t >= tf Then Exit Do
Loop
Euler = y
End Function
```

Compare esta macro con el pseudocódigo de la figura 2.7 y vea que son muy similares. Observe también cómo la lista de argumentos de la función se hizo más larga al incluir los parámetros necesarios para el modelo de la velocidad del paracaidista. La velocidad obtenida, v , pasa a la hoja de cálculo mediante el nombre de la función.

Note también cómo, para calcular la derivada, hemos usado otra función. Ésta se puede introducir en el mismo módulo tecleándola directamente debajo de la función Euler,

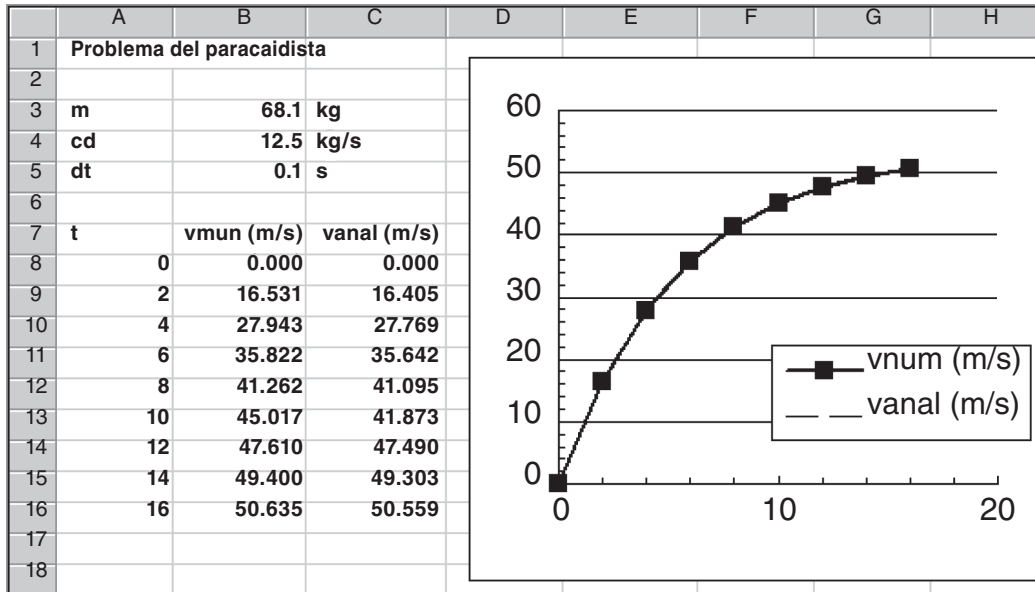
```
Function dy(t, v, m, cd)
Const g As Single = 9.8
dy = g - (cd / m) * v
End Function
```

El paso final consiste en volver a la hoja de cálculo y llamar a la función introduciendo la siguiente expresión en la celda B9.

=Euler(dt,A8,A9,B8,m,cd)

El resultado de la integración numérica, 16.531, aparecerá en la celda B9.

Vamos a ver qué ha pasado aquí. Cuando usted da la función en la celda de la hoja de cálculo, los parámetros pasan al programa VBA, donde se realizan los cálculos y, después, el resultado regresa a la celda. En efecto, el lenguaje de macros VBA le permite usar Excel como mecanismo de entradas y salidas (input/output). Esta característica resulta de mucha utilidad.



Por ejemplo, ahora que ya tiene todos los cálculos, puede jugar con ellos. Suponga que el paracaidista fuera mucho más pesado, digamos, $m = 100$ kg (alrededor de 200 libras). Introduzca 100 en la celda B3 y la hoja de cálculo se modificará de inmediato mostrando el valor 17.438 en la celda B9. Cambie la masa nuevamente a 68.1 kg y el resultado anterior, 16.531 reaparecerá de forma automática en la celda B9.

Ahora vayamos un poco más adelante dando algunos valores más para el tiempo. Introduzca los números 4, 6, ..., 16 en las celdas A10 a A16. Después copie las fórmulas de las celdas B9:C9 hacia abajo en los renglones 10 a 16. Observe cómo el programa VBA calcula correctamente los resultados numéricos en cada uno de los nuevos renglones. (Para verificar esto cambie el valor de dt por 2 y compare los resultados con los cálculos a mano obtenidos anteriormente, en el ejemplo 1.2.) Para mejorar la presentación se pueden graficar los resultados en un plano x - y usando Excel Chart Wizard.

Arriba se muestra la hoja de cálculo resultante. Hemos creado una valiosa herramienta para la solución de problemas. Puede realizar un análisis de sensibilidad cambiando los valores de cada uno de los parámetros. Cada vez que se introduce un nuevo valor, se modificarán automáticamente los cálculos y la gráfica. Tal característica de interactividad es lo que hace tan potente a Excel. No obstante, se debe reconocer que resolver este problema dependerá de la habilidad para escribir el macro en VBA.

La combinación del ambiente de Excel con el lenguaje de programación VBA nos abre un mundo de posibilidades para la solución de problemas en ingeniería. En los capítulos siguientes ilustraremos cómo se logra esto.

2.5 MATLAB

MATLAB es el principal producto de software de Mathworks, Inc., fundada por los analistas numéricos Cleve Moler y John N. Little. Como su nombre lo indica, *MATLAB* se desarrolló originalmente como un laboratorio para matrices. Hoy, el elemento principal

de MATLAB sigue siendo la matriz. La manipulación matemática de matrices se ha realizado muy adecuadamente en un ambiente interactivo fácil de utilizar. A esta manipulación matricial, MATLAB agrega varias funciones numéricas, cálculos simbólicos y herramientas para visualización. En consecuencia, la versión actual representa un ambiente computacional bastante amplio.

MATLAB tiene diferentes funciones y operadores que permiten la adecuada realización de los métodos numéricos que aquí desarrollamos. Éstos se describirán con detalle en los capítulos siguientes. Además, se pueden escribir programas como los llamados archivos M (*m-files*) que sirven para realizar cálculos numéricos. Vamos a explorar cómo funciona.

Primero, usted se dará cuenta de que el uso normal de MATLAB está estrechamente relacionado con la programación. Supongamos, por ejemplo, que queremos determinar la solución analítica al problema del paracaidista, lo cual haríamos con los siguientes comandos de MATLAB

```
>> g=9.8;
>> m=68.1;
>> cd=12.5;
>> tf=2;
>> v=g*m/cd*(1-exp(-cd/m*tf))
```

obteniéndose como resultado

```
v =
    16.4050
```

La secuencia de comandos es como la secuencia de instrucciones en un lenguaje de programación típico.

Pero, ¿qué ocurre si usted se quiere desviar de la estructura secuencial? Aunque hay algunos caminos bien definidos para establecer recursos no secuenciales en el modo estándar de comandos, para introducir decisiones y *loops*, lo mejor es crear un documento de MATLAB al que se le llama archivo-m (*m-file*). Para hacer esto haga clic en

File New Mfile

y se abrirá una ventana nueva con el encabezado “MATLAB Editor/Debugger”. En esta ventana usted puede escribir y editar programas en MATLAB. Escriba ahí el código siguiente:

```
g=9.8;
m=68.1;
cd=12.5;
tf=2;
v=g*m/cd*(1-exp(-cd/m*tf))
```

Obsérvese que los comandos se escriben exactamente en la misma forma en que se haría en el extremo frontal de MATLAB. Guarde el programa con el mismo nombre: analpara. MATLAB agregará en forma automática la extensión .m para denotar que se trata de un archivo M: analpara.m.

Para correr el programa, se debe regresar al modo de comando. La forma más directa de efectuar esto consiste en hacer clic en el botón “MATLAB Command Window”

que se encuentra en la barra de tareas (que por lo general está en la parte inferior de la pantalla).

Ahora, el programa se puede correr al hacer clic en el archivo M, analpara, que debe parecerse a lo siguiente:

```
>> analpara
```

Si usted ha hecho todo en forma correcta, MATLAB debe responder con la respuesta correcta:

```
v =  
    16.4050
```

Ahora, un problema con lo anterior es que está preparado para calcular sólo un caso. El lector lo puede hacer más flexible si hace que el usuario introduzca algunas de las variables. Por ejemplo, suponga que desea evaluar el efecto de la masa sobre la velocidad a los 2 s. Para hacer esto, el archivo M podría reescribirse como sigue:

```
g=9.8;  
m=input('masa (kg): ');  
cd=12.5;  
tf=2;  
v=g*m/cd*(1-exp(-cd/m*tf))
```

Guarde esto con el nombre de analpara2.m. Si escribió analpara2 mientras se encontraba en el modo de comando, la línea mostrará lo que sigue:

```
masa (kg):
```

Entonces, el usuario introduce un valor como 100, y el resultado aparecerá como:

```
v =  
    17.3420
```

Ahora, debe quedar bastante claro cómo se puede programar una solución numérica por medio de un archivo M. A fin de hacerlo, primero debemos entender la manera en que MATLAB maneja las estructuras lógicas y de lazo (ciclos o *loops*). En la figura 2.9 se enlista el pseudocódigo junto con el código de MATLAB para todas las estructuras de control, con base en la sección anterior. Aunque las estructuras del pseudocódigo y el código MATLAB son muy similares, existen algunas diferencias pequeñas que deben destacarse.

En especial, observe cómo hemos expresado la estructura DOEXIT. En lugar del DO usamos el WHILE(1). Como MATLAB interpreta al número 1 como correspondiente a “verdadero”, esta instrucción se repetirá indefinidamente de la misma manera que el DO. El *loop* termina con un comando de *interrupción* (*break*), el cual transfiere el control a la instrucción que se encuentra a continuación, de la instrucción *end* que termina el ciclo.

También hay que observar que los parámetros del lazo controlado por contador están ordenados de modo diferente. Para el pseudocódigo, los parámetros del lazo están

a) Seudocódigo	b) MATLAB
IF/THEN: <i>IF condición THEN</i> <i>Bloque verdadero</i> <i>ENDIF</i>	<code>if b ~= 0</code> <code> r1 = -c / b;</code> <code>end</code>
IF/THEN/ELSE: <i>IF condición THEN</i> <i>Bloque verdadero</i> <i>ELSE</i> <i>Bloque falso</i> <i>ENDIF</i>	<code>if a < 0</code> <code> b = sqrt(abs(a));</code> <code>else</code> <code> b = sqrt(a);</code> <code>end</code>
IF/THEN/ELSEIF: <i>IF condición₁ THEN</i> <i>Bloque₁</i> <i>ELSEIF condición₂</i> <i>Bloque₂</i> <i>ELSEIF condición₃</i> <i>Bloque₃</i> <i>ELSE</i> <i>Bloque₄</i> <i>ENDIF</i>	<code>if class == 1</code> <code> x = x + 8;</code> <code>elseif class < 1</code> <code> x = x - 8;</code> <code>elseif class < 10</code> <code> x = x - 32;</code> <code>else</code> <code> x = x - 64;</code> <code>end</code>
CASE: <i>SELECT CASE Expresión de prueba</i> <i>CASE Valor₁</i> <i>Bloque₁</i> <i>CASE Valor₂</i> <i>Bloque₂</i> <i>CASE Valor₃</i> <i>Bloque₃</i> <i>CASE ELSE</i> <i>Bloque₄</i> <i>END SELECT</i>	<code>switch a + b</code> <code> case 1</code> <code> x = -5;</code> <code> case 2</code> <code> x = -5 - (a + b) / 10;</code> <code> case 3</code> <code> x = (a + b) / 10;</code> <code> otherwise</code> <code> x = 5;</code> <code>end</code>
DOEXIT: <i>DO</i> <i>Bloque₁</i> <i>IF condición EXIT</i> <i>Bloque₂</i> <i>ENDIF</i>	<code>while (1)</code> <code> i = i + 1;</code> <code> if i >= 10, break, end</code> <code> j = i*x;</code> <code>end</code>
LOOP CONTROLADO POR CONTADOR: <i>DOFOR i = inicio, fin, incremento</i> <i>Bloque</i> <i>ENDO</i>	<code>for i = 1:10:2</code> <code> x = x + i;</code> <code>end</code>

FIGURA 2.9

Estructuras de control fundamentales en a) pseudocódigo y b) lenguaje de programación en MATLAB.

especificados como *start*, *finish*, *step*. Para MATLAB, los parámetros están ordenados como *start:step:finish*.

Ahora el siguiente archivo-m de MATLAB se puede desarrollar directamente, a partir del pseudocódigo dado en la figura 2.7. Escriba lo siguiente en el Editor/Debugger de MATLAB:

```
g=9.8;
m=input('mass (kg):');
cd=12.5;
ti=0;
tf=2;
vi=0;
dt=0.1;
t = ti;
v = vi;
h = dt;
while (1)
    if t + dt > tf
        h = tf - t;
    end
    dvdt = g - (cd / m) * v;
    v = v + dvdt * h;
    t = t + h;
    if t >= tf, break, end
end
disp('velocity (m/s):')
disp(v)
```

Guarde este archivo como *numpara.m*, vuelva al modo de comandos y córralo dando *numpara*. Obtendrá la siguiente salida:

```
masa (kg): 100

velocity (m/s):
17.4381
```

Por último vamos a convertir este archivo-m en una función. Esto se puede hacer en el siguiente archivo-m basado en el pseudocódigo de la figura 2.7:

```
function euler = f(dt,ti,tf,yi,m,cd)
t = ti;
y = yi;
h = dt;
while (1)
    if t + dt > tf
        h = tf - t;
    end
    dydt = dy(t, y, m, cd);
    y = y + dydt * h;
    t = t + h;
    if t >= tf, break, end
end
YY = y;
```

Guarde este archivo como euler.m y después cree otro archivo-m para calcular la derivada,

```
function dydt = dy(t, v, m, cd)
g = 9.8;
dydt = g - (cd / m) * v;
```

Guarde este archivo como dy.m y regrese al modo de comandos. Para llamar la función y ver el resultado, teclee los siguientes comandos

```
>> m=68.1;
>> cd=12.5;
>> ti=0;
>> tf=2.;
>> vi=0;
>> dt=0.1;
>> euler(dt,ti,tf,vi,m,cd)
```

Una vez dado el último comando, se desplegará el resultado

```
ans =
16.5309
```

La combinación del ambiente de MATLAB con el lenguaje de programación para los archivos-m nos abre un mundo de posibilidades para la solución de problemas en ingeniería. En el siguiente capítulo veremos cómo se hace esto.

2.6 OTROS LENGUAJES Y BIBLIOTECAS

En la sección anterior mostramos cómo se escribe una función en Excel o MATLAB, para el método de Euler, a partir de un algoritmo expresado en pseudocódigo. Funciones semejantes se escriben en los lenguajes de alto nivel como Fortran 90 y C++. Por ejemplo, una función en Fortran 90 para el método de Euler es

```
Function Euler(dt, ti, tf, yi, m, cd)

REAL dt, ti, tf, yi, m, cd
Real h, t, y, dydt

t = ti
y = yi
h = dt
Do
  If (t + dt > tf) Then
    h = tf - t
  End If
  dydt = dy(t, y, m, cd)
  y = y + dydt * h
  t = t + h
  If (t >= tf) Exit
End Do
```

```
Euler = y
End Function
```

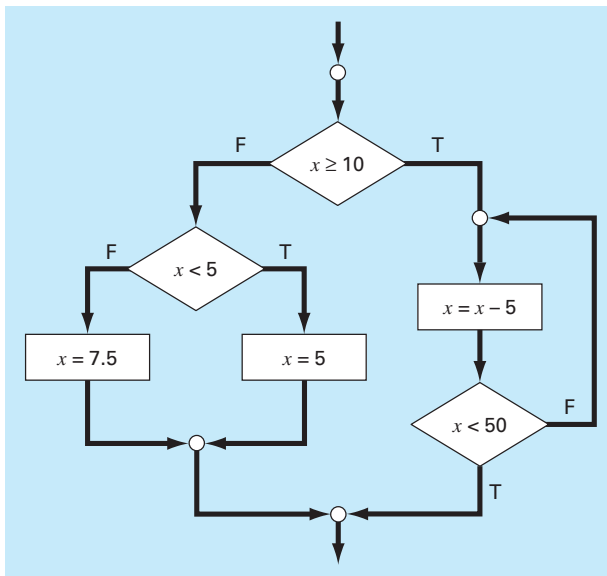
En C el resultado sería bastante similar a la función escrita en MATLAB. El punto es que una vez que se ha desarrollado bien un algoritmo estructurado en pseudocódigo, es fácil implementarlo en diversos ambientes de programación.

En este libro daremos al lector procedimientos bien estructurados escritos en pseudocódigo. Esta colección de algoritmos constituirá una biblioteca numérica, que se puede usar para realizar tareas numéricas específicas con diversas herramientas de software y lenguajes de programación.

Además de tener sus propios programas, usted debe recordar que las bibliotecas comerciales de programación tienen muchos procedimientos numéricos útiles. Por ejemplo, la biblioteca *Numerical Recipe* contiene una gran variedad de algoritmos escritos en Fortran y C.⁵ Estos procedimientos se describen tanto en libros (por ejemplo, Press *et al.*, 1992) como en forma electrónica.

En Fortran, la *IMSL* (International Mathematical and Statistical Library) ofrece más de 700 procedimientos que comprenden todas las áreas numéricas cubiertas en este libro. Dada la amplia divulgación de Fortran en la ingeniería, incluimos algunas aplicaciones de IMSL.

PROBLEMAS



2.1 Escriba el pseudocódigo para implementar el diagrama de flujo que se ilustra en la figura P2.1. Asegúrese de incluir la indentación apropiada para que la estructura sea clara.

2.2 Vuelva a escribir el pseudocódigo siguiente, con el uso de la indentación apropiada.

```
DO
  i = i + 1
  IF z > 50 EXIT
  x = x + 5
  IF x > 5 THEN
    y = x
  ELSE
    y = 0
  ENDIF
  z = x + y
ENDDO
```

⁵ Los procedimientos Numerical Recipe también están disponibles en libro y en formato electrónico para Pascal, MS BASIC y MATLAB. En <http://www.nr.com> se puede encontrar la información sobre todos los productos Numerical Recipe.

Figura P2.1

2.3 En cada una de las tarjetas de un conjunto de cartas índice, se registra un valor para la concentración de un contaminante en un lago. Al final del conjunto, se coloca una carta marcada como “fin de los datos”. Escriba un algoritmo para determinar la suma, el promedio y el máximo de dichos valores.

2.4 Escriba un diagrama de flujo estructurado para el problema 2.3.

2.5 Desarrolle, depure y documente un programa para determinar las raíces de una ecuación cuadrática, $ax^2 + bx + c$, en cualquier lenguaje de alto nivel, o de macros, de su elección. Utilice un procedimiento de subrutina para calcular las raíces (sean reales o complejas). Ejecute corridas de prueba para los casos en que a) $a = 1, b = 6, c = 2$; b) $a = 0, b = -4, c = 1.6$; c) $a = 3, b = 2.5, c = 7$.

2.6 La función coseno puede evaluarse por medio de la serie infinita siguiente:

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots$$

Escriba un algoritmo para implementar esta fórmula de modo que calcule e imprima los valores de $\cos x$ conforme se agregue cada término de la serie. En otras palabras, calcule e imprima la secuencia de valores para

$$\cos x = 1$$

$$\cos x = 1 - \frac{x^2}{2!}$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!}$$

hasta el término de orden n que usted elija. Para cada uno de los valores anteriores, calcule y haga que se muestre el error porcentual relativo:

$$\% \text{ error} = \frac{\text{valor verdadero} - \text{aproximación con la serie}}{\text{valor verdadero}} \times 100\%$$

2.7 Escriba el algoritmo para el problema 2.6 en forma de a) diagrama de flujo estructurado, y b) pseudocódigo.

2.8 Desarrolle, depure y documente un programa para el problema 2.6 en cualquier lenguaje de alto nivel o de macros, de su elección. Emplee la función coseno de la biblioteca de su computadora para determinar el valor verdadero. Haga que el programa imprima en cada paso la serie de aproximación y el error. Como caso de prueba, utilice el programa para calcular valores desde $\cos(1.25)$ hasta incluir el término $x^{10}/10!$. Interprete los resultados.

2.9 El algoritmo siguiente está diseñado para determinar la calificación de un curso que consiste en cuestionarios, tareas y un examen final:

Paso 1: Introducir la clave y nombre del curso.

Paso 2: Introducir factores de ponderación para los cuestionarios (C), tareas (T) y examen final (E).

Paso 3: Introducir las calificaciones de las preguntas y determinar su promedio (PC).

Paso 4: Introducir las calificaciones de las tareas y determinar su promedio (PT).

Paso 5: Si el curso tiene una calificación final, continuar con el paso 6. Si no, ir al paso 9.

Paso 6: Introducir la calificación del examen final, (F).

Paso 7: Determinar la calificación promedio, CP, de acuerdo con

$$CP = \frac{(C \times PC + T \times PT + E \times F)}{(C + T + E)} \times 100\%$$

Paso 8: Ir al paso 10.

Paso 9: Determinar la calificación promedio, CP, de acuerdo con

$$CP = \frac{(C \times PC + T \times PT)}{(C + T)} \times 100\%$$

Paso 10: Imprimir la clave y nombre del curso, y la calificación promedio.

Paso 11: Finalizar el cálculo.

a) Escriba un pseudocódigo bien estructurado para implementar este algoritmo.

b) Escriba, depure y documente un programa estructurado de computadora basado en este algoritmo. Pruébelo con los datos siguientes para calcular una calificación sin el examen final, y otra con éste. $C = 35$; $T = 30$; $E = 35$; cuestionario = 98, 85, 90, 65 y 99; tareas = 95, 90, 87, 100, 92 y 77; y examen final = 92.

2.10 El método antiguo de *dividir y promediar*, para obtener el valor aproximado de la raíz cuadrada de cualquier número positivo a se puede formular como

$$x = \frac{x + a/x}{2}$$

a) Escriba un pseudocódigo bien estructurado para implementar este algoritmo como se ilustra en la figura P2.10. Utilice la indentación apropiada para que la estructura sea clara.

b) Desarrolle, depure y documente un programa para implementar esta ecuación en cualquier lenguaje de alto nivel, o de macros, de su elección. Estructure su código de acuerdo con la figura P2.10.

2.11 Se invierte cierta cantidad de dinero en una cuenta en la que el interés se capitaliza al final del periodo. Debe determinarse el valor futuro, F , que se obtiene con cierta tasa de interés, i , después de n periodos, por medio de la fórmula siguiente:

$$F = P(1 + i)^n$$

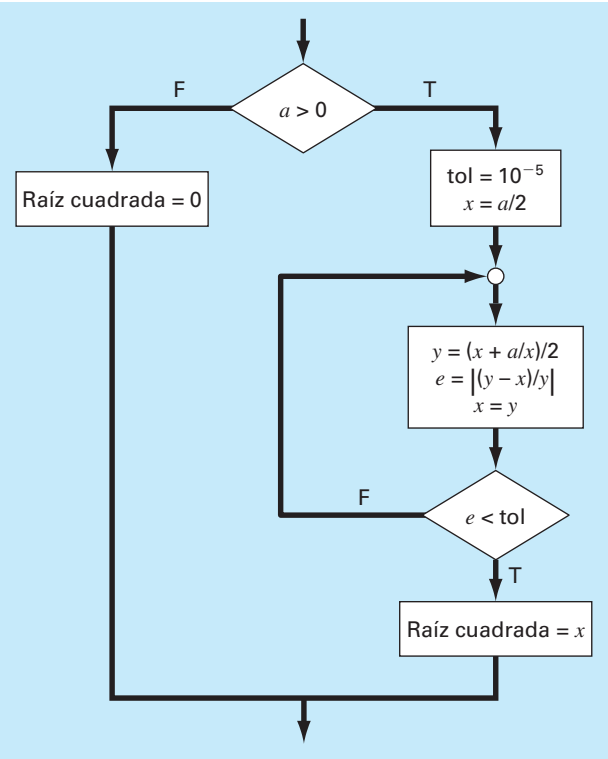


Figura P2.10

Escriba un programa que calcule el valor futuro de una inversión para cada año, desde 1 hasta n . La entrada para la función debe incluir la inversión inicial, P , la tasa de interés, i (en forma decimal), y el número de años, n , para el que ha de calcularse el valor futuro. La salida debe consistir en una tabla con encabezados y columnas para n y F . Corra el programa para $P = \$100\,000$, $i = 0.06$, y $n = 5$ años.

2.12 Las fórmulas económicas están disponibles para calcular los pagos anuales de préstamos. Suponga que obtiene en préstamo cierta cantidad de dinero P y acuerda devolverla en n pagos anuales con una tasa de interés de i . La fórmula para calcular el pago anual A es:

$$A = P \frac{i(1+i)^n}{(1+i)^n - 1}$$

Escriba un programa para calcular A . Pruébelo con $P = \$55\,000$ y una tasa de interés de 6.6% ($i = 0.066$). Calcule los resultados para $n = 1, 2, 3, 4$ y 5 , y muestre los resultados en forma de tabla con encabezados y columnas para n y A .

2.13 La temperatura promedio diaria para cierta área se aproxima por medio de la función siguiente,

$$T = T_{\text{media}} + (T_{\text{máxima}} - T_{\text{media}}) \cos(w(t - t_{\text{máxima}}))$$

donde T_{media} = temperatura promedio anual, $t_{\text{máxima}}$ = temperatura máxima, w = frecuencia de la variación anual ($= 2\pi/365$), y $t_{\text{máxima}}$ = día de la temperatura máxima ($\cong 205$ d). Desarrolle un programa que calcule la temperatura promedio entre dos días del año para una ciudad en particular. Pruébelo para a) enero-febrero ($t = 0$ a 59) en Miami, Florida ($T_{\text{media}} = 22.1^\circ\text{C}$; $T_{\text{máxima}} = 28.3^\circ\text{C}$), y b) julio-agosto ($t = 180$ a 242) en Boston, Massachusetts ($T_{\text{media}} = 10.7^\circ\text{C}$; $T_{\text{máxima}} = 22.9^\circ\text{C}$).

2.14 Desarrolle, depure y pruebe un programa en cualquier lenguaje de alto nivel, o de macros, de su elección, a fin de calcular la velocidad del paracaídas que cae como se explicó en el ejemplo 1.2. Diseñe el programa de modo que permita al usuario introducir valores para el coeficiente de arrastre y la masa. Pruebe el programa con la reproducción de los resultados del ejemplo 1.2. Repita el cálculo pero utilice tamaños de paso de 1 y 0.5 s. Compare sus resultados con la solución analítica que se obtuvo previamente, en el Ejemplo 1.1. Un tamaño de paso más pequeño, ¿hace que los resultados sean mejores o peores? Explique sus resultados.

2.15 El método de la burbuja es una técnica de ordenamiento ineficiente pero fácil de programar. La idea que subyace al ordenamiento consiste en avanzar hacia abajo a través de un arreglo, comparar los pares adyacentes e intercambiar los valores si no están en orden. Para que este método ordene por completo un arreglo, es necesario que lo recorra muchas veces. Conforme se avanza para un ordenamiento en orden ascendente, los elementos más pequeños del arreglo parecen ascender como burbujas. Eventualmente, habrá un paso por el arreglo que ya no requiera intercambios. En ese momento, el arreglo estará ordenado. Después del primer paso, el valor más grande cae directamente hasta el fondo. En consecuencia, el segundo paso sólo tiene que proceder del segundo al último valor, y así sucesivamente. Desarrolle un programa que tome un arreglo de 20 números al azar y los ordene en forma ascendente con la técnica de la burbuja (véase la figura P2.15).

2.16 En la figura P2.16 se muestra un tanque cilíndrico con base cónica. Si el nivel del líquido está muy bajo en la parte cónica, el volumen simplemente es el volumen del cono de líquido. Si el nivel del líquido está entre la parte cilíndrica, el volumen total de líquido incluye la parte cónica llena y la parte cilíndrica parcialmente llena. Escriba un procedimiento bien estructurado de función para calcular el volumen del tanque como función de los valores dados de R y d . Utilice estructuras de control de decisiones (como If/Then, Elseif, Else, End If). Diseñe la función de modo que produzca el volumen en todos los casos en los que la profundidad sea menor que $3R$. Genere un mensaje de error ("Sobrepasado") si se rebasa la altura del tanque, es decir, $d > 3R$. Pruébelo con los datos siguientes:

R	1	1	1	1
d	0.5	1.2	3.0	3.1

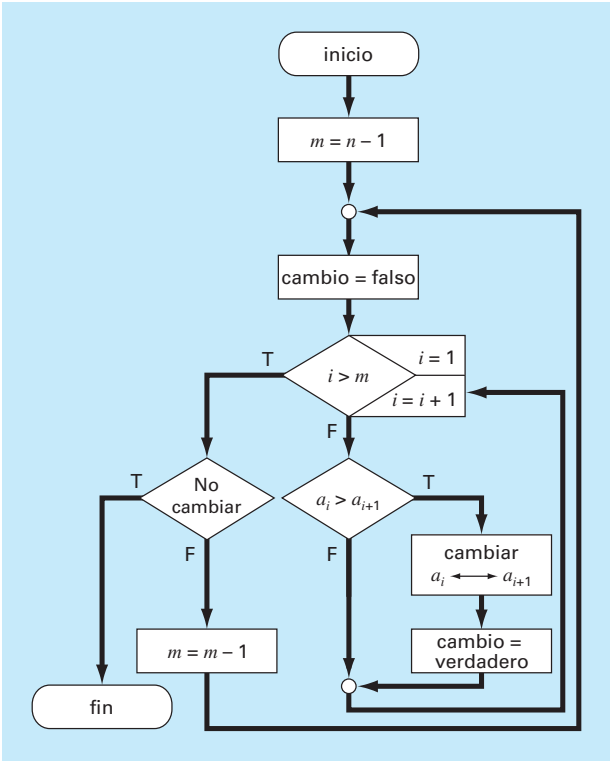


Figura P2.15

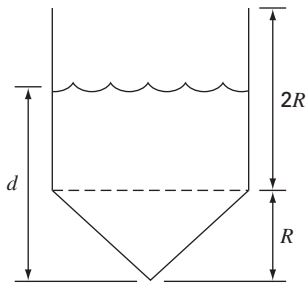


Figura P2.16

2.17 Se requieren dos distancias para especificar la ubicación de un punto en relación con el origen en un espacio de dos dimensiones (Véase la figura P2.17):

- Las distancias horizontal y vertical (x, y) en coordenadas cartesianas.
- El radio y el ángulo (r, θ) en coordenadas radiales.

Es relativamente fácil calcular las coordenadas cartesianas (x, y) sobre la base de las coordenadas polares (r, θ). El proceso inverso no es tan simple. El radio se calcula con la fórmula que sigue:

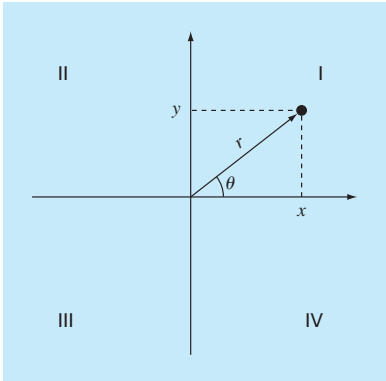


Figura P2.17

$$r = \sqrt{x^2 + y^2}$$

Si las coordenadas quedan dentro del primer o cuarto cuadrante (p. ej., $x > 0$), entonces se emplea una fórmula sencilla para el cálculo de θ :

$$\theta = \tan^{-1}\left(\frac{y}{x}\right)$$

La dificultad surge en los demás casos. La tabla siguiente resume las posibilidades:

x	y	θ
<0	>0	$\tan^{-1}(y/x) + \pi$
<0	<0	$\tan^{-1}(y/x) - \pi$
<0	$=0$	π
$=0$	>0	$\pi/2$
$=0$	<0	$-\pi/2$
$=0$	$=0$	0

- Escriba un diagrama de flujo bien estructurado para un procedimiento de subrutina a fin de calcular r y θ como función de x y y . Expresé los resultados finales para θ , en grados.
- Escriba una procedimiento bien estructurado de función con base en el diagrama de flujo. Pruebe el programa de modo que se llene la tabla que sigue:

x	y	r	θ
1	0		
1	1		
0	1		
-1	1		
-1	0		
-1	-1		
0	-1		
1	-1		
0	0		

2.18 Desarrolle un procedimiento bien estructurado de función que lea una calificación numérica entre 0 y 100 y devuelva una letra, de acuerdo con el esquema siguiente:

Letra	Criterio
A	$90 \leq \text{calificación numérica} \leq 100$
B	$80 \leq \text{calificación numérica} < 90$
C	$70 \leq \text{calificación numérica} < 80$
D	$60 \leq \text{calificación numérica} < 70$
F	$\text{calificación numérica} < 60$

2.19 Desarrolle un procedimiento bien estructurado de función para determinar *a)* el factorial de un número; *b)* el valor más pequeño de un vector, y *c)* el promedio de los valores de un vector.

2.20 Desarrolle programas bien estructurados para *a)* determinar la raíz cuadrada de la suma de los cuadrados de los elementos de un arreglo bidimensional (p. ej., una matriz), y *b)* normalizar una matriz por medio de dividir cada renglón entre el valor absoluto máximo en el renglón de modo que el elemento mayor en cada renglón sea 1.

CAPÍTULO 3

Aproximaciones y errores de redondeo

A causa de que la mayor parte de los métodos expuestos en este libro son muy sencillos en su descripción y en sus aplicaciones, en este momento resulta tentador ir directamente al cuerpo principal del texto y averiguar el empleo de dichas técnicas. Sin embargo, entender el concepto de error es tan importante para utilizar en forma efectiva los métodos numéricos que los dos siguientes capítulos se eligieron para tratar el tema.

La importancia de los errores se mencionó por primera vez en el análisis de la caída del paracaidista en el capítulo 1. Recuerde que la velocidad de caída del paracaidista se determinó por métodos analíticos y numéricos. Aunque con la técnica numérica se obtuvo una aproximación a la solución analítica exacta, hubo cierta discrepancia o *error*, debido a que los métodos numéricos dan sólo una aproximación. En realidad fuimos afortunados en este caso porque teníamos la solución analítica que nos permitía calcular el error en forma exacta. Pero en muchos problemas de aplicación en ingeniería no es posible obtener la solución analítica; por lo tanto, no se pueden calcular con exactitud los errores en nuestros métodos numéricos. En tales casos debemos usar aproximaciones o estimaciones de los errores.

La mayor parte de las técnicas desarrolladas en este libro tienen la característica de poseer errores. En primera instancia, esto puede parecer contradictorio, ya que no coincide con la imagen que se tiene de una buena ingeniería. Los estudiantes y los practicantes de la ingeniería trabajan constantemente para limitar este tipo de errores en sus actividades. Cuando hacen un examen o realizan sus tareas, son sancionados, mas no premiados por sus errores. En la práctica profesional, los errores llegan a resultar costosos y, en algunas ocasiones, catastróficos. Si una estructura o un dispositivo falla, esto puede costar vidas.

Aunque la perfección es una meta digna de alabarse, es difícil, si no imposible, alcanzarla. Por ejemplo, a pesar de que el modelo obtenido mediante la segunda ley de Newton es una aproximación excelente, en la práctica jamás predecirá con exactitud la caída del paracaidista. Fenómenos tales como la velocidad del viento y alguna ligera variación de la resistencia del aire desviarían la predicción. Si tales desviaciones son sistemáticamente grandes o pequeñas, habría entonces que formular un nuevo modelo. No obstante, si su distribución es aleatoria y se agrupan muy cerca de la predicción, entonces las desviaciones se considerarían insignificantes y el modelo parecerá adecuado. Las aproximaciones numéricas también presentan discrepancias similares en el análisis. De nuevo, las preguntas son: ¿qué tanto error se presenta en los cálculos? y ¿es tolerable?

Este capítulo y el siguiente cubren aspectos básicos relacionados con la identificación, cuantificación y minimización de dichos errores. En las primeras secciones se revisa la información referente a la cuantificación de los errores. En seguida, se estudia uno de

los dos errores numéricos más comunes: errores de redondeo. Los *errores de redondeo* se deben a que la computadora tan sólo representa cantidades con un número finito de dígitos. En el siguiente capítulo nos ocuparemos de otra clase importante de error: el de truncamiento. Los *errores de truncamiento* representan la diferencia entre una formulación matemática exacta de un problema y su aproximación obtenida por un método numérico. Por último, se analizan los errores que no están relacionados directamente con el método numérico en sí. Éstos son equivocaciones, errores de formulación o del modelo, y la incertidumbre en la obtención de los datos, entre otros.

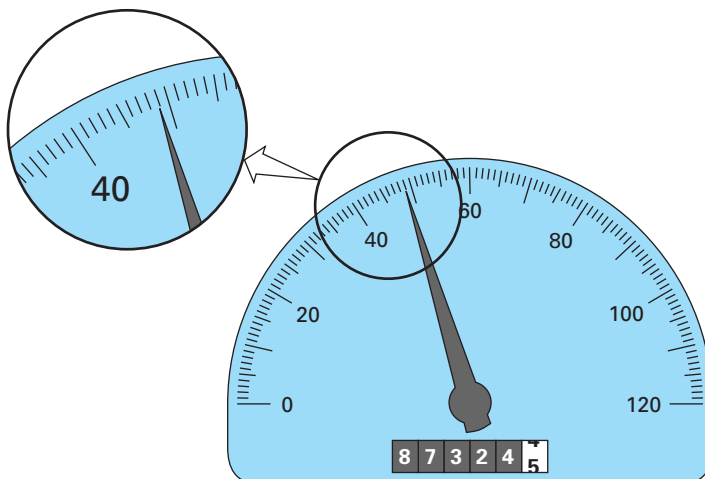
3.1 CIFRAS SIGNIFICATIVAS

En esta obra se trata de manera extensa con aproximaciones que se relacionan con el manejo de números. En consecuencia, antes de analizar los errores asociados con los métodos numéricos, es útil repasar algunos conceptos básicos referentes a la representación aproximada de los números mismos.

Cuando se emplea un número para realizar un cálculo, debe haber seguridad de que pueda usarse con confianza. Por ejemplo, la figura 3.1 muestra un velocímetro y un odómetro (contador de kilometraje) de un automóvil. Con un simple vistazo al velocímetro se observa que el vehículo viaja a una velocidad comprendida entre 48 y 49 km/h. Como la aguja está más allá de la mitad entre las marcas del indicador, es posible asegurar que el automóvil viaja aproximadamente a 49 km/h. Tenemos confianza en este resultado, ya que dos o más individuos que hicieran esta lectura llegarían a la misma conclusión. Sin embargo, supongamos que se desea obtener una cifra decimal en la estimación de la velocidad. En tal caso, alguien podría decir 48.8, mientras que otra persona podría decir 48.9 km/h. Por lo tanto, debido a los límites del instrumento,

FIGURA 3.1

El velocímetro y el odómetro de un automóvil ejemplifican el concepto de cifras significativas.



únicamente se emplean con confianza los dos primeros dígitos. Para estimaciones del tercer dígito (o más allá) sólo se considerarían aproximaciones. Sería ridículo afirmar, considerando el velocímetro de la figura, que el automóvil viaja a 48.8642138 km/h. En contraste, el odómetro muestra hasta seis dígitos confiables. De la figura 3.1 se concluye que el automóvil ha recorrido un poco menos de 87 324.5 km durante su uso. Aquí el séptimo dígito (y los siguientes) resultan inciertos.

El concepto de cifras o dígitos significativos se ha desarrollado para designar formalmente la confiabilidad de un valor numérico. Las *cifras significativas* de un número son aquellas que pueden utilizarse en forma confiable. Se trata del número de dígitos que se ofrecen con certeza, más uno estimado. Por ejemplo, el velocímetro y el odómetro de la figura 3.1 muestran lecturas de hasta tres y siete cifras significativas, respectivamente. Para el velocímetro, los dos dígitos seguros son 48. Por convención al dígito estimado se le da el valor de la mitad de la escala menor de división en el instrumento de medición. Así, la lectura del velocímetro consistirá de las tres cifras significativas: 48.5. En forma similar, el odómetro dará una lectura con siete cifras significativas, 87 324.45.

Aunque, por lo común, determinar las cifras significativas de un número es un procedimiento sencillo, en algunos casos genera cierta confusión. Por ejemplo, los ceros no siempre son cifras significativas, ya que pueden usarse sólo para ubicar el punto decimal: los números 0.00001845, 0.0001845 y 0.001845 tienen cuatro cifras significativas. Asimismo, cuando se incluye ceros en números muy grandes, no queda claro cuántos son significativos. Por ejemplo, el número 45 300 puede tener tres, cuatro o cinco dígitos significativos, dependiendo de si los ceros se conocen o no con exactitud. La incertidumbre se puede eliminar utilizando la notación científica, donde 4.53×10^4 , 4.530×10^4 , 4.5300×10^4 muestran, respectivamente, que el número tiene tres, cuatro y cinco cifras significativas.

El concepto de cifras significativas tiene dos implicaciones importantes en el estudio de los métodos numéricos.

1. Como se mencionó en el problema de la caída del paracaidista, los métodos numéricos dan resultados aproximados. Por lo tanto, se deben desarrollar criterios para especificar qué tan confiables son dichos resultados. Una manera de hacerlo es en términos de cifras significativas. Por ejemplo, es posible afirmar que la aproximación es aceptable siempre y cuando sea correcta con cuatro cifras significativas.
2. Aunque ciertas cantidades tales como π , e , o $\sqrt{7}$ representan cantidades específicas, no se pueden expresar exactamente con un número finito de dígitos. Por ejemplo,

$$\pi = 3.141592653589793238462643...$$

hasta el infinito. Como las computadoras retienen sólo un número finito de cifras significativas, tales números jamás se podrán representar con exactitud. A la omisión del resto de cifras significativas se le conoce como *error de redondeo*.

Los errores de redondeo y el uso de cifras significativas para expresar nuestra confianza en un resultado numérico se estudiarán con mayor detalle en las siguientes secciones. Además, el concepto de cifras significativas tendrá mucha importancia en la definición de exactitud y de precisión en la siguiente sección.

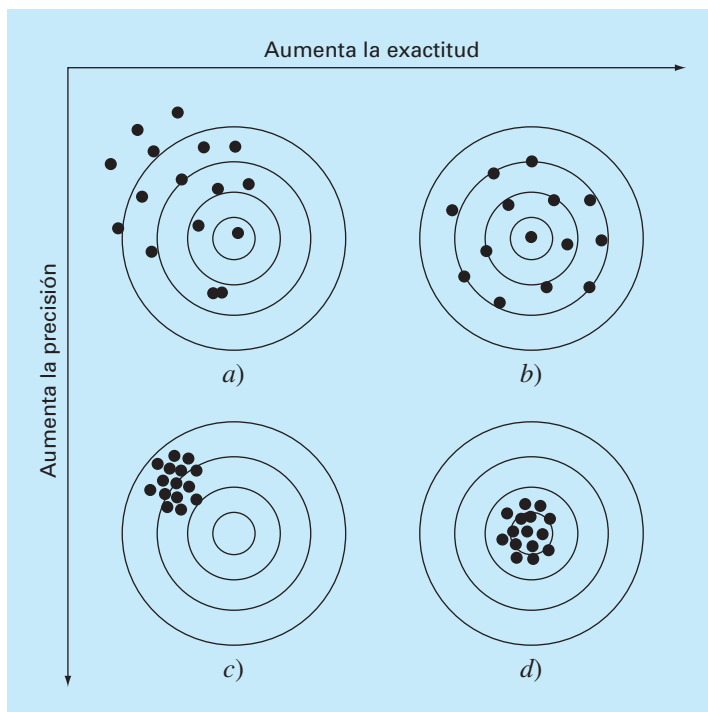
3.2 EXACTITUD Y PRECISIÓN

Los errores en cálculos y medidas se pueden caracterizar con respecto a su exactitud y su precisión. La *exactitud* se refiere a qué tan cercano está el valor calculado o medido del valor verdadero. La *precisión* se refiere a qué tan cercanos se encuentran, unos de otros, diversos valores calculados o medidos.

Estos conceptos se ilustran gráficamente utilizando la analogía con una diana en la práctica de tiro. Los agujeros en cada blanco de la figura 3.2 se consideran como las predicciones con una técnica numérica; mientras que el centro del blanco representa la verdad. La *inexactitud* (conocida también como *sesgo*) se define como una desviación sistemática del valor verdadero. Por lo tanto, aunque los disparos en la figura 3.2c están más juntos que los de la figura 3.2a, los dos casos son igualmente inexactos, ya que ambos se centran en la esquina superior izquierda del blanco. La *imprecisión* (también llamada *incertidumbre*), por otro lado, se refiere a la magnitud en la dispersión de los disparos. Por consiguiente, aunque las figuras 3.2b y 3.2d son igualmente exactas (esto es, igualmente centradas respecto al blanco), la última es más precisa, pues los disparos están agrupados en forma más compacta.

FIGURA 3.2

Un ejemplo de puntería ilustra los conceptos de exactitud y precisión. a) Inexacto e impreciso; b) exacto e impreciso; c) inexacto y preciso; d) exacto y preciso.



Los métodos numéricos deben ser lo suficientemente exactos o sin sesgo para satisfacer los requisitos de un problema particular de ingeniería. También deben ser suficientemente precisos para ser adecuados en el diseño de la ingeniería. En este libro se usa el término *error* para representar tanto la inexactitud como la imprecisión en las predicciones. Con dichos conceptos como antecedentes, ahora analizaremos los factores que contribuyen al error en los cálculos numéricos.

3.3 DEFINICIONES DE ERROR

Los errores numéricos surgen del uso de aproximaciones para representar operaciones y cantidades matemáticas exactas. Éstas incluyen los *errores de truncamiento* que resultan del empleo de aproximaciones como un procedimiento matemático exacto, y los *errores de redondeo* que se producen cuando se usan números que tienen un límite de cifras significativas para representar números exactos. Para ambos tipos de errores, la relación entre el resultado exacto, o verdadero, y el aproximado está dada por

$$\text{Valor verdadero} = \text{Valor aproximado} + \text{error} \quad (3.1)$$

Reordenando la ecuación (3.1) se encuentra que el error numérico es igual a la diferencia entre el valor verdadero y el valor aproximado, es decir

$$E_t = \text{valor verdadero} - \text{valor aproximado} \quad (3.2)$$

donde E_t se usa para denotar el valor exacto del error. El subíndice t indica que se trata del error “verdadero” (true). Como ya se mencionó brevemente, esto contrasta con los otros casos, donde se debe emplear una estimación “aproximada” del error.

Una desventaja en esta definición es que no toma en consideración el orden de la magnitud del valor que se estima. Por ejemplo, un error de un centímetro es mucho más significativo si se está midiendo un remache en lugar de un puente. Una manera de tomar en cuenta las magnitudes de las cantidades que se evalúan consiste en normalizar el error respecto al valor verdadero, es decir

$$\text{Error relativo fraccional verdadero} = \frac{\text{error verdadero}}{\text{valor verdadero}}$$

donde, como ya se mencionó en la ecuación (3.2), error = valor verdadero – valor aproximado. El error relativo también se puede multiplicar por 100% para expresarlo como

$$\varepsilon_t = \frac{\text{error verdadero}}{\text{valor verdadero}} 100\% \quad (3.3)$$

donde ε_t denota el error relativo porcentual verdadero.

EJEMPLO 3.1 Cálculo de errores

Planteamiento del problema. Suponga que se tiene que medir la longitud de un puente y la de un remache, y se obtiene 9999 y 9 cm, respectivamente. Si los valores verdaderos son 10000 y 10 cm, calcule *a)* el error verdadero y *b)* el error relativo porcentual verdadero en cada caso.

Solución

- a) El error en la medición del puente es [ecuación (3.2)]

$$E_t = 10\,000 - 9\,999 = 1 \text{ cm}$$

y en la del remache es de

$$E_t = 10 - 9 = 1 \text{ cm}$$

- b) El error relativo porcentual para el puente es [ecuación (3.3)]

$$\varepsilon_t = \frac{1}{10\,000} 100\% = 0.01\%$$

y para el remache es de

$$\varepsilon_t = \frac{1}{10} 100\% = 10\%$$

Por lo tanto, aunque ambas medidas tienen un error de 1 cm, el error relativo porcentual del remache es mucho mayor. Se concluye entonces que se ha hecho un buen trabajo en la medición del puente; mientras que la estimación para el remache dejó mucho que desear.

Observe que en las ecuaciones (3.2) y (3.3), E y ε tienen un subíndice t que significa que el error ha sido normalizado al valor verdadero. En el ejemplo 3.1 teníamos el valor verdadero. Sin embargo, en las situaciones reales a veces es difícil contar con tal información. En los métodos numéricos, el valor verdadero sólo se conocerá cuando se tengan funciones que se resuelvan analíticamente. Éste comúnmente será el caso cuando se estudie el comportamiento teórico de una técnica específica para sistemas simples. Sin embargo, en muchas aplicaciones reales, no se conoce *a priori* la respuesta verdadera. Entonces en dichos casos, una alternativa es normalizar el error, usando la mejor estimación posible al valor verdadero; es decir, para la aproximación misma, como en

$$\varepsilon_a = \frac{\text{error aproximado}}{\text{valor aproximado}} 100\% \quad (3.4)$$

donde el subíndice a significa que el error está normalizado a un valor aproximado. Observe también que en aplicaciones reales la ecuación (3.2) no se puede usar para calcular el término del error de la ecuación (3.4). Uno de los retos que enfrentan los métodos numéricos es el de determinar estimaciones del error en ausencia del conocimiento de los valores verdaderos. Por ejemplo, ciertos métodos numéricos usan un *método iterativo* para calcular los resultados. En tales métodos se hace una aproximación considerando la aproximación anterior. Este proceso se efectúa varias veces, o de forma iterativa, para calcular en forma sucesiva, esperando cada vez mejores aproximaciones. En tales casos, el error a menudo se calcula como la diferencia entre la aproximación previa y la actual. Por lo tanto, el error relativo porcentual está dado por

$$\varepsilon_a = \frac{\text{aproximación actual} - \text{aproximación anterior}}{\text{aproximación actual}} 100\% \quad (3.5)$$

En capítulos posteriores se explicarán con detalle éste y otros métodos para expresar errores.

Los signos de las ecuaciones (3.2) a (3.5) pueden ser positivos o negativos. Si la aproximación es mayor que el valor verdadero (o la aproximación previa es mayor que la aproximación actual), el error es negativo; si la aproximación es menor que el valor verdadero, el error es positivo. También en las ecuaciones (3.3) a (3.5), el denominador puede ser menor a cero, lo cual también llevaría a un error negativo. A menudo, cuando se realizan cálculos, no importa mucho el signo del error, sino más bien que su valor absoluto porcentual sea menor que una tolerancia porcentual prefijada ε_s . Por lo tanto, es útil emplear el valor absoluto de las ecuaciones (3.2) a (3.5). En tales casos, los cálculos se repiten hasta que

$$|\varepsilon_a| < \varepsilon_s \quad (3.6)$$

Si se cumple la relación anterior, entonces se considera que el resultado obtenido está dentro del nivel aceptable fijado previamente ε_s . Observe que en el resto del texto en general emplearemos exclusivamente valores absolutos cuando utilicemos errores relativos.

Es conveniente también relacionar estos errores con el número de cifras significativas en la aproximación. Es posible demostrar (Scarborough, 1966) que si el siguiente criterio se cumple, se tendrá la seguridad que el resultado es correcto en *al menos* n cifras significativas.

$$\varepsilon_s = (0.5 \times 10^{2-n})\% \quad (3.7)$$

EJEMPLO 3.2 Estimación del error con métodos iterativos

Planteamiento del problema. En matemáticas con frecuencia las funciones se representan mediante series infinitas. Por ejemplo, la función exponencial se calcula usando

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots + \frac{x^n}{n!} \quad (\text{E3.2.1})$$

Así cuanto más términos se le agreguen a la serie, la aproximación será cada vez más una mejor estimación del valor verdadero de e^x . La ecuación (E3.2.1) se conoce como *expansión en series de Maclaurin*.

Empezando con el primer término $e^x = 1$ y agregando término por término, estime el valor de $e^{0.5}$. Después de agregar cada término, calcule los errores: relativo porcentual verdadero y normalizado a un valor aproximado usando las ecuaciones (3.3) y (3.5), respectivamente. Observe que el valor verdadero es $e^{0.5} = 1.648721\ldots$ Agregue términos hasta que el valor absoluto del error aproximado ε_a sea menor que un criterio de error preestablecido ε_s con tres cifras significativas.

Solución. En primer lugar la ecuación (3.7) se emplea para determinar el criterio de error que asegura que un resultado sea correcto en al menos tres cifras significativas:

$$\varepsilon_s = (0.5 \times 10^{2-3})\% = 0.05\%$$

Por lo tanto, se agregarán términos a la serie hasta que ε_a sea menor que este valor.

La primera estimación es igual a la ecuación (E3.2.1) con un solo término. Entonces, la primera estimación es igual a 1. La segunda estimación se obtiene agregando el segundo término, así:

$$e^x = 1 + x$$

y para $x = 0.5$,

$$e^{0.5} = 1 + 0.5 = 1.5$$

Esto representa el error relativo porcentual verdadero de [ecuación (3.3)]

$$\varepsilon_t = \frac{1.648721 - 1.5}{1.648721} 100\% = 9.02\%$$

La ecuación (3.5) se utiliza para determinar una estimación aproximada del error, dada por:

$$\varepsilon_a = \frac{1.5 - 1}{1.5} 100\% = 33.3\%$$

Como ε_a no es menor que el valor requerido ε_s , se deben continuar los cálculos agregando otro término, $x^2/2!$, repitiendo el cálculo del error. El proceso continúa hasta que $\varepsilon_a < \varepsilon_s$. Todos los cálculos se resumen de la siguiente manera

Términos	Resultado	ε_t (%)	ε_a (%)
1	1	39.3	
2	1.5	9.02	33.3
3	1.625	1.44	7.69
4	1.645833333	0.175	1.27
5	1.648437500	0.0172	0.158
6	1.648697917	0.00142	0.0158

Así, después de usar seis términos, el error aproximado es menor que $\varepsilon_s = 0.05\%$, y el cálculo termina. Sin embargo, observe que, ¡el resultado es exacto con cinco cifras significativas! en vez de tres cifras significativas. Esto se debe a que, en este caso, las ecuaciones (3.5) y (3.7) son conservadoras. Es decir, aseguran que el resultado es, por lo menos, tan bueno como lo especifican. Aunque, como se analiza en el capítulo 6, éste no es siempre el caso al usar la ecuación (3.5), que es verdadera en la mayoría de las veces.

Con las definiciones anteriores como antecedente, se procede ahora a examinar los dos tipos de error relacionados directamente con los métodos numéricos: el error de redondeo y el error de truncamiento.

3.4 ERRORES DE REDONDEO

Como se mencionó antes, los errores de redondeo se originan debido a que la computadora emplea un número determinado de cifras significativas durante un cálculo. Los

números tales como π , e o $\sqrt{7}$ no pueden expresarse con un número fijo de cifras significativas. Por lo tanto, no pueden ser representados exactamente por la computadora. Además, debido a que las computadoras usan una representación en base 2, no pueden representar exactamente algunos números en base 10. Esta discrepancia por la omisión de cifras significativas se llama *error de redondeo*.

3.4.1 Representación de números en la computadora

Numéricamente los errores de redondeo se relacionan de manera directa con la forma en que se guardan los números en la memoria de la computadora. La unidad fundamental mediante la cual se representa la información se llama *palabra*. Ésta es una entidad que consiste en una cadena de *dígitos binarios* o *bits* (*binary digits*). Por lo común, los números son guardados en una o más palabras. Para entender cómo se realiza esto, se debe revisar primero algún material relacionado con los sistemas numéricos.

Sistemas numéricos. Un *sistema numérico* es simplemente una convención para representar cantidades. Debido a que se tienen 10 dedos en las manos y 10 dedos en los pies, el sistema de numeración que nos es muy familiar es el *decimal* o de *base 10*. Una base es el número que se usa como referencia para construir un sistema. El sistema de base 10 utiliza 10 dígitos (0, 1, 2, 3, 4, 5, 6, 7, 8, 9) para representar números. Tales dígitos son satisfactorios por sí mismos para contar de 0 a 9.

Para grandes cantidades se usa la combinación de estos dígitos básicos; con la posición o *valor de posición* se especifica su magnitud. El dígito en el extremo derecho de un número entero representa un número del 0 al 9. El segundo dígito a partir de la derecha representa un múltiplo de 10. El tercer dígito a partir de la derecha representa un múltiplo de 100 y así sucesivamente. Por ejemplo, si se tiene el número 86409 se tienen 8 grupos de 10000, seis grupos de 1000, cuatro grupos de 100 y cero grupos de 10, y nueve unidades, o bien

$$(8 \times 10^4) + (6 \times 10^3) + (4 \times 10^2) + (0 \times 10^1) + (9 \times 10^0) = 86409$$

La figura 3.3a ofrece una representación de cómo se formula un número en el sistema de base 10. Este tipo de representación se llama *notación posicional*.

Debido a que el sistema decimal resulta ser tan familiar, no es común darse cuenta de que existen otras alternativas. Por ejemplo, si el ser humano tuviera ocho dedos en las manos y ocho en los pies, se tendría, sin duda, una representación en un sistema *octal* o de *base 8*. En tal sentido nuestra amiga la computadora es como un animal que tiene dos dedos, limitado a dos estados: 0 o 1. Esto se relaciona con el hecho de que las unidades lógicas fundamentales de las computadoras digitales sean componentes electrónicos de apagado/encendido. Por lo tanto, los números en la computadora se representan con un sistema *binario* o de *base 2*. Del mismo modo que con el sistema decimal, las cantidades pueden representarse usando la notación posicional. Por ejemplo, el número binario 11 es equivalente a $(1 \times 2^1) + (1 \times 2^0) = 2 + 1 = 3$ en el sistema decimal. En la figura 3.3b se ilustra un ejemplo más complejo.

Representación entera. Ahora que se ha revisado cómo los números de base 10 se representan en forma binaria, es fácil concebir cómo los enteros se representan en la computadora. El método más sencillo se denomina *método de magnitud con signo* y

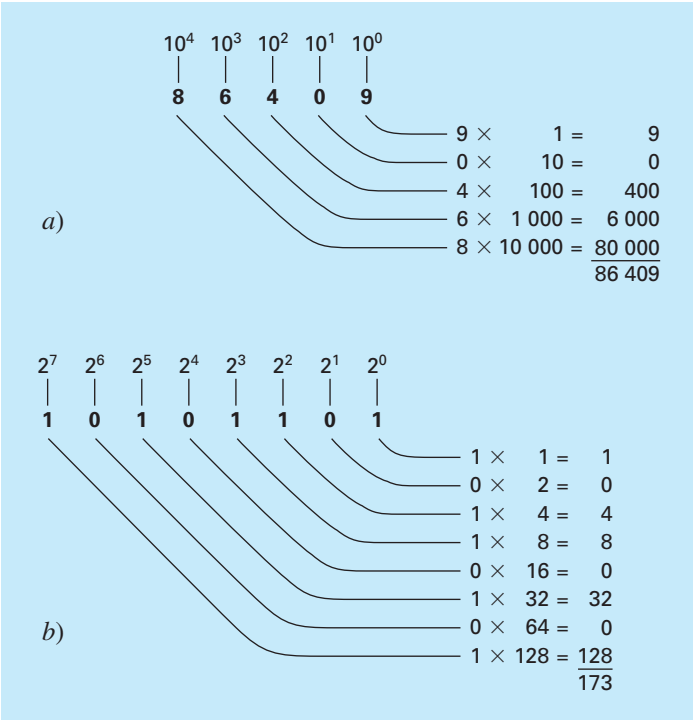
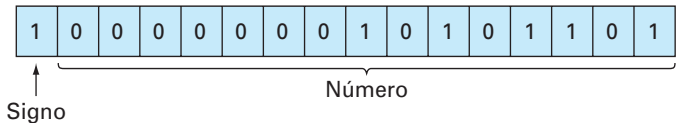


FIGURA 3.3
Cómo trabajan los sistemas a) decimal (base 10) y b) binario (base 2). En b) el número binario 10101101 es equivalente al número decimal 173.

emplea el primer bit de una palabra para indicar el signo: con un 0 para positivo y un 1 para el negativo. Los bits sobrantes se usan para guardar el número. Por ejemplo, el valor entero -173 puede guardarse en la memoria de una computadora de 16 bits como se muestra en la figura 3.4.

FIGURA 3.4
La representación de un entero decimal -173 en una computadora de 16 bits usando el método de magnitud con signo.



EJEMPLO 3.3 Rango de enteros

Planteamiento del problema. Determine el rango de enteros de base 10 que pueda representarse en una computadora de 16 bits.

Solución. De los 16 bits, se tiene el primer bit para el signo. Los 15 bits restantes pueden contener los números binarios de 0 a 111111111111111. El límite superior se convierte en un entero decimal, así

$$(1 \times 2^{14}) + (1 \times 2^{13}) + \dots + (1 \times 2^1) + (1 \times 2^0)$$

que es igual a 32 767 (observe que esta expresión puede simplemente evaluarse como $2^{15} - 1$). Así, en una computadora de 16 bits una palabra puede guardar en memoria un entero decimal en el rango de $-32\,767$ a $32\,767$. Además, debido a que el cero está ya definido como 0000000000000000, sería redundante usar el número 1000000000000000 para definir “menos cero”. Por lo tanto, es usualmente empleado para representar un número negativo adicional: $-32\,768$, y el rango va de $-32\,768$ a $32\,767$.

Observe que el método de magnitud con signo descrito antes no se utiliza para representar enteros en computadoras convencionales. Se prefiere usar una técnica llamada *complemento de 2* que incorpora en forma directa el signo dentro de la magnitud del número, en lugar de emplear un bit adicional para representar más o menos (véase Chappra y Canale, 1994). Sin embargo, en el ejemplo 3.3 sigue sirviendo para ilustrar cómo todas las computadoras digitales están limitadas en cuanto a su capacidad para representar enteros. Esto es, los números por encima o por debajo de este rango no pueden representarse. Una limitación más importante se encuentra en el almacenaje y la manipulación de cantidades fraccionarias, como se describe a continuación.

Representación del punto-flotante. Las cantidades fraccionarias generalmente se representan en la computadora usando la forma de punto flotante. Con este método, el número se expresa como una parte fraccionaria, llamada *mantisa* o *significando*, y una parte entera, denominada *exponente* o *característica*, esto es,

$$m \cdot b^e$$

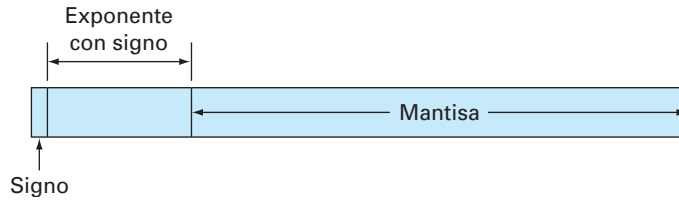
donde m = la mantisa, b = la base del sistema numérico que se va a utilizar y e = el exponente. Por ejemplo, el número 156.78 se representa como 0.15678×10^3 en un sistema de base 10 de punto flotante.

En la figura 3.5 se muestra una forma en que el número de punto flotante se guarda en una palabra. El primer bit se reserva para el signo; la siguiente serie de bits, para el exponente con signo; y los últimos bits, para la mantisa.

Observe que la mantisa es usualmente *normalizada* si tiene primero cero dígitos. Por ejemplo, suponga que la cantidad $1/34 = 0.029411765\dots$ se guarda en un sistema de base 10 con punto flotante, que únicamente permite guardar cuatro lugares decimales. Entonces, $1/34$ se guardaría como

$$0.0294 \times 10^0$$

Sin embargo, al hacerlo así, la inclusión del cero “inútil” a la derecha del punto decimal nos obliga a eliminar el dígito 1 del quinto lugar decimal. El número puede normalizarse

**FIGURA 3.5**

La forma en que un número de punto flotante se guarda en una palabra.

para eliminar el cero multiplicando la mantisa por 10 y disminuyendo el exponente en 1, para quedar

$$0.2941 \times 10^{-1}$$

Así, se conserva una cifra significativa adicional al guardar el número.

La consecuencia de la normalización es que el valor absoluto de m queda limitado. Esto es,

$$\frac{1}{b} \leq m < 1 \quad (3.8)$$

donde b = la base. Por ejemplo, para un sistema de base 10, m estaría entre 0.1 y 1; y para un sistema de base 2, entre 0.5 y 1.

La representación de punto flotante permite que tanto fracciones como números muy grandes se expresen en la computadora. Sin embargo, hay algunas desventajas. Por ejemplo, los números de punto flotante requieren más espacio y más tiempo de procesamiento que los números enteros. Más importante aun es que su uso introduce una fuente de error debido a que la mantisa conserva sólo un número finito de cifras significativas. Por lo tanto, se introduce un error de redondeo.

EJEMPLO 3.4 Conjunto hipotético de números con punto flotante

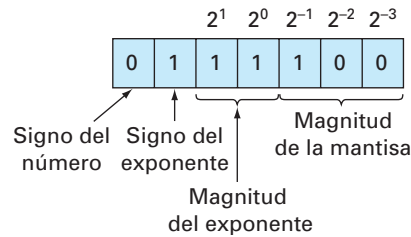
Planteamiento del problema. Determine un conjunto hipotético de números con punto flotante para una máquina que guarda información usando palabras de 7 bits. Emplee el primer bit para el signo del número, los siguientes tres para el signo y la magnitud del exponente, y los últimos tres para la magnitud de la mantisa (véase figura 3.6).

Solución. El número positivo más pequeño posible se representa en la figura 3.6. El 0 inicial señala que la cantidad es positiva. El 1 en el segundo lugar indica que el exponente tiene signo negativo. Los 1, en el tercero y cuarto lugar dan un valor máximo al exponente de

$$1 \times 2^1 + 1 \times 2^0 = 3$$

Por lo tanto, el exponente será -3 . Por último, la mantisa está especificada por el 100 en los últimos tres lugares, lo cual nos da

$$1 \times 2^{-1} + 0 \times 2^{-2} + 0 \times 2^{-3} = 0.5$$

**FIGURA 3.6**

El número positivo de punto flotante más pequeño posible del ejemplo 3.4.

Aunque es posible tomar una mantisa más pequeña (por ejemplo, 000, 001, 010, 011), se emplea el valor de 100 debido al límite impuesto por la normalización [ecuación (3.8)]. Así, el número positivo más pequeño posible en este sistema es $+0.5 \times 2^{-3}$, el cual es igual a 0.0625 en el sistema de base 10. Los siguientes números más grandes se desarrollan incrementando la mantisa como sigue:

$$0111101 = (1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3}) \times 2^{-3} = (0.078125)_{10}$$

$$0111110 = (1 \times 2^{-1} + 1 \times 2^{-2} + 0 \times 2^{-3}) \times 2^{-3} = (0.093750)_{10}$$

$$0111111 = (1 \times 2^{-1} + 1 \times 2^{-2} + 1 \times 2^{-3}) \times 2^{-3} = (0.109375)_{10}$$

Observe que las equivalencias de base 10 se esparcen de manera uniforme en un intervalo de 0.015625.

En este punto, para continuar el incremento se debe disminuir el exponente a 10, lo cual da un valor de

$$1 \times 2^1 + 0 \times 2^0 = 2$$

La mantisa disminuye hasta su valor más pequeño: 100. Por lo tanto, el siguiente número es

$$0110100 = (1 \times 2^{-1} + 0 \times 2^{-2} + 0 \times 2^{-3}) \times 2^{-2} = (0.125000)_{10}$$

Esto todavía representa una brecha o espacio de $0.125000 - 0.109375 = 0.015625$. Sin embargo, cuando los números grandes se generan incrementando la mantisa, la brecha es de 0.03125,

$$0110101 = (1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3}) \times 2^{-2} = (0.156250)_{10}$$

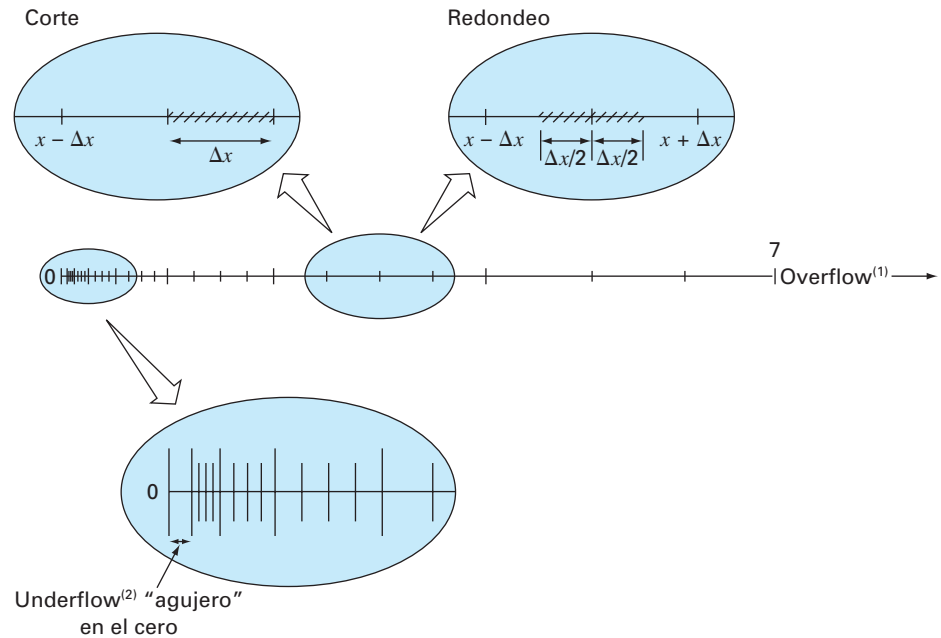
$$0110110 = (1 \times 2^{-1} + 1 \times 2^{-2} + 0 \times 2^{-3}) \times 2^{-2} = (0.187500)_{10}$$

$$0110111 = (1 \times 2^{-1} + 1 \times 2^{-2} + 1 \times 2^{-3}) \times 2^{-2} = (0.218750)_{10}$$

Este patrón se repite conforme se formula una cantidad mayor hasta que se alcanza un número máximo:

$$0011111 = (1 \times 2^{-1} + 1 \times 2^{-2} + 1 \times 2^{-3}) \times 2^3 = (7)_{10}$$

El conjunto del número final se muestra en la figura 3.7.



⁽¹⁾ Se genera una cantidad demasiado grande, en una operación aritmética, que rebasa la capacidad del registro

⁽²⁾ Se genera una cantidad, en una operación aritmética, demasiado pequeña, para que pueda ser almacenada.

FIGURA 3.7

Sistema numérico hipotético desarrollado en el ejemplo 3.4. Cada valor se indica con una marca. Tan sólo se muestran los números positivos. Un conjunto idéntico se extendería en dirección negativa.

En la figura 3.7 se presentan diversos aspectos de la representación de punto flotante, que son importantes respecto de los errores de redondeo en las computadoras.

1. *El rango de cantidades que pueden representarse es limitado.* Como en el caso de los enteros, hay números grandes positivos y negativos que no pueden representarse. Intentar emplear números fuera del rango aceptable dará como resultado el llamado *error de desbordamiento (overflow)*. Sin embargo, además de las grandes cantidades, la representación de punto flotante tiene la limitación adicional de que números muy pequeños no pueden representarse. Esto se ilustra por el "agujero" *underflow* entre el cero y el primer número positivo en la figura 3.7. Se debe observar que este agujero aumenta por las limitaciones de normalización de la ecuación (3.8).
2. *Existe sólo un número finito de cantidades que puede representarse dentro de un rango.* Así, el grado de precisión es limitado. Es evidente que los números irracionales no pueden representarse de manera exacta. Además, los números racionales que no concuerdan exactamente con uno de los valores en el conjunto tampoco pueden ser representados en forma precisa. A los errores ocasionados por la aproximación

en ambos casos se les conoce como errores de *cuantificación*. La aproximación real se realiza por dos caminos: cortando o redondeando. Por ejemplo, suponga que el valor de $\pi = 3.14159265358\dots$ se va a guardar en un sistema de numeración de base 10 con 7 cifras significativas. Un método de aproximación podría ser simplemente omitir, o “cortar”, el octavo y demás términos, como en $\pi = 3.141592$, con la introducción de un error asociado de [ecuación (3.2)]

$$E_t = 0.00000065\dots$$

Esta técnica de mantener sólo términos significativos fue originalmente conocida como “truncamiento” en la jerga computacional. Preferimos llamarla *corte* para distinguirla de los errores de truncamiento que se analizarán en el capítulo 4. Observe que en el sistema numérico de base 2 de la figura 3.7, corte significa que cualquier cantidad que esté dentro de un intervalo de longitud Δx se guardará en memoria como una cantidad en el extremo inferior del intervalo. Así, el error máximo por corte es Δx . Además, se presenta un sesgo porque todos los errores son positivos. La deficiencia del corte se atribuye al hecho de que los términos superiores de la representación decimal completa no tienen impacto en la versión cortada. Así, en el ejemplo de π , el primer dígito descartado es 6. El último dígito retenido debería redondearse a 3.141593. Tal *redondeo* reduce el error a

$$E_t = -0.00000035\dots$$

En consecuencia, el redondeo produce un error absoluto menor que el de corte. Observe que, en el sistema numérico de base 2 de la figura 3.7, redondear significa que cualquier cantidad que esté en un intervalo de longitud Δx se representará como el número más cercano permitido. Entonces, el error máximo de redondeo es $\Delta x/2$. Además, no se presenta sesgo porque ciertos errores son positivos y otros son negativos. Algunas computadoras emplean redondeo. Sin embargo, esto aumenta el trabajo computacional y, en consecuencia, muchas máquinas simplemente usan el corte. Dicho enfoque se justifica con la suposición de que el número de cifras significativas es suficientemente grande para que los errores de redondeo resultantes sean despreciables.

3. *El intervalo entre los números, Δx , aumenta conforme los números crecen en magnitud.* Ésta es la característica, por supuesto, que permite que la representación de punto flotante conserve los dígitos significativos. Sin embargo, también quiere decir que los errores de cuantificación sean proporcionales a la magnitud del número que será representado. Para normalizar los números de punto flotante, esta proporcionalidad se expresa, para los casos en que se emplea el corte, como

$$\frac{|\Delta x|}{|x|} \leq \mathcal{E} \quad (3.9)$$

y, para los casos donde se utiliza el redondeo, como

$$\frac{|\Delta x|}{|x|} \leq \frac{\mathcal{E}}{2} \quad (3.10)$$

donde a $\mathcal{E}$ se le denomina *épsilon de la máquina*, el cual se calcula como

$$\mathcal{E} = b^{1-t} \quad (3.11)$$

donde b es el número base y t es el número de dígitos significativos en la mantisa. Observe que las desigualdades en las ecuaciones (3.9) y (3.10) quieren decir que éstos son los límites de los errores. Es decir, especifican los casos extremos.

EJEMPLO 3.5 Épsilon de la máquina

Planteamiento del problema. Determine el épsilon de la máquina y verifique su efectividad para caracterizar los errores del sistema numérico del ejemplo 3.4. Suponga que se usa al corte.

Solución. El sistema de punto flotante hipotético del ejemplo 3.4 empleaba valores de base $b = 2$, y número de bits de la mantisa $t = 3$. Por lo tanto, el épsilon de la máquina debe ser [ecuación (3.11)]

$$\mathcal{E} = 2^{1-3} = 0.25$$

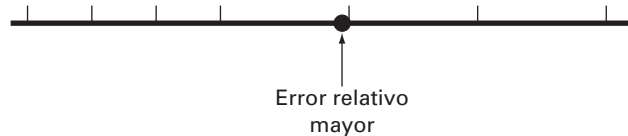
En consecuencia, el error de cuantificación relativo estará limitado por 0.25, para el corte. El error relativo más grande debería ocurrir para aquellas cantidades que caen justo debajo del límite superior del primer intervalo entre números equidistantes sucesivos (véase figura 3.8). Aquellos números que caen en los intervalos sucesivos siguientes tendrán el mismo valor de Δx pero un mayor valor de x y, por lo tanto, tendrán un error relativo bajo. Un ejemplo de un error máximo sería un valor que cae justo por debajo de límite superior del intervalo entre $(0.125000)_{10}$ y $(0.156250)_{10}$. Para este caso, el error sería menor a

$$\frac{0.03125}{0.125000} = 0.25$$

Entonces, el error es como se predijo mediante la ecuación (3.9).

FIGURA 3.8

El error de cuantificación más grande ocurrirá para aquellos valores que caigan justo debajo del límite superior del primero de una serie de intervalos equiespaciados.



El hecho de que los errores de cuantificación dependan de la magnitud tiene varias aplicaciones prácticas en los métodos numéricos. Muchas de ellas están relacionadas con la comúnmente empleada operación de probar si dos números son iguales. Ello

```

epsilon = 1
DO
  IF (epsilon+1 ≤ 1)
    EXIT
  epsilon = epsilon/2
END DO
epsilon = 2 × epsilon

```

FIGURA 3.9

Seudocódigo para determinar el ϵ de la máquina en una computadora binaria.

ocurre cuando se prueba la convergencia de cantidades, así como en los mecanismos para detener procesos iterativos (véase el ejemplo 3.2). En estos casos deberá ser claro que más que probar si las dos cantidades son iguales, es recomendable probar si su diferencia es menor que una pequeña tolerancia aceptable. Además, deberá ser evidente que más que la diferencia absoluta, deberá compararse la diferencia normalizada, en especial cuando se trabaja con números de gran magnitud. El ϵ de la máquina, además, se emplea al formular criterios de paro o de convergencia. Esto asegura que los programas sean portátiles, es decir, que no sean dependientes de la computadora sobre la cual se hayan implementado. En la figura 3.9 se presenta un pseudocódigo que automáticamente determina el ϵ de la máquina en una computadora binaria.

Precisión extendida. Aquí se debe observar que, aunque los errores de redondeo llegan a ser importantes en contextos tales como pruebas de convergencia, el número de dígitos significativos que tiene la mayoría de las computadoras permite que muchos cálculos de ingeniería se realicen con una precisión más que aceptable. Por ejemplo, el sistema numérico hipotético de la figura 3.7 es una enorme exageración que se usó con propósitos ilustrativos. En las computadoras comerciales se utilizan conjuntos mucho más grandes y por consiguiente se permite que los números queden expresados con una precisión adecuada. Por ejemplo, las computadoras que usan el formato IEEE permiten 24 bits para ser usados por la mantisa, lo cual se traduce en cerca de siete cifras significativas de precisión¹ en dígitos de base 10 con un rango aproximado de 10^{-38} a 10^{39} .

Se debe reconocer que aún hay casos donde el error de redondeo resulta crítico. Por tal razón muchas computadoras permiten la especificación de precisión extendida. La más común de estas especificaciones es la doble precisión, en la cual se duplica el número de palabras utilizado para guardar números de punto flotante. Esto proporciona de 15 a 16 dígitos decimales de precisión y un rango aproximado de 10^{-308} a 10^{308} .

En muchos casos el uso de cantidades de doble precisión llega a reducir, en gran medida, el efecto del error de redondeo. Sin embargo, el precio que se paga por tales medidas remediales consiste en mayores requerimientos de memoria y de tiempo de ejecución. La diferencia en el tiempo de ejecución de un cálculo pequeño podría parecer insignificante. No obstante, conforme los programas van siendo cada vez más grandes y complicados, el tiempo de ejecución agregado se vuelve más considerable y repercute de manera negativa para resolver el problema en forma efectiva. Por lo tanto, la precisión extendida no debería utilizarse en forma generalizada. Por el contrario, deberá ser empleada en forma selectiva, donde se obtenga un máximo beneficio al menor costo en términos de tiempo de ejecución. En las siguientes secciones veremos más de cerca cómo los errores de redondeo afectan los cálculos y ello nos servirá para comprender los fundamentos que nos guían en el uso de la capacidad de la doble precisión.

Antes de proseguir, debemos observar que algunos paquetes de software de uso común (por ejemplo, Excel o Mathcad) normalmente utilizan doble precisión para representar las cantidades numéricas. Así, quienes desarrollaron estos paquetes decidieron reducir los errores de redondeo sacrificando velocidad para usar una precisión extendida. Otros, como el MATLAB, permiten usar la precisión extendida, si se desea.

¹ Observe que, de hecho, únicamente 23 bits se emplean en la memoria para la mantisa. Sin embargo, debido a la normalización, el primer bit de la mantisa es siempre 1 y, por lo tanto, no se guarda. Así, el primer bit junto con los 23 bits de memoria dan 24 bits en total para la precisión de la mantisa.

3.4.2 Manipulación aritmética de números en la computadora

Junto con las limitaciones del sistema numérico de una computadora, las manipulaciones aritméticas que se usan con tales números también pueden dar como resultado errores de redondeo. En la siguiente sección se ilustrará primero cómo afectan las operaciones aritméticas comunes a los errores de redondeo. De este modo, investigaremos varias manipulaciones que son especialmente propensas a errores de redondeo.

Operaciones aritméticas comunes. A causa de que estamos familiarizados con los números de base 10, los emplearemos para ilustrar el efecto del error de redondeo en las operaciones básicas: suma, resta, multiplicación y división. Otras bases de números pueden tener un comportamiento similar. Para simplificar el análisis, emplearemos una computadora decimal hipotética con una mantisa de 4 dígitos y un exponente de 1 dígito. Además, se usará el corte. El redondeo puede implicar errores similares, aunque menos dramáticos.

Cuando se suman dos números de punto flotante, el número de la mantisa con el exponente menor se modifica de tal forma que los exponentes sean los mismos. Esto tiene el efecto de alinear los puntos decimales. Por ejemplo, suponga que se quiere sumar $0.1557 \cdot 10^1 + 0.4381 \cdot 10^{-1}$. El decimal de la mantisa del segundo número se recorre a la izquierda un número de lugares igual a la diferencia de los exponentes [$1 - (-1) = 2$], así,

$$0.4381 \cdot 10^{-1} \rightarrow 0.004381 \cdot 10^1$$

Ahora se suman los números,

$$\begin{array}{r} 0.1557 \cdot 10^1 \\ 0.004381 \cdot 10^1 \\ \hline 0.160081 \cdot 10^1 \end{array}$$

y el resultado es cortado a $0.1600 \cdot 10^1$. Note cómo los últimos dos dígitos del segundo número que se recorrieron a la derecha fueron eliminados de los cálculos.

La resta se realiza en forma idéntica a la suma, con la excepción del signo del sustraendo, que es negativo. Por ejemplo, suponga que hacemos la resta 36.41 menos 26.86. Esto es,

$$\begin{array}{r} 0.3641 \cdot 10^2 \\ -0.2686 \cdot 10^2 \\ \hline 0.0955 \cdot 10^2 \end{array}$$

Aquí el resultado no está normalizado y se debe recorrer el decimal un lugar a la derecha para obtener $0.9550 \cdot 10^1 = 9.550$. Observe que el cero sumado al final de la mantisa no es relevante, tan sólo llena el espacio vacío creado al recorrer los números. Es posible obtener resultados más dramáticos todavía, cuando las cantidades estén muy cercanas, como por ejemplo,

$$\begin{array}{r} 0.7642 \cdot 10^3 \\ -0.7641 \cdot 10^3 \\ \hline 0.0001 \cdot 10^3 \end{array}$$

que podría convertirse en $0.1000 \cdot 10^0 = 0.1000$. Así, en este caso, se agregan tres ceros no significativos, lo cual introduce un error sustancial de cálculo debido a que las manipulaciones siguientes actúan como si los ceros fueran significativos. Como se verá más adelante en otra sección, la pérdida significativa durante la resta de números casi iguales es una de las principales fuentes de errores de redondeo en los métodos numéricos.

La multiplicación y la división resultan un poco más sencillos que la suma y la resta. Los exponentes se suman y la mantisa se multiplica. Debido a que la multiplicación de dos mantisas de n dígitos da como resultado $2n$ dígitos, muchas computadoras ofrecen resultados intermedios en un registro de doble longitud. Por ejemplo,

$$0.1363 \cdot 10^3 \times 0.6423 \cdot 10^{-1} = 0.08754549 \cdot 10^2$$

Si, como en este caso, se introduce un cero, el resultado es normalizado,

$$0.08754549 \cdot 10^2 \rightarrow 0.8754549 \cdot 10^1$$

y cortando resulta

$$0.8754 \cdot 10^1$$

La división se realiza en forma similar, aunque las mantisas se dividen y los exponentes se restan. Entonces el resultado es normalizado y cortado.

Cálculos grandes. Ciertos métodos requieren un número extremadamente grande de manipulaciones aritméticas para llegar a los resultados finales. Además, dichos cálculos a menudo son interdependientes; es decir, los cálculos son dependientes de los resultados previos. En consecuencia, aunque el error de redondeo individual sea pequeño, el efecto acumulativo durante el proceso de muchos cálculos puede ser relevante.

EJEMPLO 3.6 Un número grande de cálculos interdependientes

Planteamiento del problema. Investigue el efecto del error de redondeo en un gran número de cálculos interdependientes. Desarrolle un programa que sume un número 100 000 veces. Sume el número 1 con precisión simple, y 0.00001 con precisiones simple y doble.

Solución. En la figura 3.10 se muestra un programa en Fortran 90 que realiza la suma. Mientras que la suma con precisión simple de 1 dará el resultado esperado, la precisión simple en la suma de 0.00001 tiene una gran discrepancia. Este error se reduce de manera importante cuando 0.00001 se suma con precisión doble.

Los errores de cuantificación son la fuente de las discrepancias. Debido a que el entero 1 puede ser representado en forma exacta en la computadora, puede sumarse exactamente. En contraste, 0.00001 no puede representarse con exactitud y se cuantifica con un valor que es ligeramente diferente de su valor verdadero. Aunque esta ligera discrepancia resultará insignificante para un cálculo pequeño, se acumula después de la repetición de sumas. Tal problema ocurre también con la precisión doble, pero se reduce en forma relevante porque el error de cuantificación es mucho más pequeño.

FIGURA 3.10

Programa en Fortran 90 para sumar un número 10^5 veces. Aquí se suma el número 1 con precisión simple y el número 10^{-5} con precisiones simple y doble.

```
PROGRAM fig0310
IMPLICIT none
INTEGER::i
REAL::sum1, sum2, x1, x2
DOUBLE PRECISION::sum3, x3
sum1=0.
sum2=0.
sum3=0.
x1=1.
x2=1.e-5
x3=1.d-5
DO i=1, 100000
    sum1=sum1+x1
    sum2=sum2+x2
    sum3=sum3+x3
END DO
PRINT *, sum1
PRINT *, sum2
PRINT *, sum3
END
output:
100000.000000
      1.000990
 9.999999999980838E-001
```

Observe que el tipo de error ilustrado en el ejemplo anterior es algo atípico porque todos los errores en las operaciones que se repiten tienen el mismo signo. En muchos casos, los errores en grandes cálculos alternan el signo de manera aleatoria y, entonces, con frecuencia se cancelan. Sin embargo, hay también algunos casos donde tales errores no se cancelan pero, en efecto, llevan a resultados finales dudosos. En las siguientes secciones se mostrará cómo puede ocurrir esto.

Suma de un número grande y uno pequeño. Suponga que se desea sumar un número pequeño, 0.0010, con un número grande, 4 000, utilizando una computadora hipotética con una mantisa de 4 dígitos y un exponente de 1 dígito. Modificamos el número pequeño para que su exponente sea igual al del grande,

$$\begin{array}{r} 0.4000 \quad \cdot 10^4 \\ 0.0000001 \cdot 10^4 \\ \hline 0.4000001 \cdot 10^4 \end{array}$$

el cual se corta a $0.4000 \cdot 10^4$. Así, ¡resultó lo mismo que si no hubiéramos realizado la suma!

Este tipo de error puede ocurrir cuando se calculan series infinitas. Por ejemplo, si el término inicial de una serie es relativamente grande en comparación con los demás términos, después de que se han sumado unos pocos términos, estamos en la situación de sumar una cantidad pequeña a una cantidad grande.

Una manera de reducir este tipo de errores consiste en sumar la serie en sentido inverso: esto es, en orden ascendente en lugar de descendente. De esta manera, cada nuevo término será comparable en magnitud con el de la suma acumulada (véase el problema 3.4).

Cancelación por resta. Se refiere al redondeo inducido cuando se restan dos números de punto flotante casi iguales.

Un caso común donde esto ocurre es en la determinación de las raíces de una ecuación cuadrática o parábola utilizando la fórmula cuadrática,

$$\begin{aligned} x_1 &= \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \\ x_2 & \end{aligned} \quad (3.12)$$

En los casos donde $b^2 \gg 4ac$, la diferencia en el numerador puede ser muy pequeña. En tales casos, la precisión doble llega a reducir el problema. Además, una formulación alternativa puede usarse para minimizar la cancelación por resta.

$$\begin{aligned} x_1 &= \frac{-2c}{b \pm \sqrt{b^2 - 4ac}} \\ x_2 & \end{aligned} \quad (3.13)$$

Una ilustración del problema y del uso de esta fórmula alternativa se ofrecen en el siguiente ejemplo.

EJEMPLO 3.7 Cancelación por resta

Planteamiento del problema. Calcule el valor de las raíces de una ecuación cuadrática con $a = 1$, $b = 3\,000.001$ y $c = 3$. Compare el valor calculado con las raíces verdaderas $x_1 = -0.001$ y $x_2 = -3\,000$.

Solución. En la figura 3.11 se muestra un programa en Fortran 90 que calcula las raíces x_1 y x_2 usando la fórmula cuadrática [(ecuación (3.12))]. Observe que se dan las versiones tanto de la precisión simple como la precisión doble. Mientras que los resultados para x_2 son adecuados, el error relativo porcentual para x_1 es pobre para la precisión simple, $\varepsilon_t = 2.4\%$. Este valor quizá resulte para muchos problemas de aplicaciones en ingeniería. ¡Este resultado es en particular sorprendente porque se emplea una fórmula analítica para obtener la solución!

La pérdida de significancia ocurre en la línea del programa donde dos números grandes se restan. No ocurren problemas semejantes cuando los mismos números se suman.

Considerando lo anterior podemos obtener la conclusión general de que la fórmula cuadrática será susceptible de cancelación por resta cada vez que $b^2 \gg 4ac$. Una manera de evitar este problema consiste en usar precisión doble. Otra es reacomodar la fórmula cuadrática en la forma de la ecuación (3.13). Ya que en la salida del programa, ambas opciones dan un error mucho menor porque se minimiza o evita la cancelación por resta.

```

PROGRAM fig0311
IMPLICIT none
REAL::a,b,c,d,x1,x2,x1r
DOUBLE PRECISION::aa,bb,cc,dd,x11,x22
a = 1.
b = 3000.001
c = 3.
d = SQRT(b * b - 4. * a * c)
x1 = (-b + d) / (2. * a)
x2 = (-b - d) / (2. * a)
PRINT *, 'resultados con precisión
simple:'
PRINT '(1x,a10,f20.14)', 'x1 = ', x1
PRINT '(1x,a10,f10.4)', 'x2 = ', x2
PRINT *
aa = 1.
bb = 3000.001
cc = 3.
dd = SQRT(bb * bb - 4. * aa * cc)
x11 = (-bb + dd) / (2. * aa)
x22 = (-bb - dd) / (2. * aa)

PRINT *, 'resultados con precisión
doble:'
PRINT '(1x,a10,f20.14)', 'x1 = ', x11
PRINT '(1x,a10,f10.4)', 'x2 = ', x22
PRINT *
PRINT *, 'fórmula modificada para la
primer raíz:'
x1r = -2. * c / (b + d)
PRINT '(1x,a10,f20.14)', 'x1 = ', x1r
END

```

SALIDA

```

resultados con precisión simple:
    x1 =      -0.00097656250000
    x2 = -3000.0000
resultados con precisión doble:
    x1 =      -0.001000000000771
    x2 = -3000.0000
fórmula modificada para la primera raíz:
    x1 = -.0010000000000000

```

FIGURA 3.11

Programa en Fortran 90 para determinar las raíces de una ecuación cuadrática. Con precisiones simple y doble.

Considere que, como en el ejemplo anterior, hay veces en las que la cancelación por resta se evita empleando una transformación. No obstante, el único remedio general es usar la precisión extendida.

Dispersión. La dispersión ocurre generalmente cuando los términos individuales en la sumatoria son más grandes que la sumatoria misma. Como en el siguiente ejemplo, casos como éstos ocurren en las series con signos alternados.

EJEMPLO 3.8 Evaluación de e^x usando series infinitas

Planteamiento del problema. La función exponencial $y = e^x$ está dada por la serie infinita

$$y = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots$$

Evalúe esta función para $x = 10$ y $x = -10$; esté atento al problema del error de redondeo.

Solución. En la figura 3.12a se muestra un programa en Fortran 90 que utiliza una serie infinita para evaluar e^x . La variable i es el número de términos en la serie, $term$ es el valor de término actual que se le agrega a la serie, y sum es el valor acumulado de la serie. La variable $test$ es el valor acumulado precedente de la serie antes de la suma de $term$. La serie se termina cuando la computadora no puede detectar la diferencia entre $test$ y sum .

a) Programa

```

PROGRAM fig0312
IMPLICIT none
Real::term, test, sum,x
INTEGER::i
i = 0
term = 1.
sum = 1.
test = 0.
PRINT *, 'x = '
READ *, x
PRINT *, 'i', 'term', 'sum'
DO
    IF (sum.EQ.test) EXIT
    PRINT *, i, term, sum
    i = i + 1
    term = term*x/i
    test = sum
    sum = sum+term
END DO
PRINT *, 'valor exacto =', exp(x)
END

```

b) Evaluación de e^{10}

x=			
10	i	term	sum
	0	1.000000	1.000000
	1	10.000000	11.000000
	2	50.000000	61.000000
	3	166.666700	227.666700
	4	416.666700	644.333400
	5	833.333400	1477.667000
		.	
		.	
		.	
	27	9.183693E-02	22026.420000
	28	3.279890E-02	22026.450000
	29	1.130997E-02	22026.460000
	30	3.769989E-03	22026.470000
	31	1.216126E-03	22026.470000
	valor exacto = 22026.460000		

c) Evaluación de e^{-10}

x=			
-10	i	term	sum
	0	1.000000	1.000000
	1	-10.000000	-9.000000
	2	50.000000	41.000000
	3	-166.666700	-125.666700
	4	416.666700	291.000000
	5	-833.333400	-542.333400
		.	
		.	
		.	
	1	-2.989312E-09	8.137590E-05
	42	7.117410E-10	8.137661E-05
	43	-1.655212E-10	8.137644E-05
	44	3.761845E-11	8.137648E-05
	45	-8.359655E-12	8.137647E-05
	valor exacto = 4.539993E-05		

FIGURA 3.12a) Un programa en Fortran 90 para evaluar e^x usando series infinitas. b) Evaluación de e^x . c) Evaluación de e^{-x} .

La figura 3.12b muestra los resultados de la ejecución del programa para $x = 10$. Observe que este caso es completamente satisfactorio. El resultado final se alcanza en 31 términos con la serie idéntica para el valor de la función en la biblioteca con siete cifras significativas.

En la figura 3.12c se muestran los resultados para $x = -10$. Sin embargo, en este caso, los resultados de la serie calculada no coinciden ni en el signo con respecto al resultado verdadero. De hecho, los resultados negativos abren una gama de preguntas serias porque e^x nunca puede ser menor que cero. El problema es causado por el error de redondeo. Observe que muchos de los términos que conforman la suma son mucho más grandes que el resultado final de la suma. Además, a diferencia del caso anterior, los términos individuales varían de signo. Así, en efecto, estamos sumando y restando números grandes (cada uno con algún error pequeño) y dando gran significancia a las diferencias; esto es, cancelación por resta. Entonces, puede verse que el culpable en este ejemplo de dispersión es, en efecto, la cancelación por resta. En tales casos es apropiado buscar alguna otra estrategia de cálculo. Por ejemplo, uno podría tratar de calcular $y = e^{-10}$ como $y = (e^{-1})^{10}$. En lugar de una reformulación, ya que el único recurso general es la precisión extendida.

Productos internos. De las secciones anteriores debe quedar claro que, algunas series infinitas son particularmente propensas a errores por redondeo. Por fortuna, el cálculo de series no es una de las operaciones más comunes en métodos numéricos. Una manipulación más frecuente es el cálculo de productos internos, esto es,

$$\sum_{i=1}^n x_i y_i = x_1 y_1 + x_2 y_2 + \cdots + x_n y_n$$

Esta operación es muy común, en particular en la solución de ecuaciones simultáneas lineales algebraicas. Tales sumatorias son propensas a errores por redondeo. En consecuencia, a menudo es deseable calcular tales sumas con precisión extendida.

Aunque en las secciones siguientes se ofrecerán reglas prácticas para reducir el error de redondeo, no son un medio directo mejor que el método de prueba y error para determinar realmente el efecto de tales errores en los cálculos. En el próximo capítulo se presentará la serie de Taylor, la cual proporcionará un enfoque matemático para estimar esos efectos.

PROBLEMAS

3.1 Convierta los números siguientes en base 2 a números en base 10: a) 1011101. b) 101.101, y c) 0.01101.

3.2 Realice su propio programa con base en la figura 3.9 y úselo para determinar el épsilon de máquina de su computadora.

3.3 En forma similar a la de la figura 3.9, escriba un programa corto para determinar el número más pequeño, $x_{\min}$, que utiliza la computadora que empleará con este libro. Observe que su computadora será incapaz de diferenciar entre cero y una cantidad más pequeña que dicho número.

3.4 La serie infinita

$$f(n) = \sum_{i=1}^n \frac{1}{i^4}$$

converge a un valor de $f(n) = \pi^4/90$ conforme n se tiende a infinito. Escriba un programa de precisión sencilla para calcular $f(n)$ para $n = 10000$ por medio de calcular la suma desde $i = 1$ hasta 10000. Después repita el cálculo pero en sentido inverso, es

decir, desde $i = 10000$ a 1, con incrementos de -1 . En cada caso, calcule el error relativo porcentual verdadero. Explique los resultados.

3.5 Evalúe e^{-5} con el uso de dos métodos

$$e^{-x} = 1 - x + \frac{x^2}{2} - \frac{x^3}{3!} + \dots$$

y

$$e^{-x} = \frac{1}{e^x} = \frac{1}{1 + x + \frac{x^2}{2} + \frac{x^3}{3!} + \dots}$$

y compárelo con el valor verdadero de 6.737947×10^{-3} . Utilice 20 términos para evaluar cada serie y calcule los errores relativos aproximado y verdadero como términos que se agregaran.

3.6 La derivada de $f(x) = 1/(1 - 3x^2)^2$ está dada por

$$\frac{6x}{(1 - 3x^2)^2}$$

¿Esperaría el lector dificultades para evaluar esta función para $x = 0.577$? Inténtelo con aritmética de 3 y 4 dígitos con corte.

3.7 a) Evalúe el polinomio

$$y = x^3 - 7x^2 + 8x + 0.35$$

en $x = 1.37$. Utilice aritmética de 3 dígitos con corte. Evalúe el error relativo porcentual.

b) Repita el inciso a) pero exprese a y como

$$y = [(x - 7)x + 8]x + 0.35$$

Evalúe el error y compárelo con el inciso a).

3.8 Calcule la memoria de acceso al azar (RAM) en megabytes, que es necesaria para almacenar un arreglo multidimensional de $20 \times 40 \times 120$. Este arreglo es de doble precisión, y cada valor requiere una palabra de 64 bits. Recuerde que una palabra de 64 bits = 8 bytes, y un kilobyte = 2^{10} bytes. Suponga que el índice comienza en 1.

3.9 Determine el número de términos necesarios para aproximar $\cos x$ a 8 cifras significativas con el uso de la serie de McLaurin.

$$\cos x = 1 - \frac{x^2}{2} + \frac{x^4}{4!} - \frac{x^6}{6!} + \frac{x^8}{8!} - \dots$$

Calcule la aproximación con el empleo del valor de $x = 0.3\pi$. Escriba un programa para determinar el resultado.

3.10 Utilice aritmética de 5 dígitos con corte para determinar las raíces de la ecuación siguiente, por medio de las ecuaciones (3.12) y (3.13).

$$x^2 - 5000.002x + 10$$

Calcule los errores relativos porcentuales de sus resultados.

3.11 ¿Cómo puede emplearse el épsilon de la máquina para formular un criterio de detención ε_s para sus programas? Dé un ejemplo.

CAPÍTULO 4

Errores de truncamiento y la serie de Taylor

Los *errores de truncamiento* son aquellos que resultan al usar una aproximación en lugar de un procedimiento matemático exacto. Por ejemplo, en el capítulo 1 aproximamos la derivada de la velocidad de caída de un paracaidista mediante una ecuación en diferencia finita dividida de la forma [ecuación (1.11)]

$$\frac{dv}{dt} \cong \frac{\Delta v}{\Delta t} = \frac{v(t_{i+1}) - v(t_i)}{t_{i+1} - t_i} \quad (4.1)$$

Se presentó un error de truncamiento en la solución numérica, ya que la ecuación en diferencia sólo aproxima el valor verdadero de la derivada (véase figura 1.4). Para obtener un conocimiento sobre las características de estos errores, debe considerar una formulación matemática que se utiliza ampliamente en los métodos numéricos para expresar funciones de manera aproximada: la serie de Taylor.

4.1 LA SERIE DE TAYLOR

El teorema de Taylor (véase cuadro 4.1) y su fórmula, la serie de Taylor, es de gran valor en el estudio de los métodos numéricos. En esencia, la *serie de Taylor* proporciona un medio para predecir el valor de una función en un punto en términos del valor de la función y sus derivadas en otro punto. En particular, el teorema establece que cualquier función suave puede aproximarse por un polinomio.

Una buena manera de comprender la serie de Taylor consiste en construirla término por término. Por ejemplo, el primer término de la serie es:

$$f(x_{i+1}) \cong f(x_i) \quad (4.2)$$

Esta relación, llamada la *aproximación de orden cero*, indica que el valor de f en el nuevo punto es el mismo que su valor en el punto anterior. Tal resultado tiene un sentido intuitivo, ya que si x_i y x_{i+1} están muy próximas entre sí, entonces es muy probable que el nuevo valor sea similar al anterior.

La ecuación (4.2) ofrece una estimación perfecta si la función que se va a aproximar es, de hecho, una constante. Sin embargo, si la función cambia en el intervalo, entonces se requieren los términos adicionales de la serie de Taylor, para obtener una mejor aproximación. Por ejemplo, la *aproximación de primer orden* se obtiene sumando otro término para obtener:

$$f(x_{i+1}) \cong f(x_i) + f'(x_i)(x_{i+1} - x_i) \quad (4.3)$$

Cuadro 4.1 Teorema de Taylor

Teorema de Taylor

Si la función f y sus primeras $n + 1$ derivadas son continuas en un intervalo que contiene a y x , entonces el valor de la función en x está dado por

$$\begin{aligned} f(x) = & f(a) + f'(a)(x-a) + \frac{f''(a)}{2!}(x-a)^2 \\ & + \frac{f^{(3)}(a)}{3!}(x-a)^3 + \cdots \\ & + \frac{f^{(n)}(a)}{n!}(x-a)^n + R_n \end{aligned} \quad (\text{C4.1.1})$$

donde el residuo R_n se define como

$$R_n = \int_a^x \frac{(x-t)^n}{n!} f^{(n+1)}(t) dt \quad (\text{C4.1.2})$$

donde $t = a$ es una variable muda. La ecuación (C4.1.1) se llama *serie de Taylor* o *fórmula de Taylor*. Si se omite el residuo, el lado derecho de la ecuación (C4.1.1) es la aproximación del polinomio de Taylor para $f(x)$. En esencia, el teorema establece que cualquier función suave puede aproximarse mediante un polinomio.

La ecuación (C4.1.2) es sólo una manera, denominada la *forma integral*, mediante la cual puede expresarse el residuo. Se obtiene una formulación alternativa basándose en el teorema del valor medio para integrales.

Primer teorema del valor medio para integrales

Si la función g es continua e integrable en un intervalo que contenga a y x , entonces existe un punto ξ entre a y x tal que

$$\int_a^x g(t) dt = g(\xi)(x-a) \quad (\text{C4.1.3})$$

En otras palabras, el teorema establece que la integral puede representarse por un valor promedio de la función $g(\xi)$ multiplicado por la longitud del intervalo $x - a$. Como el promedio debe encontrarse entre los valores mínimo y máximo del intervalo, existe un punto $x = \xi$ en el cual la función toma el valor promedio.

El primer teorema es, de hecho, un caso especial del segundo teorema del valor medio para integrales.

Segundo teorema del valor medio para integrales

Si las funciones g y h son continuas e integrables en un intervalo que contiene a y x , y h no cambia de signo en el intervalo, entonces existe un punto ξ entre a y x tal que

$$\int_a^x g(t)h(t) dt = g(\xi) \int_a^x h(t) dt \quad (\text{C4.1.4})$$

La ecuación (C4.1.3) es equivalente a la ecuación (C4.1.4) con $h(t) = 1$.

El segundo teorema se aplica a la ecuación (C4.1.2) con

$$g(t) = f^{(n+1)}(t) \quad h(t) = \frac{(x-t)^n}{n!}$$

Conforme t varía de a a x , $h(t)$ es continua y no cambia de signo. Por lo tanto, si $f^{(n+1)}(t)$ es continua, entonces se satisface el teorema del valor medio para integrales y

$$R_n = \frac{f^{(n+1)}(\xi)}{(n+1)!} (x-a)^{n+1}$$

Esta ecuación es conocida como la *forma de Lagrange* del residuo.

El término adicional de primer orden consiste en una pendiente $f'(x_i)$ multiplicada por la distancia entre x_i y x_{i+1} . Por lo tanto, la expresión representa ahora una línea recta y es posible predecir un incremento o un decremento de la función entre x_i y x_{i+1} .

Aunque la ecuación (4.3) puede predecir un cambio, sólo es exacta para una línea recta o una tendencia *lineal*. Por lo tanto, se le agrega a la serie un término de *segundo orden* para obtener algo de la curvatura, que pudiera presentar la función:

$$f(x_{i+1}) \cong f(x_i) + f'(x_i)(x_{i+1} - x_i) + \frac{f''(x_i)}{2!}(x_{i+1} - x_i)^2 \quad (4.4)$$

De manera similar, se agregan términos adicionales para desarrollar la expansión completa de la serie de Taylor:

$$f(x_{i+1}) = f(x_i) + f'(x_i)(x_{i+1} - x_i) + \frac{f''(x_i)}{2!}(x_{i+1} - x_i)^2 + \frac{f^{(3)}(x_i)}{3!}(x_{i+1} - x_i)^3 + \dots + \frac{f^{(n)}(x_i)}{n!}(x_{i+1} - x_i)^n + R_n \quad (4.5)$$

Observe que debido a que la ecuación (4.5) es una serie infinita, el signo igual reemplaza al signo de aproximación que se utiliza en las ecuaciones (4.2) a (4.4). Se incluye un término residual para considerar todos los términos desde el $n + 1$ hasta infinito:

$$R_n = \frac{f^{(n+1)}(\xi)}{(n+1)!}(x_{i+1} - x_i)^{n+1} \quad (4.6)$$

donde el subíndice n indica que éste es el residuo de la aproximación de n -ésimo orden y ξ es un valor de x que se encuentra en algún punto entre x_i y x_{i+1} . La ξ es tan importante que se dedica una sección completa (sección 4.1.1) para su estudio. Por ahora es suficiente darse cuenta de que existe este valor que da una estimación exacta del error.

Con frecuencia es conveniente simplificar la serie de Taylor definiendo un tamaño de paso o incremento $h = x_{i+1} - x_i$ y expresando la ecuación (4.5) como:

$$f(x_{i+1}) = f(x_i) + f'(x_i)h + \frac{f''(x_i)}{2!}h^2 + \frac{f^{(3)}(x_i)}{3!}h^3 + \dots + \frac{f^{(n)}(x_i)}{n!}h^n + R_n \quad (4.7)$$

donde el término residual es ahora

$$R_n = \frac{f^{(n+1)}(\xi)}{(n+1)!}h^{n+1} \quad (4.8)$$

EJEMPLO 4.1 Aproximaciones de un polinomio mediante la serie de Taylor

Planteamiento del problema. Use expansiones de la serie de Taylor de los órdenes cero hasta cuatro para aproximar la función

$$f(x) = -0.1x^4 - 0.15x^3 - 0.5x^2 - 0.25x + 1.2$$

desde $x_i = 0$ con $h = 1$. Esto es, prediga el valor de la función en $x_{i+1} = 1$.

Solución. Ya que se trata de una función conocida, es posible calcular valores de $f(x)$ entre 0 y 1. Los resultados (véase figura 4.1) indican que la función empieza en $f(0) = 1.2$ y hace una curva hacia abajo hasta $f(1) = 0.2$. Por lo tanto, el valor verdadero que se trata de predecir es 0.2.

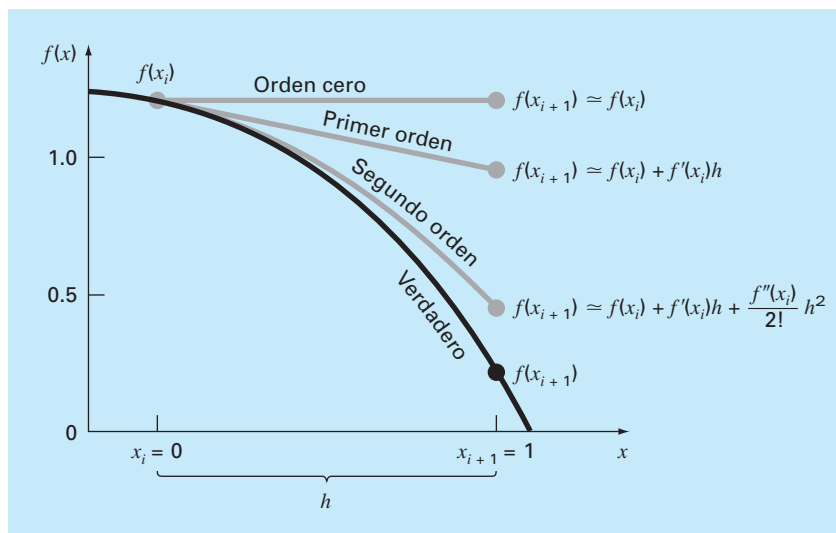
La aproximación de la serie de Taylor con $n = 0$ es [ecuación (4.2)]

$$f(x_{i+1}) \cong 1.2$$

Como se muestra en la figura 4.1, la aproximación de orden cero es una constante. Usando esta formulación resulta un error de truncamiento [recuerde la ecuación (3.2)] de

$$E_t = 0.2 - 1.2 = -1.0$$

en $x = 1$.

**FIGURA 4.1**

Aproximación de $f(x) = -0.1x^4 - 0.15x^3 - 0.5x^2 - 0.25x + 1.2$ en $x = 1$ mediante expansiones de la serie de Taylor de órdenes cero, primero y segundo.

Para $n = 1$, se debe determinar y evaluar la primer derivada en $x = 0$:

$$f'(0) = -0.4(0.0)^3 - 0.45(0.0)^2 - 1.0(0.0) - 0.25 = -0.25$$

La aproximación de primer orden es entonces [véase ecuación (4.3)]

$$f(x_{i+1}) \approx 1.2 - 0.25h$$

que se emplea para calcular $f(1) = 0.95$. La aproximación empieza a coincidir con la trayectoria hacia abajo de la función en forma de una línea recta inclinada (véase figura 4.1). De esta manera, el error de truncamiento se reduce a

$$E_t = 0.2 - 0.95 = -0.75$$

Para $n = 2$, se evalúa la segunda derivada en $x = 0$:

$$f''(0) = -1.2(0.0)^2 - 0.9(0.0) - 1.0 = -1.0$$

Entonces, de acuerdo con la ecuación (4.4)

$$f(x_{i+1}) \approx 1.2 - 0.25h - 0.5h^2$$

y sustituyendo $h = 1$, $f(1) = 0.45$. Al incluirse la segunda derivada se añade una curvatura descendente que proporciona una mejor estimación, como se muestra en la figura 4.1. Además, el error de truncamiento se reduce a $0.2 - 0.45 = -0.25$.

Los términos adicionales mejoran aún más la aproximación. En efecto, la inclusión de la tercera y de la cuarta derivadas da como resultado exactamente la misma ecuación del principio:

$$f(x) = 1.2 - 0.25h - 0.5h^2 - 0.15h^3 - 0.1h^4$$

donde el término residual es

$$R_4 = \frac{f^{(5)}(\xi)}{5!} h^5 = 0$$

ya que la quinta derivada de un polinomio de cuarto orden es cero. Por consiguiente, la expansión de la serie de Taylor hasta la cuarta derivada da una estimación exacta para $x_{i+1} = 1$:

$$f(1) = 1.2 - 0.25(1) - 0.5(1)^2 - 0.15(1)^3 - 0.1(1)^4 = 0.2$$

En general, la expansión de la serie de Taylor de n -ésimo orden será exacta para un polinomio de n -ésimo orden. Para otras funciones continuas y diferenciables, como las exponenciales y las senoidales, no se obtiene una estimación exacta con un número finito de términos. Cada uno de los términos adicionales contribuye, aunque sea con poco, al mejoramiento de la aproximación. Esto se muestra en el ejemplo 4.2, donde se obtendría un resultado exacto únicamente si se le agrega un número infinito de términos.

Aunque lo anterior es cierto, el valor práctico de las expansiones de la serie de Taylor estriba, en la mayoría de los casos, en el uso de pocos términos que darán una aproximación lo suficientemente cercana a la solución verdadera para propósitos prácticos. La determinación de cuántos términos se requieren para obtener una “aproximación razonable” se basa en el término residual de la expansión. Recuerde que el término residual es de la forma general de la ecuación (4.8). Dicha fórmula tiene dos grandes inconvenientes. Primero, ξ no se conoce con exactitud, sino que sólo se sabe que está entre x_i y x_{i+1} . Segundo, para la evaluación de la ecuación (4.8) se requiere determinar la $(n + 1)$ ésima derivada de $f(x)$. Para hacerlo, se necesita conocer $f(x)$. Pero si ya se conoce $f(x)$, entonces no hay razón para realizar la expansión de la serie de Taylor.

A pesar de este dilema, la ecuación (4.8) aún resulta útil para la evaluación de errores de truncamiento. Esto se debe a que se *tiene* control sobre el término h de la ecuación. En otras palabras, es posible decidir qué tan lejos de x se desea evaluar $f(x)$ y controlar el número de términos que queremos tener en la expansión. Por esto, la ecuación (4.8) se expresa usualmente como

$$R_n = O(h^{n+1})$$

donde la nomenclatura $O(h^{n+1})$ significa que el error de truncamiento es de orden h^{n+1} . Es decir, el error es proporcional al incremento h elevado a la $(n + 1)$ ésima potencia. Aunque esta aproximación no implica nada en relación con la magnitud de las derivadas que multiplican h^{n+1} , es extremadamente útil para evaluar el error comparativo de los métodos numéricos que se basan en expansiones de la serie de Taylor. Por ejemplo, si el error es $O(h)$ y el incremento se reduce a la mitad, entonces el error también se reducirá a la mitad. Por otro lado, si el error es $O(h^2)$ y el incremento se reduce a la mitad, entonces el error se reducirá a una cuarta parte.

En general, se considera que el error de truncamiento disminuye agregando términos a la serie de Taylor. En muchos casos, si h es suficientemente pequeño, entonces el término de primer orden y otros términos de orden inferior causan un porcentaje desproporcionadamente alto del error. Esta propiedad se ilustra en el ejemplo siguiente.

EJEMPLO 4.2 Uso de la expansión de la serie de Taylor para aproximar una función con un número infinito de derivadas

Planteamiento del problema. Utilice expansiones de la serie de Taylor con n desde 0 hasta 6 para aproximar $f(x) = \cos x$ en $x_{i+1} = \pi/3$ con base en el valor de $f(x)$ y sus derivadas en $x_i = \pi/4$. Observe que esto significa que $h = \pi/3 - \pi/4 = \pi/12$.

Solución. Como en el ejemplo 4.1, el conocimiento de la función original implica que se puede determinar el valor exacto de $f(\pi/3) = 0.5$.

La aproximación de orden cero es [ecuación (4.3)]

$$f\left(\frac{\pi}{3}\right) \cong \cos\left(\frac{\pi}{4}\right) = 0.707106781$$

que representa un error relativo porcentual de

$$\varepsilon_t = \frac{0.5 - 0.707106781}{0.5} 100\% = -41.4\%$$

Para la aproximación de primer orden, se agrega el término de la primera derivada donde $f'(x) = -\sin x$:

$$f\left(\frac{\pi}{3}\right) \cong \cos\left(\frac{\pi}{4}\right) - \sin\left(\frac{\pi}{4}\right)\left(\frac{\pi}{12}\right) = 0.521986659$$

que tiene $\varepsilon_t = -4.40$ por ciento.

Para la aproximación de segundo orden, se agrega el término de la segunda derivada donde $f''(x) = -\cos x$:

$$f\left(\frac{\pi}{3}\right) \cong \cos\left(\frac{\pi}{4}\right) - \sin\left(\frac{\pi}{4}\right)\left(\frac{\pi}{12}\right) - \frac{\cos(\pi/4)}{2}\left(\frac{\pi}{12}\right)^2 = 0.497754491$$

con $\varepsilon_t = 0.449$ por ciento. Entonces, al agregar más términos a la serie se obtiene una mejor aproximación.

Este proceso continúa y sus resultados se enlistan, como en la tabla 4.1. Observe que las derivadas nunca se aproximan a cero, como es el caso con el polinomio del ejemplo 4.1. Por lo tanto, cada término que se le agrega a la serie genera una mejor aproximación.

TABLA 4.1 Aproximaciones mediante la serie de Taylor de $f(x) = \cos x$ en $x_{i+1} = \pi/3$ usando como punto base $\pi/4$. Los valores se presentan para varios órdenes (n) de aproximación.

Orden n	$f^{(n)}(x)$	$f(\pi/3)$	ε_t
0	$\cos x$	0.707106781	-41.4
1	$-\sin x$	0.521986659	-4.4
2	$-\cos x$	0.497754491	0.449
3	$\sin x$	0.499869147	2.62×10^{-2}
4	$\cos x$	0.500007551	-1.51×10^{-3}
5	$-\sin x$	0.500000304	-6.08×10^{-5}
6	$-\cos x$	0.499999988	2.40×10^{-6}

Sin embargo, observe también que la mejor aproximación se consigue con los primeros términos. En este caso, al agregar el tercer término, el error se redujo al $2.62 \times 10^{-2}\%$, lo cual significa que se alcanzó el 99.9738% del valor exacto. Por consiguiente, aunque se le agreguen más términos a la serie el error decrece, aunque la mejoría será mínima.

4.1.1 El residuo en la expansión de la serie de Taylor

Antes de mostrar cómo se utiliza la serie de Taylor en la estimación de errores numéricos, se debe explicar por qué se incluye el argumento ξ en la ecuación (4.8). Un desarrollo matemático se presenta en el cuadro 4.1. Ahora se expondrá una interpretación más visual. Después se extiende este caso específico a una formulación más general.

Suponga que se trunca la expansión de la serie de Taylor [ecuación (4.7)] después del término de orden cero para obtener:

$$f(x_{i+1}) \cong f(x_i)$$

En la figura 4.2 se muestra una representación gráfica de esta predicción de orden cero. El residuo o error de esta predicción, que se indica también en la figura, consiste de la serie infinita de términos que fueron truncados:

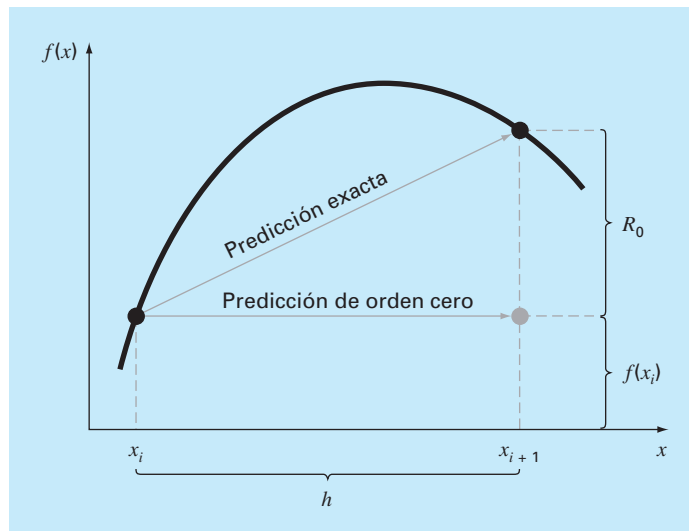
$$R_0 = f'(x_i)h + \frac{f''(x_i)}{2!}h^2 + \frac{f^{(3)}(x_i)}{3!}h^3 + \dots$$

Obviamente no resulta conveniente manipular el residuo en este formato de serie infinita. Se obtiene una simplificación truncando el residuo mismo de la siguiente manera

$$R_0 \cong f'(x_i)h \quad (4.9)$$

FIGURA 4.2

Representación gráfica de una predicción de orden cero con la serie de Taylor y del residuo.



Aunque como se mencionó en la sección previa, por lo común las derivadas de orden inferior cuentan mucho más en el residuo que los términos de las derivadas de orden superior; este resultado todavía es inexacto, ya que se han despreciado los términos de segundo orden y de órdenes superiores. Esta “inexactitud” se denota mediante el símbolo de aproximación a la igualdad ($\approx$) empleado en la ecuación (4.9).

Una simplificación alternativa que transforma la aproximación en una equivalencia está basada en un esquema gráfico. Como se muestra en la figura 4.3 el *teorema del valor medio para la derivada* establece que si una función $f(x)$ y su primera derivada son continuas en el intervalo de x_i a x_{i+1} , entonces existe al menos un punto en la función que tiene una pendiente, denotada por $f'(\xi)$, que es paralela a la línea que une $f(x_i)$ y $f(x_{i+1})$. El parámetro ξ marca el valor x donde se presenta la pendiente (figura 4.3). Una ilustración física de este teorema es la siguiente: si usted viaja entre dos puntos a una velocidad promedio, habrá al menos un momento durante el curso del viaje en que usted se mueve a esa velocidad promedio.

Al utilizar este teorema resulta fácil darse cuenta, como se muestra en la figura (4.3), de que la pendiente $f'(\xi)$ es igual al cociente de la elevación R_0 entre el recorrido h , o

$$f'(\xi) = \frac{R_0}{h}$$

que se puede reordenar para obtener

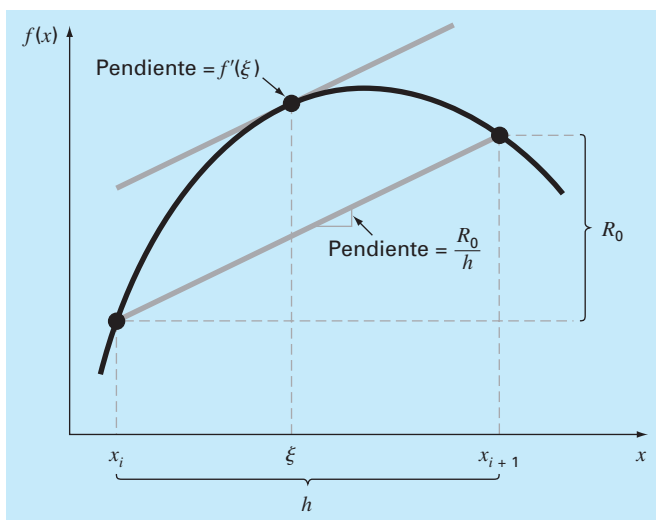
$$R_0 = f'(\xi)h \quad (4.10)$$

Por lo tanto, se ha obtenido la versión de orden cero de la ecuación (4.8). Las versiones de orden superior son tan sólo una extensión lógica del razonamiento usado para encontrar la ecuación (4.10). La versión de primer orden es

$$R_1 = \frac{f''(\xi)}{2!} h^2 \quad (4.11)$$

FIGURA 4.3

Representación gráfica del teorema del valor medio para la derivada.



En este caso, el valor de ξ será el valor de x que corresponde a la derivada de segundo orden que hace exacta a la ecuación (4.11). Es posible obtener versiones similares de orden superior a partir de la ecuación (4.8).

4.1.2 Uso de la serie de Taylor para estimar los errores de truncamiento

Aunque la serie de Taylor será muy útil en la estimación de los errores de truncamiento a lo largo de este libro, quizá no resulte claro cómo la expansión se aplica a los métodos numéricos. De hecho, esto ya se hizo en el ejemplo de la caída del paracaidista. Recuerde que el objetivo de los ejemplos 1.1 y 1.2 fue predecir la velocidad como una función del tiempo. Es decir, se deseaba determinar $v(t)$. Como se especificó en la ecuación (4.5), $v(t)$ se puede expandir en una serie de Taylor del siguiente modo:

$$v(t_{i+1}) = v(t_i) + v'(t_i)(t_{i+1} - t_i) + \frac{v''(t_i)}{2!}(t_{i+1} - t_i)^2 + \cdots + R_n \quad (4.12)$$

Ahora, truncando la serie después del término con la primera derivada, se obtiene:

$$v(t_{i+1}) = v(t_i) + v'(t_i)(t_{i+1} - t_i) + R_1 \quad (4.13)$$

En la ecuación (4.13) se despeja obteniendo

$$v'(t_i) = \underbrace{\frac{v(t_{i+1}) - v(t_i)}{t_{i+1} - t_i}}_{\text{Aproximación de primer orden}} - \underbrace{\frac{R_1}{t_{i+1} - t_i}}_{\text{Error de truncamiento}} \quad (4.14)$$

La primera parte de la ecuación (4.14) es exactamente la misma relación que se usó para aproximar la derivada del ejemplo 1.2 [ecuación (1.11)]. Sin embargo, con el método de la serie de Taylor se ha obtenido una estimación del error de truncamiento asociado con esta aproximación de la derivada. Utilizando las ecuaciones (4.6) y (4.14) se tiene

$$\frac{R_1}{t_{i+1} - t_i} = \frac{v'(\xi)}{2!}(t_{i+1} - t_i) \quad (4.15)$$

o

$$\frac{R_1}{t_{i+1} - t_i} = O(t_{i+1} - t_i) \quad (4.16)$$

Por lo tanto, la estimación de la derivada [ecuación (1.11) o la primera parte de la ecuación (4.14)] tiene un error de truncamiento de orden $t_{i+1} - t_i$. En otras palabras, el error en nuestra aproximación de la derivada debería ser proporcional al tamaño del incremento. Entonces, si éste se divide a la mitad, se esperaría que el error de la derivada se reduzca a la mitad.

EJEMPLO 4.3 El efecto de no linealidad y del tamaño del incremento en la aproximación de la serie de Taylor

Planteamiento del problema. En la figura 4.4 se grafica la función

$$f(x) = x^m \quad (\text{E4.3.1})$$

para $m = 1, 2, 3$ y 4 en el rango de $x = 1$ a 2 . Observe que para $m = 1$ la función es lineal, y conforme m se incrementa, se presenta mayor curvatura o no linealidad dentro de la función.

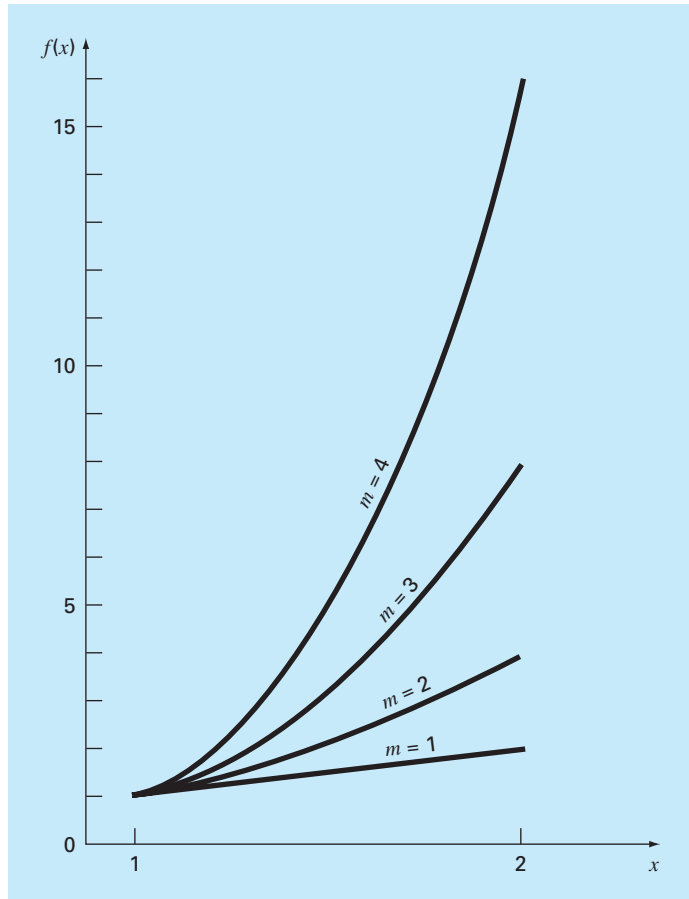


FIGURA 4.4

Gráfica de la función $f(x) = x^m$ para $m = 1, 2, 3$ y 4 . Note que la función tiende a ser más no lineal cuando aumenta m .

Utilizar la serie de Taylor de primer orden para aproximar la función con diversos valores del exponente m y del tamaño de incremento h .

Solución. La ecuación (E4.3.1) se aproxima por una expansión de la serie de Taylor de primer orden:

$$f(x_{i+1}) = f(x_i) + mx_i^{m-1}h \quad (\text{E4.3.2})$$

la cual tiene un residuo de

$$R_1 = \frac{f''(x_i)}{2!}h^2 + \frac{f^{(3)}(x_i)}{3!}h^3 + \frac{f^{(4)}(x_i)}{4!}h^4 + \dots$$

Primero, puede examinarse cómo se comporta la aproximación conforme m aumenta; es decir, conforme la función se vuelve más no lineal. Para $m = 1$, el valor verdadero de la función en $x = 2$ es 2. La serie de Taylor nos da

$$f(2) = 1 + 1(1) = 2$$

y

$$R_1 = 0$$

El residuo es cero porque la segunda derivada y las derivadas de orden superior de una función lineal son cero. Entonces, como es de esperarse, la expansión de la serie de Taylor de primer orden es perfecta cuando la función de que se trata es lineal.

Para $m = 2$, el valor real es $f(2) = 2^2 = 4$. La aproximación de la serie de Taylor de primer orden es

$$f(2) = 1 + 2(1) = 3$$

y

$$R_1 = \frac{2}{2}(1)^2 + 0 + 0 + \dots = 1$$

Debido a que la función es una parábola, la aproximación mediante una línea recta da por resultado una discrepancia. Observe que el residuo se determina en forma exacta.

Para $m = 3$, el valor real es $f(2) = 2^3 = 8$. La aproximación con la serie de Taylor es

$$f(2) = 1 + 3(1)^2(1) = 4$$

y

$$R_1 = \frac{6}{2}(1)^2 + \frac{6}{6}(1)^3 + 0 + 0 + \dots = 4$$

Otra vez, hay una discrepancia que se puede determinar exactamente a partir de la serie de Taylor.

Para $m = 4$, el valor real es $f(2) = 2^4 = 16$. La aproximación con la serie de Taylor es

$$f(2) = 1 + 4(1)^3(1) = 5$$

y

$$R_1 = \frac{12}{2}(1)^2 + \frac{24}{6}(1)^3 + \frac{24}{24}(1)^4 + 0 + 0 + \dots = 11$$

Considerando estos cuatro casos, se observa que R_1 se incrementa conforme la función empieza a ser cada vez más no lineal. Además, R_1 da cuenta exacta de la discrepancia, porque la ecuación (E4.3.1) es un simple monomio con un número finito de derivadas. Esto permite una completa determinación del residuo de la serie de Taylor.

Ahora examinemos la ecuación (E4.3.2) para el caso en que $m = 4$ y observe cómo R_1 cambia cuando el tamaño del incremento h varía. Para $m = 4$, la ecuación (E4.3.2) es

$$f(x+h) = f(x) + 4x_i^3 h$$

Si $x = 1$, $f(1) = 1$ y esta ecuación se expresa como

$$f(1+h) = 1 + 4h$$

con un residuo de

$$R_1 = 6h^2 + 4h^3 + h^4$$

Lo cual nos lleva a la conclusión de que la discrepancia disminuirá conforme h se reduzca. Entonces, para valores suficientemente pequeños de h , el error debería ser proporcional a h^2 . Es decir, conforme h se reduce a la mitad, el error se reduce a la cuarta parte. Este comportamiento se confirma en la tabla 4.2 y en la figura 4.5.

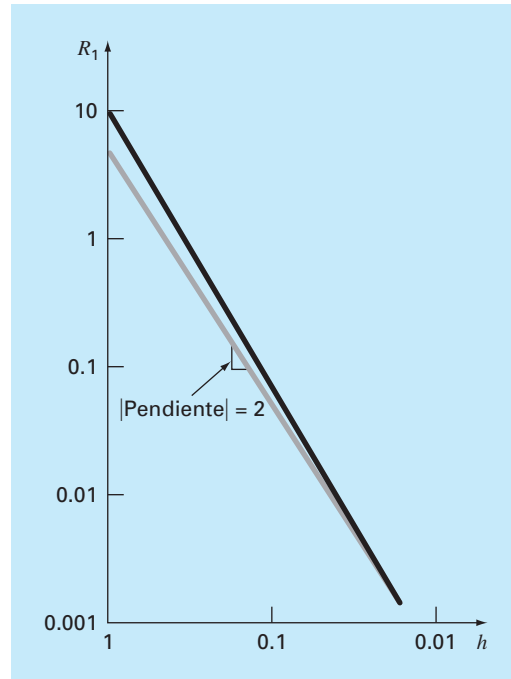


FIGURA 4.5

Gráfica en escala log-log del residuo R_1 para la aproximación de la función $f(x) = x^4$ mediante la serie de Taylor de primer orden contra el tamaño del incremento h . La línea con la pendiente 2 también se muestra para indicar que conforme h disminuye, el error se vuelve proporcional a h^2 .

TABLA 4.2 Comparación del valor exacto de la función $f(x) = x^4$ con la aproximación de la serie de Taylor de primer orden. Ambos, la función y la aproximación, se evalúan en $x + h$, donde $x = 1$.

h	Verdadero	Aproximación de primer orden	R_1
1	16	5	11
0.5	5.0625	3	2.0625
0.25	2.441406	2	0.441406
0.125	1.601807	1.5	0.101807
0.0625	1.274429	1.25	0.024429
0.03125	1.130982	1.125	0.005982
0.015625	1.063980	1.0625	0.001480

De esta forma, se concluye que el error de la aproximación por serie de Taylor de primer orden disminuye conforme m se aproxima a 1 y conforme h disminuye. Intuitivamente, esto significa que la serie de Taylor adquiere más exactitud cuando la función que se está aproximando se vuelve más semejante a una línea recta sobre el intervalo de interés. Esto se logra reduciendo el tamaño del intervalo o “enderezando” la función por reducción de m . Es obvio que dicha opción usualmente no está disponible en el mundo real porque las funciones para analizar son, en forma general, dictadas en el contexto del problema físico. En consecuencia, no se tiene control sobre la falta de linealidad y el único recurso consiste en reducir el tamaño del incremento o incluir términos adicionales de la expansión de la serie de Taylor.

4.1.3 Diferenciación numérica

A la ecuación (4.14) se le conoce con un nombre especial en el análisis numérico: *diferencia finita dividida* y generalmente se representa como

$$f'(x_i) = \frac{f(x_{i+1}) - f(x_i)}{x_{i+1} - x_i} + O(x_{i+1} - x_i) \quad (4.17)$$

o

$$f'(x_i) = \frac{\Delta f_i}{h} + O(h) \quad (4.18)$$

donde a Δf_i se le conoce como la *primera diferencia hacia adelante* y a h se le llama el tamaño del paso o incremento; esto es, la longitud del intervalo sobre el cual se realiza la aproximación. Se le llama diferencia “hacia adelante”, porque usa los datos en i e $i + 1$ para estimar la derivada (figura 4.6a). Al término completo $\Delta f/h$ se le conoce como *primer diferencia finita dividida*.

Esta diferencia dividida hacia adelante es sólo una de tantas que pueden desarrollarse a partir de la serie de Taylor para la aproximación de derivadas numéricas. Por ejemplo, las aproximaciones de la primera derivada utilizando *diferencias hacia atrás* o *diferencias centradas* se pueden desarrollar de una manera similar a la de la ecuación

(4.14). Las primeras usan valores en x_{i-1} y x_i (figura 4.6b); mientras que las segundas utilizan valores igualmente espaciados alrededor del punto donde la derivada está estimada (figura 4.6c). Es posible desarrollar aproximaciones más exactas de la primera derivada incluyendo términos de orden más alto de la serie de Taylor. Finalmente, todas las versiones anteriores se pueden desarrollar para derivadas de segundo orden, de tercer orden y de órdenes superiores. En las siguientes secciones se dan resúmenes breves que ilustran cómo se obtienen algunos de estos casos.

Aproximación a la primera derivada con diferencia hacia atrás. La serie de Taylor se expande hacia atrás para calcular un valor anterior sobre la base del valor actual,

$$f(x_{i-1}) = f(x_i) - f'(x_i)h + \frac{f''(x_i)}{2!}h^2 - \dots \quad (4.19)$$

Truncando la ecuación después de la primera derivada y reordenando los términos se obtiene

$$f'(x_i) \cong \frac{f(x_i) - f(x_{i-1})}{h} = \frac{\nabla f_i}{h} \quad (4.20)$$

donde el error es $O(h)$, y a ∇f_i se le conoce como *primera diferencia dividida hacia atrás*. Véase la figura 4.6b para una representación gráfica.

Aproximación a la primera derivada con diferencias centradas. Una tercera forma de aproximar la primera derivada consiste en restar la ecuación (4.19) de la expansión de la serie de Taylor hacia adelante:

$$f(x_{i+1}) = f(x_i) + f'(x_i)h + \frac{f''(x_i)}{2!}h^2 + \dots \quad (4.21)$$

para obtener

$$f(x_{i+1}) - f(x_{i-1}) = 2f'(x_i)h + \frac{2f^{(3)}(x_i)}{3!}h^3 + \dots$$

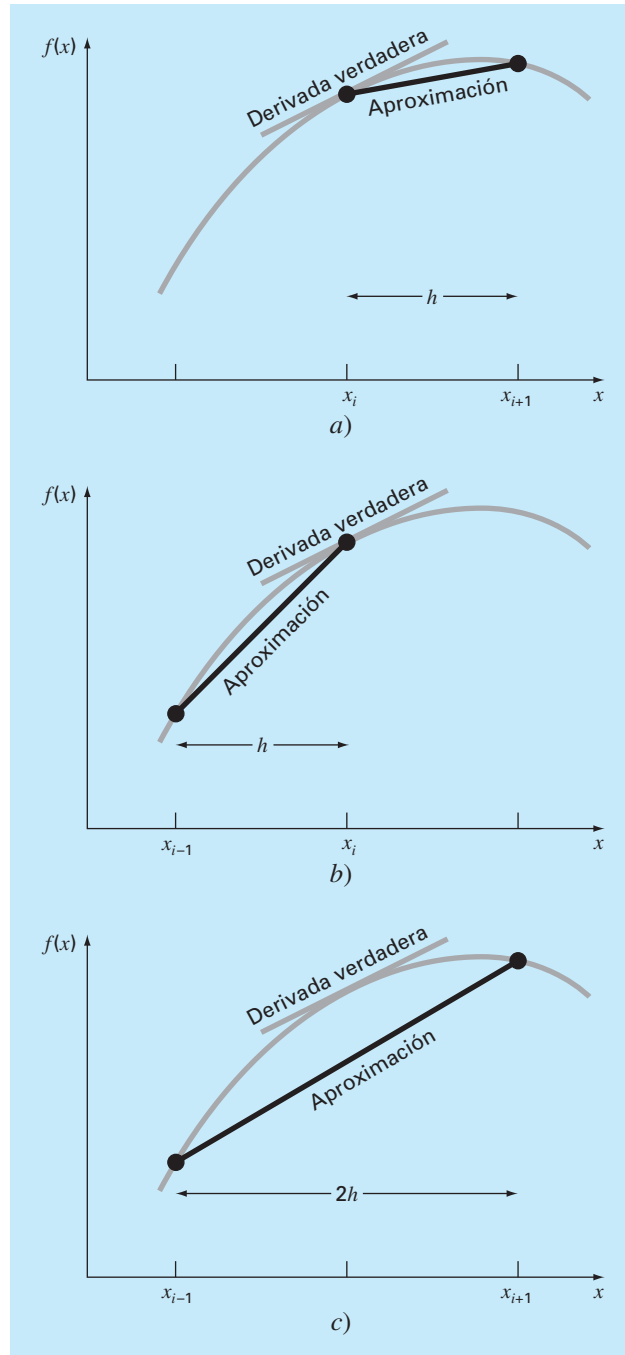
de donde se despeja

$$f'(x_i) = \frac{f(x_{i+1}) - f(x_{i-1}))}{2h} - \frac{f^{(3)}(x_i)}{6}h^2 - \dots$$

o

$$f'(x_i) = \frac{f(x_{i+1}) - f(x_{i-1}))}{2h} - O(h^2) \quad (4.22)$$

La ecuación (4.22) es una representación de las *diferencias centradas* de la primera derivada. Observe que el error de truncamiento es del orden de h^2 en contraste con las aproximaciones hacia adelante y hacia atrás, que fueron del orden de h . Por lo tanto, el análisis de la serie de Taylor ofrece la información práctica de que la diferencia centrada es una representación más exacta de la derivada (figura 4.6c). Por ejemplo, si disminuimos el tamaño del incremento a la mitad, usando diferencias hacia atrás o hacia adelante, el error de truncamiento se reducirá aproximadamente a la mitad; mientras que con diferencias centradas el error se reduciría a la cuarta parte.

**FIGURA 4.6**

Gráfica de aproximaciones con diferencias finitas divididas de la primera derivada: a) hacia delante, b) hacia atrás, c) centrales.

EJEMPLO 4.4 Aproximación de derivadas por diferencias finitas divididas

Planteamiento del problema. Use aproximaciones con diferencias finitas hacia adelante y hacia atrás de $O(h)$ y una aproximación de diferencia centrada de $O(h^2)$ para estimar la primera derivada de

$$f(x) = -0.1x^4 - 0.15x^3 - 0.5x^2 - 0.25x + 1.2$$

en $x = 0.5$ utilizando un incremento de $h = 0.5$. Repita el cálculo con $h = 0.25$. Observe que la derivada se calcula directamente como

$$f'(x) = -0.4x^3 - 0.45x^2 - 1.0x - 0.25$$

y se puede utilizar para calcular el valor verdadero como $f'(0.5) = -0.9125$.

Solución. Para $h = 0.5$, la función se emplea para determinar

$$\begin{array}{ll} x_{i-1} = 0 & f(x_{i-1}) = 1.2 \\ x_i = 0.5 & f(x_i) = 0.925 \\ x_{i+1} = 1.0 & f(x_{i+1}) = 0.2 \end{array}$$

Esos valores sirven para calcular las diferencias divididas hacia adelante [ecuación (4.17)],

$$f'(0.5) \cong \frac{0.2 - 0.925}{0.5} = -1.45 \quad |\varepsilon_t| = 58.9\%$$

la diferencia dividida hacia atrás [ecuación (4.20)],

$$f'(0.5) \cong \frac{0.925 - 1.2}{0.5} = -0.55 \quad |\varepsilon_t| = 39.7\%$$

y la diferencia dividida centrada [ecuación (4.22)],

$$f'(0.5) \cong \frac{0.2 - 1.2}{1.0} = -1.0 \quad |\varepsilon_t| = 9.6\%$$

Para $h = 0.25$,

$$\begin{array}{ll} x_{i-1} = 0.25 & f(x_{i-1}) = 1.10351563 \\ x_i = 0.5 & f(x_i) = 0.925 \\ x_{i+1} = 0.75 & f(x_{i+1}) = 0.63632813 \end{array}$$

que se utilizan para calcular la diferencia dividida hacia adelante,

$$f'(0.5) \cong \frac{0.63632813 - 0.925}{0.25} = -1.155 \quad |\varepsilon_t| = 26.5\%$$

la diferencia dividida hacia atrás,

$$f'(0.5) \cong \frac{0.925 - 1.10351563}{0.25} = -0.714 \quad |\varepsilon_t| = 21.7\%$$

y la diferencia dividida centrada,

$$f'(0.5) \cong \frac{0.63632813 - 1.10351563}{0.5} = -0.934 \quad |\varepsilon_t| = 2.4\%$$

Para ambos tamaños de paso, la aproximación en diferencias centrales es más exacta que las diferencias hacia adelante y hacia atrás. También, como se pronosticó con el análisis de la serie de Taylor, dividiendo a la mitad el incremento, se tiene aproximadamente la mitad del error en las diferencias hacia atrás y hacia adelante y una cuarta parte de error en la diferencia centrada.

Aproximaciones por diferencias finitas para derivadas de orden superior. Además de las primeras derivadas, la expansión en serie de Taylor sirve para obtener estimaciones numéricas de las derivadas de orden superior. Para esto, se escribe la expansión en serie de Taylor hacia adelante para $f(x_{i+2})$ en términos de $f(x_i)$:

$$f(x_{i+2}) = f(x_i) + f'(x_i)(2h) + \frac{f''(x_i)}{2!}(2h)^2 + \dots \quad (4.23)$$

La ecuación (4.21) se multiplica por 2 y se resta de la ecuación (4.23) para obtener

$$f(x_{i+2}) - 2f(x_{i+1}) = -f(x_i) + f''(x_i)h^2 + \dots$$

de donde se despeja

$$f''(x_i) = \frac{f(x_{i+2}) - 2f(x_{i+1}) + f(x_i)}{h^2} + O(h) \quad (4.24)$$

Esta relación se llama la *segunda diferencia finita dividida hacia adelante*. Manipulaciones similares se emplean para obtener la versión hacia atrás

$$f''(x_i) = \frac{f(x_i) - 2f(x_{i-1}) + f(x_{i-2}))}{h^2} + O(h)$$

y la versión centrada

$$f''(x_i) = \frac{f(x_{i+1}) - 2f(x_i) + f(x_{i-1}))}{h^2} + O(h^2)$$

Como fue el caso con las aproximaciones de la primer derivada, el caso centrado es más exacto. Observe también que la versión centrada puede ser expresada en forma alternativa como

$$f''(x_i) \cong \frac{\frac{f(x_{i+1}) - f(x_i)}{h} - \frac{f(x_i) - f(x_{i-1}))}{h}}{h}$$

Así, como la segunda derivada es una derivada de la derivada, la aproximación de la segunda diferencia finita dividida es una diferencia de dos primeras diferencias divididas.

Se volverá al tema de la diferenciación numérica en el capítulo 23. Aquí hemos presentado este tema porque es un muy buen ejemplo de por qué la serie de Taylor es importante en los métodos numéricos. Además, varias de las fórmulas vistas en esta sección se emplearán antes del capítulo 23.

4.2 PROPAGACIÓN DEL ERROR

El propósito de esta sección consiste en estudiar cómo los errores en los números pueden propagarse a través de las funciones matemáticas. Por ejemplo, si se multiplican dos números que tienen errores, nos gustaría estimar el error de este producto.

4.2.1 Funciones de una sola variable

Suponga que se tiene la función $f(x)$ que es dependiente de una sola variable independiente x . Considere que $\tilde{x}$ es una aproximación de x . Por lo tanto, se desearía evaluar el efecto de la discrepancia entre x y $\tilde{x}$ en el valor de la función. Esto es, se desearía estimar

$$\Delta f(\tilde{x}) = |f(x) - f(\tilde{x})|$$

El problema para evaluar $\Delta f(\tilde{x})$ es que se desconoce $f(x)$ porque se desconoce x . Se supera esta dificultad si $\tilde{x}$ está cercana a x y $f(\tilde{x})$ es continua y diferenciable. Si se satisfacen estas condiciones se utiliza una serie de Taylor para calcular $f(x)$ cerca de $f(\tilde{x})$,

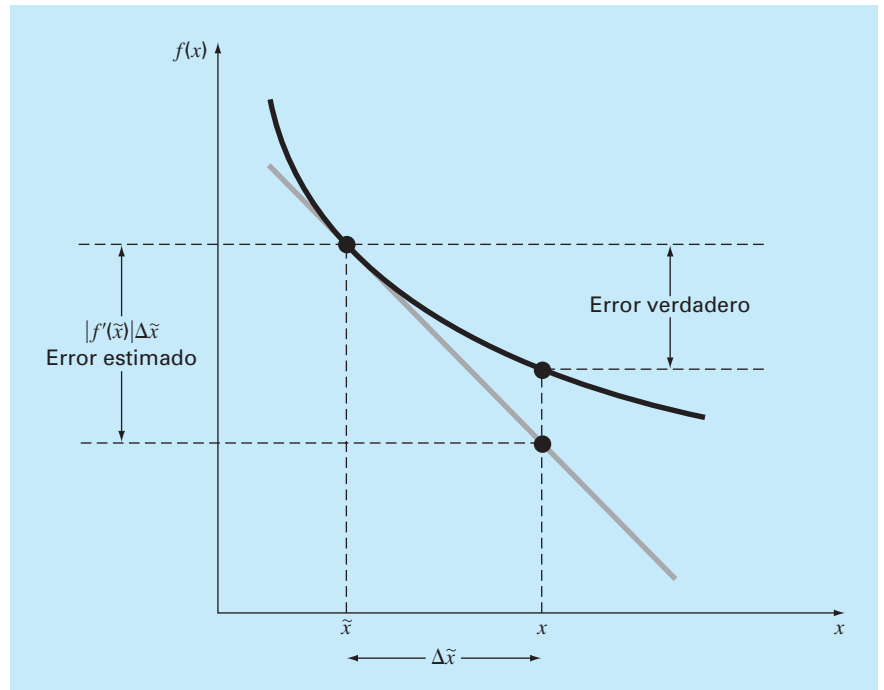
$$f(x) = f(\tilde{x}) + f'(\tilde{x})(x - \tilde{x}) + \frac{f''(\tilde{x})}{2}(x - \tilde{x})^2 + \dots$$

Quitando el segundo término, los de orden superior, y reordenando, se obtiene

$$f(x) - f(\tilde{x}) \cong f'(\tilde{x})(x - \tilde{x})$$

FIGURA 4.7

Representación gráfica de la propagación del error de primer orden.



o

$$\Delta f(\tilde{x}) = |f'(\tilde{x})|(x - \tilde{x}) \quad (4.25)$$

donde $\Delta f(\tilde{x}) = |f(x) - f(\tilde{x})|$ representa una estimación del error de la función y $\Delta \tilde{x} = |x - \tilde{x}|$ representa una estimación del error de x . La ecuación (4.25) proporciona la capacidad de aproximar el error en $f(x)$ dando la derivada de una función y una estimación del error en la variable independiente. La figura 4.7 es una gráfica que representa esta operación.

EJEMPLO 4.5 Propagación del error en una función de una variable

Planteamiento del problema. Dado un valor de $\tilde{x} = 2.5$ con un error $\Delta \tilde{x} = 0.01$, estime el error resultante en la función $f(x) = x^3$.

Solución. Con la ecuación (4.25),

$$\Delta f(\tilde{x}) \cong 3(2.5)^2(0.01) = 0.1875$$

Ya que $f(2.5) = 15.625$, se pronostica que

$$f(2.5) = 15.625 \pm 0.1875$$

o que el valor verdadero se encuentra entre 15.4375 y 15.8125. De hecho, si x fuera realmente 2.49, la función se evaluaría como 15.4382, y si x fuera 2.51, el valor de la función sería 15.8132. Para este caso, el análisis del error de primer orden proporciona una estimación adecuada del error verdadero.

4.2.2 Funciones de más de una variable

El enfoque anterior puede generalizarse a funciones que sean dependientes de más de una variable independiente, lo cual se realiza con una versión para varias variables de la serie de Taylor. Por ejemplo, si se tiene una función de dos variables independientes, u y v , la serie de Taylor se escribe como

$$\begin{aligned} f(u_{i+1}, v_{i+1}) &= f(u_i, v_i) + \frac{\partial f}{\partial u}(u_{i+1} - u_i) + \frac{\partial f}{\partial v}(v_{i+1} - v_i) \\ &\quad + \frac{1}{2!} \left[\frac{\partial^2 f}{\partial u^2}(u_{i+1} - u_i)^2 + 2 \frac{\partial^2 f}{\partial u \partial v}(u_{i+1} - u_i)(v_{i+1} - v_i) \right. \\ &\quad \left. + \frac{\partial^2 f}{\partial v^2}(v_{i+1} - v_i)^2 \right] + \dots \end{aligned} \quad (4.26)$$

donde todas las derivadas parciales se evalúan en el punto base i . Si no se consideran todos los términos de segundo orden y de orden superior, de la ecuación (4.26) puede despejarse

$$\Delta f(\tilde{u}, \tilde{v}) = \left| \frac{\partial f}{\partial u} \right| \Delta \tilde{u} + \left| \frac{\partial f}{\partial v} \right| \Delta \tilde{v}$$

donde $\Delta \tilde{u}$ y $\Delta \tilde{v}$ son estimaciones del error en u y v , respectivamente.

Para n variables independientes $\tilde{x}_1, \tilde{x}_2, \dots, \tilde{x}_n$ teniendo errores $\Delta\tilde{x}_1, \Delta\tilde{x}_2, \dots, \Delta\tilde{x}_n$ se satisface la siguiente relación general:

$$\Delta f(\tilde{x}_1, \tilde{x}_2, \dots, \tilde{x}_n) \cong \left| \frac{\partial f}{\partial x_1} \right| \Delta\tilde{x}_1 + \left| \frac{\partial f}{\partial x_2} \right| \Delta\tilde{x}_2 + \dots + \left| \frac{\partial f}{\partial x_n} \right| \Delta\tilde{x}_n \quad (4.27)$$

EJEMPLO 4.6 Propagación del error en una función con varias variables

Planteamiento del problema. La deflexión y de la punta de un mástil en un bote de vela es

$$y = \frac{FL^4}{8EI}$$

donde F = una carga lateral uniforme (lb/ft), L = altura (ft), E = el módulo de elasticidad (lb/ft²), e I = el momento de inercia (ft⁴). Estime el error en y , dados los siguientes datos:

$$\tilde{F} = 50 \text{ lb/ft}$$

$$\Delta\tilde{F} = 2 \text{ lb/ft}$$

$$\tilde{L} = 30 \text{ ft}$$

$$\Delta\tilde{L} = 0.1 \text{ ft}$$

$$\tilde{E} = 1.5 \times 10^8 \text{ lb/ft}^2$$

$$\Delta\tilde{E} = 0.01 \times 10^8 \text{ lb/ft}^2$$

$$\tilde{I} = 0.06 \text{ ft}^4$$

$$\Delta\tilde{I} = 0.0006 \text{ ft}^4$$

Solución. Empleando la ecuación (4.27) se tiene

$$\Delta y(\tilde{F}, \tilde{L}, \tilde{E}, \tilde{I}) = \left| \frac{\partial y}{\partial F} \right| \Delta\tilde{F} + \left| \frac{\partial y}{\partial L} \right| \Delta\tilde{L} + \left| \frac{\partial y}{\partial E} \right| \Delta\tilde{E} + \left| \frac{\partial y}{\partial I} \right| \Delta\tilde{I}$$

o

$$\Delta y(\tilde{F}, \tilde{L}, \tilde{E}, \tilde{I}) \cong \frac{\tilde{L}^4}{8\tilde{E}\tilde{I}} \Delta\tilde{F} + \frac{\tilde{F}\tilde{L}^3}{2\tilde{E}\tilde{I}} \Delta\tilde{L} + \frac{\tilde{F}\tilde{L}^4}{8\tilde{E}^2\tilde{I}} \Delta\tilde{E} + \frac{\tilde{F}\tilde{L}^4}{8\tilde{E}\tilde{I}^2} \Delta\tilde{I}$$

Al sustituir los valores apropiados se tiene

$$\Delta y = 0.0225 + 0.0075 + 0.00375 + 0.005625 = 0.039375$$

Por lo tanto, $y = 0.5625 \pm 0.039375$. En otras palabras y está entre 0.523125 y 0.601875 ft. La validez de estas estimaciones se verifica sustituyendo los valores extremos para las variables dentro de la ecuación que genera un mínimo exacto de

$$y_{\min} = \frac{48(29.9)^4}{8(1.51 \times 10^8)0.0606} = 0.52407$$

y

$$y_{\max} = \frac{52(30.1)^4}{8(1.49 \times 10^8)0.0594} = 0.60285$$

Así, las estimaciones de primer orden están razonablemente cercanas de los valores exactos.

La ecuación (4.27) se utiliza para definir relaciones en la propagación de errores con las operaciones matemáticas comunes. Los resultados se resumen en la tabla 4.3. Se deja el desarrollo de estas fórmulas como un ejercicio de tarea.

4.2.3 Estabilidad y condición

La *condición* de un problema matemático relaciona su sensibilidad con los cambios en los datos de entrada. Se dice que un cálculo es *numéricamente inestable* si la inexactitud de los valores de entrada se aumenta considerablemente por el método numérico.

Estas ideas pueden estudiarse usando una serie de Taylor de primer orden

$$f(x) = f(\tilde{x}) + f'(\tilde{x})(x - \tilde{x})$$

Esta relación se emplea para estimar el *error relativo* de $f(x)$ como en

$$\frac{f(x) - f(\tilde{x})}{f(x)} \approx \frac{f'(\tilde{x})(x - \tilde{x})}{f(\tilde{x})}$$

El *error relativo* de x está dado por

$$\frac{x - \tilde{x}}{\tilde{x}}$$

TABLA 4.3 El error estimado relacionado con las operaciones matemáticas comunes usando números inexactos $\tilde{u}$ y $\tilde{v}$.

Operación		Error estimado
Adición	$\Delta(\tilde{u} + \tilde{v})$	$\Delta\tilde{u} + \Delta\tilde{v}$
Sustracción	$\Delta(\tilde{u} - \tilde{v})$	$\Delta\tilde{u} + \Delta\tilde{v}$
Multipliación	$\Delta(\tilde{u} \times \tilde{v})$	$ \tilde{u} \Delta\tilde{v} + \tilde{v} \Delta\tilde{u}$
División	$\Delta\left(\frac{\tilde{u}}{\tilde{v}}\right)$	$\frac{ \tilde{u} \Delta\tilde{v} + \tilde{v} \Delta\tilde{u}}{ \tilde{v} ^2}$

Un *número de condición* puede definirse como la razón entre estos errores relativos

$$\text{Número de condición} = \frac{\tilde{x} f'(\tilde{x})}{f(\tilde{x})} \quad (4.28)$$

El número de condición proporciona una medida de qué tanto una inexactitud de x se aumenta por $f(x)$. Un valor de 1 nos indica que el error relativo de la función es idéntico al error relativo de x . Un valor mayor que 1 nos señala que el error relativo se amplifica; mientras que para un valor menor que 1 nos dice que se atenúa. En funciones con valores muy grandes se dice que están *mal condicionadas*. Cualquier combinación de los factores en la ecuación (4.28), que aumente el valor numérico del número de condición, tendería a aumentar inexactitudes al calcular $f(x)$.

EJEMPLO 4.7 Número de condición

Planteamiento del problema. Calcule e interprete el número de condición para

$$f(x) = \tan x \quad \text{para } \tilde{x} = \frac{\pi}{2} + 0.1 \left(\frac{\pi}{2} \right)$$

$$f(x) = \tan x \quad \text{para } \tilde{x} = \frac{\pi}{2} + 0.01 \left(\frac{\pi}{2} \right)$$

Solución. El número de condición se calcula como

$$\text{Número de condición} = \frac{\tilde{x}(1 / \cos^2 x)}{\tan \tilde{x}}$$

Para $\tilde{x} = \pi/2 + 0.1(\pi/2)$

$$\text{Número de condición} = \frac{1.7279(40.86)}{-6.314} = -11.2$$

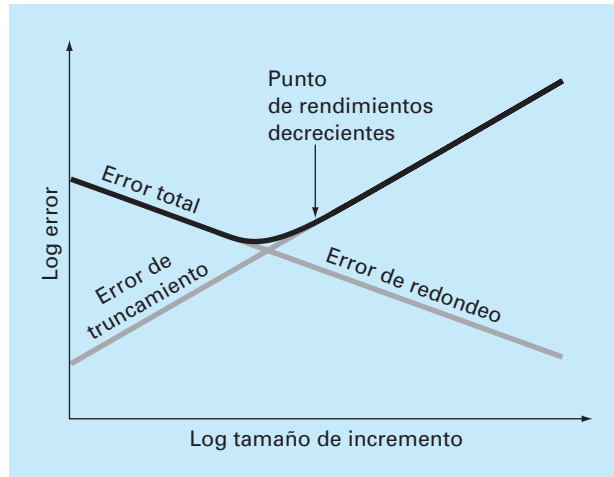
Así, la función está mal condicionada. Para $\tilde{x} = \pi/2 + 0.01(\pi/2)$, esta situación es aún peor:

$$\text{Número de condición} = \frac{1.5865(4\,053)}{-63.66} = -101$$

En este caso, la causa principal del mal condicionamiento parece ser la derivada. Esto tiene sentido, ya que en la vecindad de $\pi/2$, la tangente tiende tanto a infinito positivo como a infinito negativo.

4.3 ERROR NUMÉRICO TOTAL

El *error numérico total* es la suma de los errores de truncamiento y de redondeo. En general, la única forma para minimizar los errores de redondeo consiste en incrementar el número de cifras significativas en la computadora. Adicionalmente, hemos notado que el error de redondeo *aumentará* debido a la cancelación por resta o debido a que en el análisis aumente el número de cálculos. En contraste, el ejemplo 4.4 demuestra que el error de truncamiento se reduce disminuyendo el tamaño del incremento. Como una disminución al tamaño del incremento puede llevar a una cancelación por resta o a un incremento de los cálculos, los errores de truncamiento *disminuyen* conforme los errores de redondeo se *incrementan*. En consecuencia, se debe afrontar el siguiente dilema: la estrategia para disminuir un componente del error total conduce a un incremento en el otro componente. En un cálculo, se podría disminuir el tamaño del incremento para minimizar los errores de truncamiento únicamente para descubrir que el error de redondeo empieza a dominar la solución y ¡el error total crece! Así, el remedio empieza a ser un problema (figura 4.8). Es un reto determinar el tamaño del incremento apropiado para

**FIGURA 4.8**

Representación gráfica de las relaciones entre el error de redondeo y el error de truncamiento que juegan un papel importante en el curso de métodos numéricos. Se presenta el punto de regreso disminuido, donde el error de redondeo no muestra los beneficios de la reducción del tamaño del incremento.

un cálculo en particular. Se deberá seleccionar un tamaño de incremento grande con la finalidad de disminuir la cantidad de cálculos y errores de redondeo para no tener como consecuencia grandes errores de truncamiento. Si el error total es como se muestra en la figura 4.8, el reto es identificar un punto llamado de regreso disminuido donde los errores de redondeo no muestran los beneficios de la reducción del tamaño del incremento.

En casos reales, sin embargo, tales situaciones son relativamente poco comunes, porque muchas computadoras utilizan suficientes cifras significativas para que los errores de redondeo no predominen. Aunque, algunas veces estos errores ocurren y surge una clase de “principio numérico de incertidumbre” que da un límite absoluto sobre la exactitud que puede obtenerse usando ciertos métodos numéricos computarizados.

4.3.1 Control de errores numéricos

En la mayoría de los casos prácticos, no se conoce el error exacto asociado con el método numérico. Con excepción, claro está, de cuando obtenemos la solución exacta que vuelve innecesaria la aproximación numérica. Por lo tanto, en la mayoría de las aplicaciones en ingeniería debe tenerse algún estimado del error en los cálculos.

No hay una forma sistemática ni general para evaluar el error numérico en todos los problemas. En muchos casos, la estimación del error se basa en la experiencia y en el buen juicio del ingeniero.

Aunque el análisis de error es hasta cierto punto un arte, se sugieren varios lineamientos prácticos de cálculo: lo primero, y principal, implica tratar de evitar la resta de dos números casi iguales. Cuando esto ocurre, casi siempre se pierden cifras significativas. Algunas veces puede reordenarse o reformularse el problema para evitar la cancelación por resta. Y si esto no es posible, se utiliza la aritmética de precisión extendida.

Además, cuando se suman o se restan números, es mejor ordenarlos y trabajar primero con los números más pequeños, lo cual evita perder cifras significativas.

Más allá de estas sugerencias de cálculo, se puede intentar predecir el error numérico total usando formulaciones teóricas. La serie de Taylor es la primera herramienta de análisis tanto para el error de truncamiento como para el error de redondeo. Varios ejemplos se han presentado en este capítulo. La predicción del error numérico total es muy complicada para, incluso, un problema de tamaño moderado, y tiende a resultar pesimista. Por lo tanto, únicamente se utiliza para tareas a pequeña escala.

La tendencia es avanzar con los cálculos numéricos e intentar estimar la exactitud de sus resultados. Esto algunas veces se puede hacer observando si los resultados satisfacen alguna condición o ecuación de prueba. O se pueden sustituir los resultados en la ecuación original para verificar si se satisface dicha ecuación.

Por último, usted debería estar preparado para realizar experimentos numéricos que aumenten su conocimiento de los errores de cálculo y de posibles problemas mal condicionados. Tales experimentos pueden consistir en repetir los cálculos con diferentes tamaños de incremento o método, y comparar los resultados. Llega a emplearse un análisis sensitivo para observar cómo la solución cambia cuando se modifican los parámetros del modelo o los valores de entrada. Es factible probar distintos algoritmos numéricos que tengan diferente fundamento matemático, que se basan en distintas estrategias de cálculo o que tengan diferentes características de convergencia y de estabilidad.

Cuando los resultados del cálculo numérico son extremadamente críticos y pueden implicar la pérdida de vidas humanas o tener severas repercusiones económicas, es apropiado tomar precauciones especiales. Esto implicaría el uso de dos o más técnicas independientes para resolver el mismo problema y luego comparar los resultados.

El papel de los errores será un tópico de preocupación y análisis en todas las secciones de este libro. Se dejan estas investigaciones en secciones específicas.

4.4 EQUIVOCACIONES, ERRORES DE FORMULACIÓN E INCERTIDUMBRE EN LOS DATOS

Aunque las siguientes fuentes de error no están directamente relacionadas con la mayor parte de los métodos numéricos de este libro, en algunas ocasiones llegan a tener un gran impacto en el éxito al realizar un modelado. Por lo tanto, se deben tener siempre en cuenta cuando se apliquen técnicas numéricas en el contexto de los problemas del mundo real.

4.4.1 Errores por equivocación

A todos nos son familiares los errores por negligencia o por equivocación. En los primeros años de las computadoras, los resultados numéricos erróneos algunas veces se atribuían a las fallas de la propia computadora. En la actualidad esta fuente de error es muy improbable y la mayor parte de las equivocaciones se atribuyen a fallas humanas.

Las equivocaciones llegan a ocurrir a cualquier nivel del proceso de modelación matemática y pueden contribuir con todas las otras componentes del error. Es posible evitarlos únicamente con un sólido conocimiento de los principios fundamentales y mediante el cuidado con el que se enfoque y diseñe la solución del problema.

Las equivocaciones por lo general se pasan por alto en el estudio de un método numérico. Esto se debe sin duda al hecho de que los errores son, hasta cierto punto,

inevitables. No obstante, recuerde que hay varias formas con las cuales se puede minimizar su aparición. En particular, los buenos hábitos de programación que se esbozaron en el capítulo 2 son muy útiles para disminuir las equivocaciones. Además, hay formas simples de verificar si un método numérico funciona correctamente. A lo largo del texto, se estudian algunas formas de verificar los resultados de un cálculo numérico.

4.4.2 Errores de formulación

Los *errores de formulación* o *de modelo* pueden atribuirse al sesgo que implica un modelo matemático incompleto. Un ejemplo de un error de formulación insignificante es el hecho de que la segunda ley de Newton no toma en cuenta los efectos relativísticos. Esto no desvirtúa la validez de la solución del ejemplo 1.1, ya que estos errores son mínimos en las escalas de tiempo y espacio asociadas con el problema de la caída del paracaidista.

Sin embargo, suponga que la resistencia del aire no es linealmente proporcional a la velocidad de caída, como en la ecuación (1.7), sino que está en función del cuadrado de la velocidad. Si éste fuera el caso, las soluciones analíticas y numéricas obtenidas en el primer capítulo serían falsas debido al error en la formulación. En algunas aplicaciones de ingeniería del libro se presentan consideraciones adicionales a los errores de formulación. Se debe estar consciente de estos problemas y darse cuenta de que, si se está usando un modelo deficiente, ningún método numérico generará los resultados adecuados.

4.4.3 Incertidumbre en los datos

Algunas veces se introducen errores en un análisis debido a la incertidumbre en los datos físicos obtenidos, sobre los que se basa el modelo. Por ejemplo, suponga que se desea probar el modelo de la caída del paracaidista, haciendo que un individuo salte repetidas veces, midiendo su velocidad después de un intervalo de tiempo específico. Sin duda, se asociaría cada medición con una incertidumbre, ya que el paracaidista caerá con más rapidez en unos saltos que en otros. Estos errores pueden mostrar inexactitud e imprecisión. Si los instrumentos constantemente subevalúan o sobrevalúan las mediciones de la velocidad, se estará tratando con un instrumento inexacto o desviado. Por otro lado, si las medidas son aleatoriamente grandes y pequeñas, entonces se trata de una cuestión de precisión.

Los errores de medición se pueden cuantificar resumiendo los datos con uno o más estadísticos, que den tanta información como sea posible, respecto a características específicas de los datos. Tales estadísticos descriptivos a menudo se seleccionan para obtener 1. la posición del centro de la distribución de los datos y 2. el grado de dispersión de los datos. Como tales, estos estadísticos ofrecen una medida de la desviación e imprecisión, respectivamente. En la parte cinco se regresa el tema de caracterización de incertidumbre de datos.

Aunque se debe estar consciente de los errores por equivocación, de los errores de formulación y de la incertidumbre en los datos, los métodos numéricos utilizados para construir modelos pueden estudiarse, en la mayoría de los casos, en forma independiente de estos errores. Por consiguiente, en la mayor parte de este libro se supondrá que no hay errores por equivocaciones, que el modelo es adecuado y que se está trabajando sin errores en las mediciones de los datos. En estas condiciones es posible estudiar los métodos numéricos sin complicaciones.

PROBLEMAS

4.1 La serie infinita

$$e^x = 1 + x + \frac{x^2}{2} + \frac{x^3}{3!} + \cdots + \frac{x^n}{n!}$$

se utiliza para aproximar e^x .

- Muestre que la expansión en serie de Maclaurin es un caso especial de la expansión en la serie de Taylor [ecuación (4.7)] con $x_i = 0$ y $h = x$.
- Use la serie de Taylor para estimar $f(x) = e^{-x}$ en $x_{i+1} = 1$ para $x_i = 0.25$. Emplee versiones de cero, primero, segundo y tercer orden, y calcule $|e_i|$ para cada caso.

4.2 La expansión en serie de Maclaurin para $\cos x$ es

$$\cos x = 1 - \frac{x^2}{2} + \frac{x^4}{4!} - \frac{x^6}{6!} + \frac{x^8}{8!} - \cdots$$

Iniciando con el primer término $\cos x = 1$, agregue los términos uno a uno para estimar $\cos(\pi/4)$. Después de que agregue cada uno de los términos, calcule los errores relativos porcentuales exactos y aproximados. Use una calculadora para determinar el valor exacto. Agregue términos hasta que el valor absoluto del error aproximado se encuentre dentro de cierto criterio de error, considerando dos cifras significativas.

4.3 Repita los cálculos del problema 4.2, pero ahora usando la expansión de la serie de Maclaurin para $\sin x$,

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \cdots$$

para evaluar el $\sin(\pi/4)$.

4.4 Emplee la expansión de la serie de Taylor de cero hasta tercer orden para predecir $f(2)$ si

$$f(x) = 25x^3 - 6x^2 + 7x - 88$$

usando como punto base $x = 1$. Calcule el error relativo porcentual verdadero ε_r para cada aproximación.

4.5 Use la expansión de la serie de Taylor de cero al cuarto orden para estimar $f(3)$ si $f(x) = \ln x$ utilizando $x = 1$ como punto base. Calcule el error relativo porcentual ε_r para cada aproximación. Analice los resultados.

4.6 Utilice aproximaciones en diferencias de $O(h)$ hacia atrás y hacia adelante y una aproximación de diferencia central de $O(h^2)$ para estimar la primera derivada de la función mencionada en el problema 4.4. Evalúe la derivada en $x = 2$ usando un tamaño del incremento 0.2. Compare los resultados con el valor exacto de las derivadas. Interprete los resultados considerando el término residual de la expansión en la serie de Taylor.

4.7 Con la aproximación en diferencias centrales de $O(h^2)$ estime la segunda derivada de la función examinada en el problema 4.4. Realice la evaluación para $x = 2$ usando un tamaño de incremento 0.25 y 0.125. Compare lo estimado con el valor exacto de

la segunda derivada. Interprete sus resultados considerando el término residual de la expansión en la serie de Taylor.

4.8 Recuerde que la velocidad de caída del paracaidista puede calcularse con [ecuación (1.10)]

$$v(t) = \frac{gm}{c}(1 - e^{-(c/m)t})$$

Use un análisis de error de primer orden para estimar el error de v para $t = 6$, si $g = 9.8$ y $m = 50$, pero $c = 12.5 \pm 2$.

4.9 Repita el problema 4.8 con $g = 9.8$, $t = 6$, $c = 12.5 \pm 1.5$ y $m = 50 \pm 2$.

4.10 La ley de Stefan-Boltzmann se utiliza para estimar la velocidad de cambio de la energía H para una superficie, esto es,

$$H = Ae\sigma T^4$$

donde H está en watts, A = área de la superficie (m^2), e = la emisividad que caracteriza la propiedad de emisión de la superficie (adimensional), σ = una constante universal llamada constante de Stefan-Boltzmann ($= 5.67 \times 10^{-8} \text{ W m}^{-2} \text{ K}^{-4}$) y T = temperatura absoluta (K). Determine el error de H para una placa de acero con $A = 0.15 \text{ m}^2$, $e = 0.90$ y $T = 650 \pm 20$. Compare los resultados con el error exacto. Repita los cálculos pero con $T = 650 \pm 40$. Interprete los resultados.

4.11 Repita el problema 4.10, pero para una esfera de cobre con radio $= 0.15 \pm 0.01 \text{ m}$, $e = 0.90 \pm 0.05$ y $T = 550 \pm 20$.

4.12 Evalúe e interprete los números de condición para

$$a) \quad f(x) = \sqrt{|x-1|} + 1 \quad \text{para } x = 1.0001$$

$$b) \quad f(x) = e^{-x} \quad \text{para } x = 9$$

$$c) \quad f(x) = \sqrt{x^2 + 1} - x \quad \text{para } x = 300$$

$$d) \quad f(x) = \frac{e^x - 1}{x} \quad \text{para } x = 0.001$$

$$e) \quad f(x) = \frac{\sin x}{1 + \cos x} \quad \text{para } x = 1.0001\pi$$

4.13 Empleando las ideas de la sección 4.2, muestre las relaciones de la tabla 4.3.

4.14 Muestre que la ecuación (4.4) es exacta para todos los valores de x , si $f(x) = ax^2 + bx + c$.

4.15 La fórmula de Manning para un canal rectangular se escribe como

$$Q = \frac{1}{n} \frac{(BH)^{5/3}}{(B + 2H)^{2/3}} S^{1/2}$$

donde Q = flujo (m^3/s), n = coeficiente de rugosidad, B = ancho (m), H = profundidad (m) y S = pendiente. Aplique la fórmula para un arroyo donde se conoce que el ancho = 20 m y la profun-

dad = 0.3 m. Por desgracia conocemos el coeficiente de rugosidad y la pendiente con una precisión de sólo $\pm 10\%$. Es decir, la rugosidad tiene un valor de 0.03 con un rango de 0.027 a 0.033, y la pendiente es 0.0003 con un rango de 0.00027 a 0.00033. Use un análisis de error de primer orden para determinar la sensibilidad en la predicción del flujo para cada uno de esos dos factores. ¿Cuál se debería intentar medir para una mejor precisión?

4.16 Si $|x| < 1$, se sabe que

$$\frac{1}{1-x} = 1 + x + x^2 + x^3 + \dots$$

Repita el problema 4.2 para esta serie con $x = 0.1$.

4.17 Un misil sale de la Tierra con una velocidad inicial v_0 formando con la vertical un ángulo ϕ_0 como se muestra en la figura

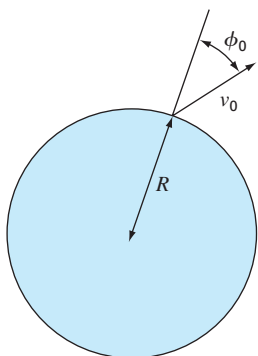


Figura P4.17

P4.17. La altitud máxima deseada es αR donde R es el radio de la Tierra. Usando las leyes de la mecánica se demuestra que

$$\sin \phi_0 = (1 + \alpha) \sqrt{1 - \frac{\alpha}{1 + \alpha} \left(\frac{v_e}{v_0} \right)^2}$$

donde v_e es la velocidad de escape del misil. Se quiere disparar el misil y alcanzar la velocidad máxima proyectada con una exactitud de $\pm 1\%$. Determine el rango de valores de f_0 si $v_e/v_0 = 2$ y $\alpha = 0.2$.

4.18 Para calcular las coordenadas espaciales de un planeta tenemos que resolver la función

$$f(x) = x - 1 - 0.5 \sin x$$

Sea $a = x_i = \pi/2$ en el intervalo $[0, \pi]$ el punto base. Determine la expansión de la serie de Taylor de orden superior que da un error máximo de 0.015 en el intervalo dado. El error es igual al valor absoluto de la diferencia entre la función dada y la expansión de la serie de Taylor especificada. (Sugerencia: Resuelva gráficamente.)

4.19 Considere la función $f(x) = x^3 - 2x + 4$ en el intervalo $[-2, 2]$ con $h = 0.25$. Use las aproximaciones en diferencias finitas hacia adelante, hacia atrás y centrada para la primera y segunda derivadas, e ilustre gráficamente qué aproximación es más exacta. Grafique las tres aproximaciones a la primera derivada por diferencias finitas, junto con los valores exactos, y haga lo mismo con la segunda derivada.

CAPÍTULO 6

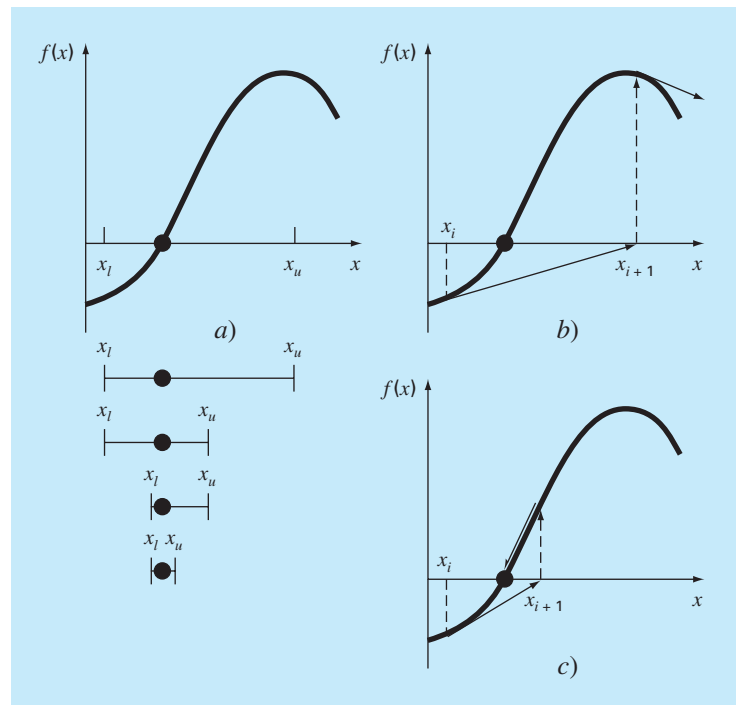
Métodos abiertos

En los métodos cerrados del capítulo anterior la raíz se encuentra dentro de un intervalo predeterminado por un límite inferior y otro superior. La aplicación repetida de estos métodos siempre genera aproximaciones cada vez más cercanas a la raíz. Se dice que tales métodos son *convergentes* porque se acercan progresivamente a la raíz a medida que se avanza en el cálculo (figura 6.1a).

En contraste, los *métodos abiertos* descritos en este capítulo se basan en fórmulas que requieren únicamente de un solo valor de inicio x o que empiecen con un par de ellos, pero que no necesariamente encierran la raíz. Éstos, algunas veces *divergen* o se alejan de la raíz verdadera a medida que se avanza en el cálculo (figura 6.1b). Sin embargo, cuando los métodos abiertos convergen (figura 6.1c), en general lo hacen mucho más rápido que los métodos cerrados. Empecemos el análisis de los métodos abiertos con una versión simple que es útil para ilustrar su forma general y también para demostrar el concepto de convergencia.

FIGURA 6.1

Representación gráfica de las diferencias fundamentales entre los métodos a) cerrados, b) y c) los métodos abiertos para el cálculo de raíces. En a) se ilustra el método de bisección, donde la raíz está contenida dentro del intervalo dado por x_l y x_u . En contraste, en los métodos abiertos, ilustrados en b) y c), se utiliza una fórmula para dirigirse de x_i a x_{i+1} , con un esquema iterativo. Así, el método puede b) diverger o c) converger rápidamente, dependiendo de los valores iniciales.



6.1 ITERACIÓN SIMPLE DE PUNTO FIJO

Como se dijo antes, los métodos abiertos emplean una fórmula para predecir la raíz. Esta fórmula puede desarrollarse como una *iteración simple de punto fijo* (también llamada iteración de un punto o sustitución sucesiva o método de punto fijo), al arreglar la ecuación $f(x) = 0$ de tal modo que x esté del lado izquierdo de la ecuación:

$$x = g(x) \quad (6.1)$$

Esta transformación se realiza mediante operaciones algebraicas o simplemente sumando x a cada lado de la ecuación original. Por ejemplo,

$$x^2 - 2x + 3 = 0$$

se arregla para obtener

$$x = \frac{x^2 + 3}{2}$$

mientras que $\text{sen } x = 0$ puede transformarse en la forma de la ecuación (6.1) sumando x a ambos lados para obtener

$$x = \text{sen } x + x$$

La utilidad de la ecuación (6.1) es que proporciona una fórmula para predecir un nuevo valor de x en función del valor anterior de x . De esta manera, dado un valor inicial para la raíz x_i , la ecuación (6.1) se utiliza para obtener una nueva aproximación x_{i+1} , expresada por la fórmula iterativa

$$x_{i+1} = g(x_i) \quad (6.2)$$

Como en otras fórmulas iterativas de este libro, el error aproximado de esta ecuación se calcula usando el error normalizado [ecuación (3.5)]:

$$\varepsilon_a = \left| \frac{x_{i+1} - x_i}{x_{i+1}} \right| 100\%$$

EJEMPLO 6.1 Iteración simple de punto fijo

Planteamiento del problema. Use una iteración simple de punto fijo para localizar la raíz de $f(x) = e^{-x} - x$.

Solución. La función se puede separar directamente y expresarse en la forma de la ecuación (6.2) como

$$x_{i+1} = e^{-x_i}$$

Empezando con un valor inicial $x_0 = 0$, se aplica esta ecuación iterativa para calcular

i	x_i	ε_a (%)	ε_t (%)
0	0		100.0
1	1.000000	100.0	76.3
2	0.367879	171.8	35.1
3	0.692201	46.9	22.1
4	0.500473	38.3	11.8
5	0.606244	17.4	6.89
6	0.545396	11.2	3.83
7	0.579612	5.90	2.20
8	0.560115	3.48	1.24
9	0.571143	1.93	0.705
10	0.564879	1.11	0.399

De esta manera, se puede observar que cada iteración se acerca cada vez más al valor aproximado al valor verdadero de la raíz: 0.56714329.

6.1.1 Convergencia

Note que el error relativo porcentual verdadero en cada iteración del ejemplo 6.1 es proporcional (por un factor de 0.5 a 0.6) al error de la iteración anterior. Esta propiedad, conocida como *convergencia lineal*, es característica de la iteración simple de punto fijo.

Además de la “velocidad” de convergencia, en este momento debemos enfatizar la “posibilidad” de convergencia. Los conceptos de convergencia y divergencia se pueden ilustrar gráficamente. Recuerde que en la sección 5.1 se graficó una función para visualizar su estructura y comportamiento (ejemplo 5.1). Ese método se emplea en la figura 6.2a para la función $f(x) = e^{-x} - x$. Un método gráfico alternativo consiste en separar la ecuación en dos partes, de esta manera

$$f_1(x) = f_2(x)$$

Entonces las dos ecuaciones

$$y_1 = f_1(x) \tag{6.3}$$

y

$$y_2 = f_2(x) \tag{6.4}$$

se grafican por separado (figura 6.2b). Así, los valores de x correspondientes a las intersecciones de estas dos funciones representan las raíces de $f(x) = 0$.

EJEMPLO 6.2 El método gráfico de las dos curvas

Planteamiento del problema. Separe la ecuación $e^{-x} - x = 0$ en dos partes y determine su raíz en forma gráfica.

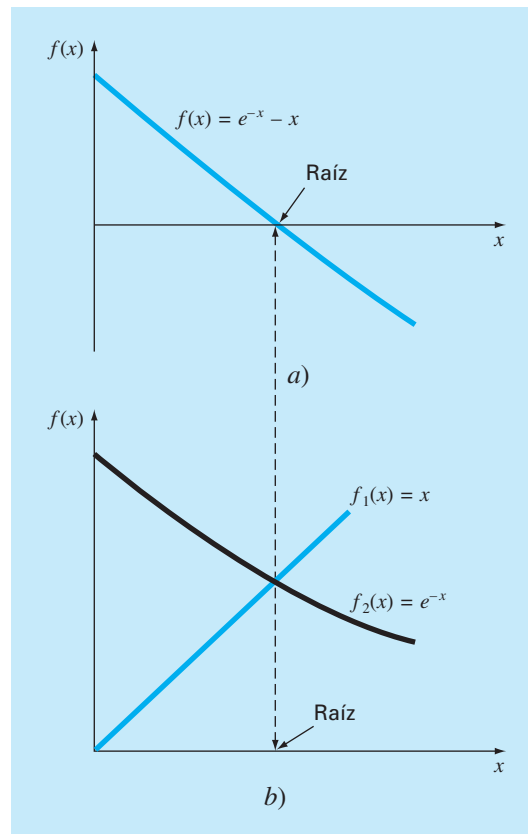
Solución. Reformule la ecuación como $y_1 = x$ y $y_2 = e^{-x}$. Al tabular las funciones se obtienen los siguientes valores:

x	y_1	y_2
0.0	0.0	1.000
0.2	0.2	0.819
0.4	0.4	0.670
0.6	0.6	0.549
0.8	0.8	0.449
1.0	1.0	0.368

Estos puntos se grafican en la figura 6.2b. La intersección de las dos curvas indica una raíz estimada de aproximadamente $x = 0.57$, que corresponde al valor donde la curva de la figura 6.2a cruza el eje x .

FIGURA 6.2

Dos métodos gráficos para determinar la raíz de $f(x) = e^{-x} - x$. a) La raíz como un punto donde la función cruza el eje x ; b) la raíz como la intersección de las dos funciones componentes.



El método de las dos curvas también se utiliza para ilustrar la convergencia y divergencia de la iteración de punto fijo. En primer lugar, la ecuación (6.1) se reexpresa como un par de ecuaciones $y_1 = x$ y $y_2 = g(x)$. Estas dos ecuaciones se grafican por separado. Entonces, las raíces de $f(x) = 0$ corresponden al valor de la abscisa para la intersección de las dos curvas. En la figura 6.3 se grafican la función $y_1 = x$ y cuatro formas diferentes de la función $y_2 = g(x)$.

En el primer caso (figura 6.3a), el valor inicial x_0 sirve para determinar el punto $[x_0, g(x_0)]$ correspondiente a la curva y_2 . El punto (x_1, x_1) se encuentra moviéndose horizontalmente a la izquierda hasta la curva y_1 . Estos movimientos son el equivalente a la primera iteración en el método de punto fijo:

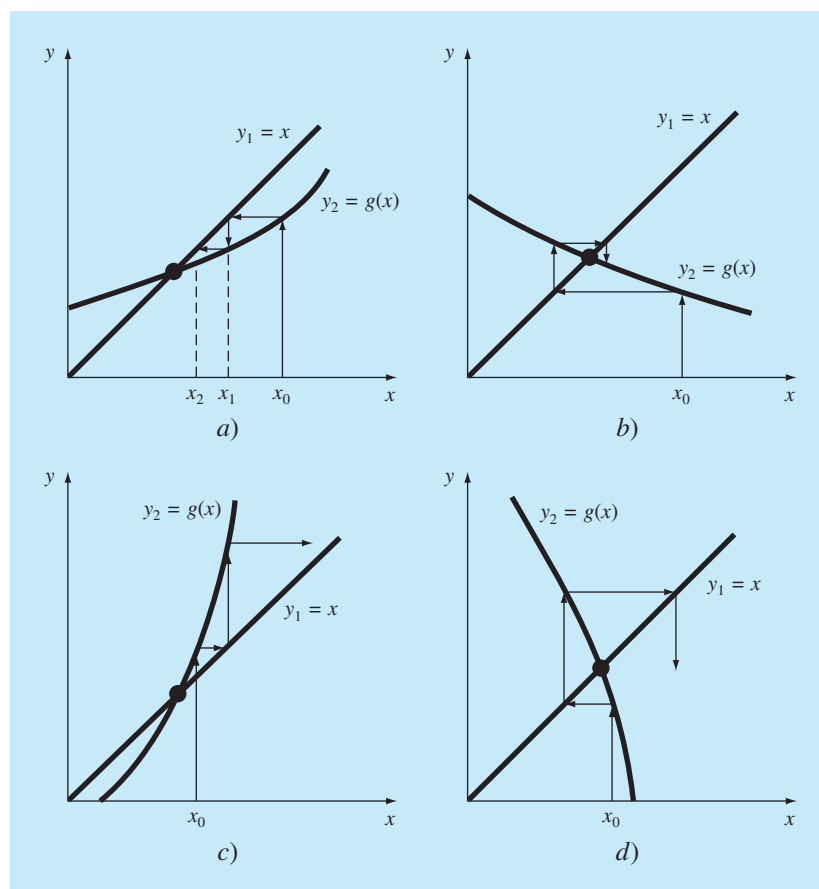
$$x_1 = g(x_0)$$

De esta manera, tanto en la ecuación como en la gráfica se usa un valor inicial x_0 para obtener una aproximación de x_1 . La siguiente iteración consiste en moverse al punto $[x_1, g(x_1)]$ y después a (x_2, x_2) . Esta iteración es equivalente a la ecuación:

$$x_2 = g(x_1)$$

FIGURA 6.3

Representación gráfica en a) y b) de la convergencia. En c) y d) de la divergencia del método de punto fijo. Las gráficas a) y c) tienen un comportamiento monótono; mientras que b) y d) tienen un comportamiento oscilatorio o en espiral. Deberá notar que la convergencia se obtiene cuando $|g'(x)| < 1$.



Cuadro 6.1 Convergencia del método de punto fijo

Al analizar la figura 6.3, se debe notar que la iteración de punto fijo converge si, en la región de interés, $|g'(x)| < 1$. En otras palabras, la convergencia ocurre si la magnitud de la pendiente de $g(x)$ es menor que la pendiente de la recta $f(x) = x$. Esta observación puede demostrarse teóricamente. Recuerde que la ecuación iterativa es

$$x_{i+1} = g(x_i)$$

Suponga que la solución verdadera es

$$x_r = g(x_r)$$

Restando estas dos ecuaciones se obtiene

$$x_r - x_{i+1} = g(x_r) - g(x_i) \quad (\text{C6.1.1})$$

El *teorema del valor medio de la derivada* (recuerde la sección 4.1.1) establece que si una función $g(x)$ y su primer derivada son continuas en un intervalo $a \leq x \leq b$, entonces existe al menos un valor de $x = \xi$ dentro del intervalo para el que

$$g'(\xi) = \frac{g(b) - g(a)}{b - a} \quad (\text{C6.1.2})$$

El lado derecho de esta ecuación es la pendiente de la recta que une a $g(a)$ y $g(b)$. Así, el teorema del valor medio establece que existe al menos un punto entre a y b que tiene una pendiente, denotada por $g'(\xi)$, que es paralela a la línea que une $g(a)$ con $g(b)$ (recuerde la figura 4.3).

Ahora, si se hace $a = x_i$ y $b = x_r$, el lado derecho de la ecuación (C6.1.1) se expresa como

$$g(x_r) - g(x_i) = (x_r - x_i)g'(\xi)$$

donde ξ se encuentra en alguna parte entre x_i y x_r . Este resultado se sustituye en la ecuación (C6.1.1) para obtener

$$x_r - x_{i+1} = (x_r - x_i)g'(\xi) \quad (\text{C6.1.3})$$

Si el error verdadero en la iteración i se define como

$$E_{t,i} = x_r - x_i$$

entonces la ecuación (C6.1.3) se convierte en

$$E_{t,i+1} = g'(\xi)E_{t,i}$$

En consecuencia, si $|g'(x)| < 1$, entonces los errores disminuyen con cada iteración. Si $|g'(x)| > 1$, los errores crecen. Observe también que si la derivada es positiva, los errores serán positivos y, por lo tanto, la solución iterativa será monótona (figuras 6.3a y 6.3c). Si la derivada es negativa, entonces los errores oscilarán (figuras 6.3b y 6.3d).

Un corolario de este análisis establece que cuando el método converge, el error es proporcional y menor que el error en la iteración anterior. Por tal razón se dice que la iteración simple de punto fijo es *linealmente convergente*.

La solución en la figura 6.3a es *convergente*, ya que la aproximación de x se acerca más a la raíz con cada iteración. Lo mismo ocurre en la figura 6.3b. Sin embargo, éste no es el caso en las figuras 6.3c y 6.3d, donde las iteraciones divergen de la raíz. Observe que la convergencia ocurre únicamente cuando el valor absoluto de la pendiente de $y_2 = g(x)$ es menor al valor de la pendiente de $y_1 = x$, es decir, cuando $|g'(x)| < 1$. En el cuadro 6.1 se presenta un desarrollo teórico de este resultado.

6.1.2 Algoritmo para el método de punto fijo

El algoritmo para la iteración de punto fijo es simple en extremo. Consta de un loop o ciclo que calcula en forma iterativa nuevas aproximaciones hasta satisfacer el criterio de terminación. En la figura 6.4 se muestra el pseudocódigo para el algoritmo. Se pueden programar de manera similar otros métodos abiertos, la modificación principal consiste en cambiar la fórmula iterativa que se utiliza para calcular la nueva raíz.

```

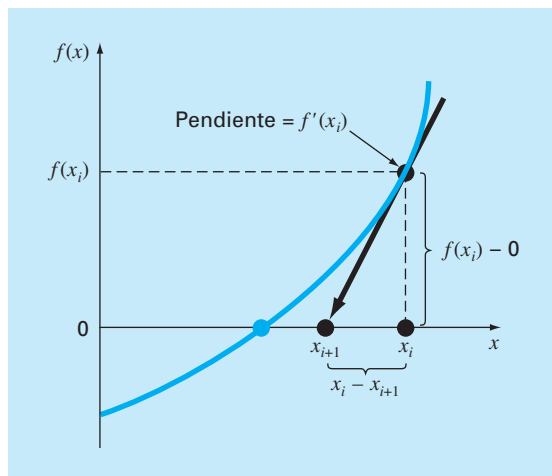
FUNCTION Fixpt(x0, es, imax iter, ea)
  xr = x0
  iter = 0
  DO
    xrold = xr
    xr = g(xrold)
    iter = iter + 1
    IF xr ≠ 0 THEN
      
$$ea = \left| \frac{xr - xrold}{xr} \right| \cdot 100$$

    END IF
    IF ea < es OR iter ≥ imax EXIT
  END DO
  Fixpt = xr
END Fixpt

```

FIGURA 6.4

Seudocódigo para el método de punto fijo. Note que otros métodos abiertos pueden diseñarse en este formato general.

**FIGURA 6.5**

Representación gráfica del método de Newton-Raphson. Se extrapola una tangente a la función en x_i [esto es, $f'(x_i)$] hasta el eje x para obtener una estimación de la raíz en x_{i+1} .

6.2 MÉTODO DE NEWTON-RAPHSON

Tal vez, de las fórmulas para localizar raíces, la fórmula de Newton-Raphson (figura 6.5) sea la más ampliamente utilizada. Si el valor inicial para la raíz es x_i , entonces se puede trazar una tangente desde el punto $[x_i, f(x_i)]$ de la curva. Por lo común, el punto donde esta tangente cruza al eje x representa una aproximación mejorada de la raíz.

El método de Newton-Raphson se deduce a partir de esta interpretación geométrica (un método alternativo basado en la serie de Taylor se describe en el cuadro 6.2). De la figura 6.5, se tiene que la primera derivada en x es equivalente a la pendiente:

$$f'(x_i) = \frac{f(x_i) - 0}{x_i - x_{i+1}} \quad (6.5)$$

que se arregla para obtener

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)} \quad (6.6)$$

la cual se conoce como *fórmula de Newton-Raphson*.

EJEMPLO 6.3 Método de Newton-Raphson

Planteamiento del problema. Utilice el método de Newton-Raphson para calcular la raíz de $f(x) = e^{-x} - x$ empleando como valor inicial $x_0 = 0$.

Solución. La primera derivada de la función es

$$f'(x) = -e^{-x} - 1$$

que se sustituye, junto con la función original en la ecuación (6.6), para tener

$$x_{i+1} = x_i - \frac{e^{-x_i} - x_i}{-e^{-x_i} - 1}$$

Empezando con un valor inicial $x_0 = 0$, se aplica esta ecuación iterativa para calcular

i	x_i	$\varepsilon_i(\%)$
0	0	100
1	0.500000000	11.8
2	0.566311003	0.147
3	0.567143165	0.0000220
4	0.567143290	$< 10^{-8}$

Así, el método converge rápidamente a la raíz verdadera. Observe que el error relativo porcentual verdadero en cada iteración disminuye mucho más rápido que con la iteración simple de punto fijo (compare con el ejemplo 6.1).

6.2.1 Criterio de terminación y estimación de errores

Como en los otros métodos para localizar raíces, la ecuación (3.5) se utiliza como un criterio de terminación. No obstante, el desarrollo del método con base en la serie de Taylor (cuadro 6.2), proporciona una comprensión teórica respecto a la velocidad de convergencia expresada por $E_{i+1} = O(E_i^2)$. De esta forma, el error debe ser proporcional al cuadrado del error anterior. En otras palabras, el número de cifras significativas de precisión aproximadamente se duplica en cada iteración. Dicho comportamiento se examina en el siguiente ejemplo.

Cuadro 6.2 Deducción y análisis del error del método de Newton-Raphson

Además de la deducción geométrica [ecuaciones (6.5) y (6.6)], el método de Newton-Raphson también se desarrolla a partir de la expansión de la serie de Taylor. Esta deducción alternativa es muy útil en el sentido de que provee cierta comprensión sobre la velocidad de convergencia del método.

Recuerde del capítulo 4 que la expansión de la serie de Taylor se puede representar como

$$f(x_{i+1}) = f(x_i) + f'(x_i)(x_{i+1} - x_i) + \frac{f''(\xi)}{2!}(x_{i+1} - x_i)^2 \quad (\text{C6.2.1})$$

donde ξ se encuentra en alguna parte del intervalo desde x_i hasta x_{i+1} . Truncando la serie de Taylor después del término de la primera derivada, se obtiene una versión aproximada:

$$f(x_{i+1}) \cong f(x_i) + f'(x_i)(x_{i+1} - x_i)$$

En la intersección con el eje x , $f(x_{i+1})$ debe ser igual a cero, o

$$0 = f(x_i) + f'(x_i)(x_{i+1} - x_i) \quad (\text{C6.2.2})$$

de donde se puede despejar a x_{i+1} , así

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$$

que es idéntica a la ecuación (6.6). De esta forma, se ha deducido la fórmula de Newton-Raphson usando una serie de Taylor.

Además de este desarrollo, la serie de Taylor sirve para estimar el error de la fórmula. Esto se logra observando que si se utilizan todos los términos de la serie de Taylor se obtendrá un resultado exacto. En tal situación $x_{i+1} = x_r$, donde x es el valor

verdadero de la raíz. Sustituyendo este valor junto con $f(x_r) = 0$ en la ecuación (C6.2.1) se obtiene

$$0 = f(x_i) + f'(x_i)(x_r - x_i) + \frac{f''(\xi)}{2!}(x_r - x_i)^2 \quad (\text{C6.2.3})$$

La ecuación (C6.2.2) se resta de la ecuación (C6.2.3) para obtener

$$0 = f'(x_i)(x_r - x_{i+1}) + \frac{f''(\xi)}{2!}(x_r - x_i)^2 \quad (\text{C6.2.4})$$

Ahora, observe que el error es igual a la diferencia entre x_{i+1} y el valor verdadero x_r , como en

$$E_{i,i+1} = x_r - x_{i+1}$$

y la ecuación (C6.2.4) se expresa como

$$0 = f'(x_i)E_{i,i+1} + \frac{f''(\xi)}{2!}E_{i,i}^2 \quad (\text{C6.2.5})$$

Si se supone que hay convergencia, entonces tanto x_i como ξ se deberán aproximar a la raíz x_r y la ecuación (C6.2.5) se reordena para obtener

$$E_{i,i+1} = \frac{-f''(x_r)}{2f'(x_r)}E_{i,i}^2 \quad (\text{C6.2.6})$$

De acuerdo con la ecuación (C6.2.6), el error es proporcional al cuadrado del error anterior. Esto significa que el número de cifras decimales correctas aproximadamente se duplica en cada iteración. A este comportamiento se le llama *convergencia cuadrática*. El ejemplo 6.4 ilustra esta propiedad.

EJEMPLO 6.4 Análisis de error en el método de Newton-Raphson

Planteamiento del problema. Como se dedujo del cuadro 6.2, el método de Newton-Raphson es convergente en forma cuadrática. Es decir, el error es proporcional al cuadrado del error anterior:

$$E_{i,i+1} \cong \frac{-f''(x_r)}{2f'(x_r)}E_{i,i}^2 \quad (\text{E6.4.1})$$

Examine esta fórmula y observe si concuerda con los resultados del ejemplo 6.3.

Solución. La primera derivada de $f(x) = e^{-x} - x$ es

$$f'(x) = -e^{-x} - 1$$

que se evalúa en $x_r = 0.56714329$ para dar $f'(0.56714329) = -1.56714329$. La segunda derivada es:

$$f''(x) = e^{-x}$$

la cual se evalúa como $f''(0.56714329) = 0.56714329$. Estos resultados se sustituyen en la ecuación (E6.4.1):

$$E_{t,i+1} \cong -\frac{0.56714329}{2(-1.56714329)} E_{t,i}^2 = 0.18095 E_{t,i}^2$$

En el ejemplo 6.3, el error inicial fue $E_{t,0} = 0.56714329$, el cual se sustituye en la ecuación de error que predice

$$E_{t,1} \cong 0.18095(0.56714329)^2 = 0.0582$$

que es cercano al error verdadero de 0.06714329. En la siguiente iteración,

$$E_{t,2} \cong 0.18095(0.06714329)^2 = 0.0008158$$

que también se compara de manera favorable con el error verdadero 0.0008323. Para la tercera iteración,

$$E_{t,3} \cong 0.18095(0.0008323)^2 = 0.000000125$$

que es el error obtenido en el ejemplo 6.3. Así, la estimación del error mejora, ya que conforme nos acercamos a la raíz, x y ξ se aproximan mejor mediante x_r [recuerde nuestra suposición al ir de la ecuación (C6.2.5) a la ecuación (C6.2.6) en el cuadro 6.2]. Finalmente:

$$E_{t,4} \cong 0.18095(0.000000125)^2 = 2.83 \times 10^{-15}$$

Así, este ejemplo ilustra que el error en el método de Newton-Raphson para este caso es, de hecho, proporcional (por un factor de 0.18095) al cuadrado del error en la iteración anterior.

6.2.2 Desventajas del método de Newton-Raphson

Aunque en general el método de Newton-Raphson es muy eficiente, hay situaciones donde se comporta de manera deficiente. Por ejemplo en el caso especial de raíces múltiples que se analizará más adelante en este capítulo. Sin embargo, también cuando se trata de raíces simples, se encuentran dificultades, como en el siguiente ejemplo.

EJEMPLO 6.5 Ejemplo de una función que converge lentamente con el método de Newton-Raphson

Planteamiento del problema. Determine la raíz positiva de $f(x) = x^{10} - 1$ usando el método de Newton-Raphson y un valor inicial $x = 0.5$.

Solución. La fórmula de Newton-Raphson en este caso es:

$$x_{i+1} = x_i - \frac{x_i^{10} - 1}{10x_i^9}$$

que se utiliza para calcular:

Iteración	x
0	0.5
1	51.65
2	46.485
3	41.8365
4	37.65285
5	33.887565
.	
.	
.	
∞	1.0000000

De esta forma, después de la primera predicción deficiente, la técnica converge a la raíz verdadera, 1, pero muy lentamente.

Además de la convergencia lenta debido a la naturaleza de la función, es posible que se presenten otras dificultades, como se ilustra en la figura 6.6. Por ejemplo, la figura 6.6a muestra el caso donde un punto de inflexión [esto es, $f''(x) = 0$] ocurre en la vecindad de una raíz. Observe que las iteraciones que empiezan con x_0 divergen progresivamente de la raíz. En la figura 6.6b se ilustra la tendencia del método de Newton-Raphson a oscilar alrededor de un mínimo o máximo local. Tales oscilaciones pueden persistir o, como en la figura 6.6b, alcanzar una pendiente cercana a cero, después de lo cual la solución se aleja del área de interés. En la figura 6.6c se muestra cómo un valor inicial cercano a una raíz salta a una posición varias raíces más lejos. Esta tendencia a alejarse del área de interés se debe a que se encuentran pendientes cercanas a cero. En efecto, una pendiente cero [$f'(x) = 0$] es un verdadero desastre, ya que causa una división entre cero en la fórmula de Newton-Raphson [ecuación (6.6)]. En forma gráfica (figura 6.6d), esto significa que la solución se dispara horizontalmente y jamás toca al eje x .

De manera que no hay un criterio general de convergencia para el método de Newton-Raphson. Su convergencia depende de la naturaleza de la función y de la exactitud del valor inicial. La única solución en estos casos es tener un valor inicial que sea “suficientemente” cercano a la raíz. ¡Y para algunas funciones ningún valor inicial funcionará! Los buenos valores iniciales por lo común se predicen con un conocimiento del problema físico o mediante el uso de recursos alternativos, tales como las gráficas, que proporcionan mayor claridad en el comportamiento de la solución. Ante la falta de un criterio general de convergencia se sugiere el diseño de programas computacionales eficientes que reconozcan la convergencia lenta o la divergencia. La siguiente sección está enfocada hacia dichos temas.

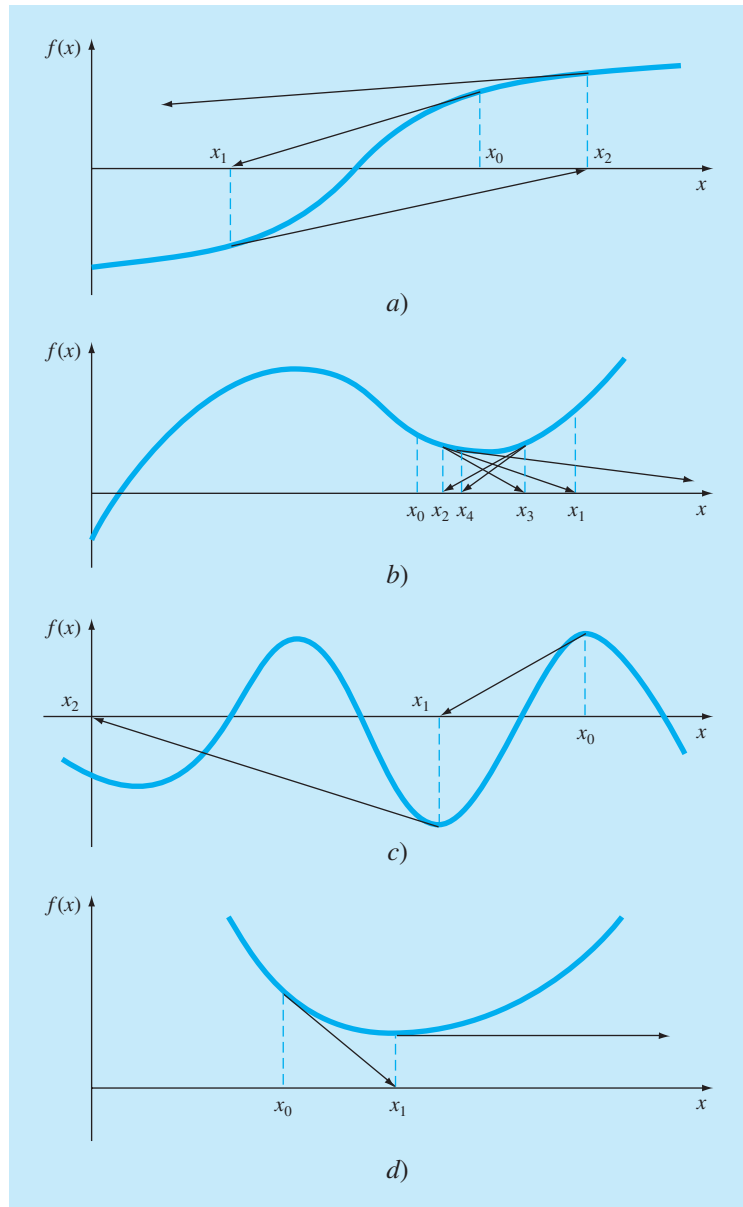
6.2.3 Algoritmo para el método de Newton-Raphson

Un algoritmo para el método de Newton-Raphson se obtiene fácilmente al sustituir la ecuación (6.6) por la fórmula predictiva [ecuación (6.2)] en la figura 6.4. Observe, sin embargo, que el programa también debe modificarse para calcular la primera derivada. Esto se logra incluyendo simplemente una función definida por el usuario.

Además, a la luz del análisis anterior sobre los problemas potenciales del método de Newton-Raphson, el programa se podría mejorar incorporando algunas consideraciones adicionales:

FIGURA 6.6

Cuatro casos donde el método de Newton-Raphson exhibe una convergencia deficiente.



1. Se debe incluir una rutina de graficación en el programa.
2. Al final de los cálculos, se necesitará sustituir siempre la raíz final calculada en la función original, para determinar si el resultado se acerca a cero. Esta prueba protege el desarrollo del programa contra aquellos casos en los que se presenta convergencia lenta u oscilatoria, la cual puede llevar a valores pequeños de ε_a , mientras que la solución aún está muy lejos de una raíz.
3. El programa deberá incluir siempre un límite máximo permitido del número de iteraciones para estar prevenidos contra soluciones oscilantes, de lenta convergencia o divergentes que podrían persistir en forma interminable.
4. El programa deberá alertar al usuario para que tome en cuenta la posibilidad de que $f'(x)$ sea igual a cero en cualquier momento durante el cálculo.

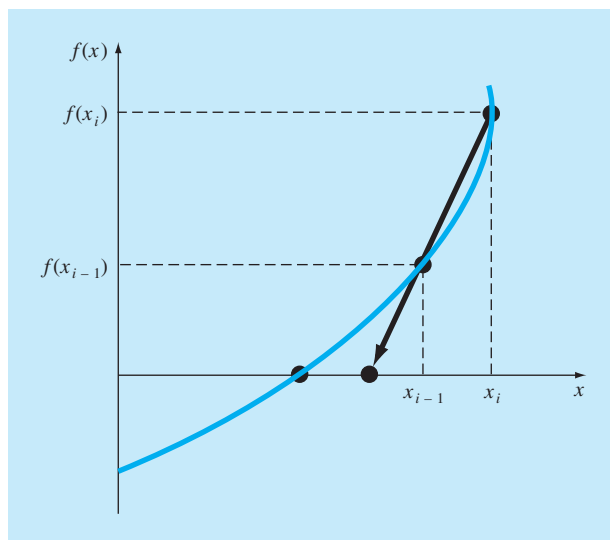
6.3 EL MÉTODO DE LA SECANTE

Un problema potencial en la implementación del método de Newton-Raphson es la evaluación de la derivada. Aunque esto no es un inconveniente para los polinomios ni para muchas otras funciones, existen algunas funciones cuyas derivadas en ocasiones resultan muy difíciles de calcular. En dichos casos, la derivada se puede aproximar mediante una diferencia finita dividida hacia atrás, como en (figura 6.7)

$$f'(x_i) \cong \frac{f(x_{i-1}) - f(x_i)}{x_{i-1} - x_i}$$

FIGURA 6.7

Representación gráfica del método de la secante. Esta técnica es similar a la del método de Newton-Raphson (figura 6.5) en el sentido de que una aproximación de la raíz se predice extrapolando una tangente de la función hasta el eje x. Sin embargo, el método de la secante usa una diferencia dividida en lugar de una derivada para estimar la pendiente.



Esta aproximación se sustituye en la ecuación (6.6) para obtener la siguiente ecuación iterativa:

$$x_{i+1} = x_i - \frac{f(x_i)(x_{i-1} - x_i)}{f(x_{i-1}) - f(x_i)} \quad (6.7)$$

La ecuación (6.7) es la fórmula para el *método de la secante*. Observe que el método requiere de dos valores iniciales de x . Sin embargo, debido a que no se necesita que $f(x)$ cambie de signo entre los valores dados, este método no se clasifica como un método cerrado.

EJEMPLO 6.6 El método de la secante

Planteamiento del problema. Con el método de la secante calcule la raíz de $f(x) = e^{-x} - x$. Comience con los valores iniciales $x_{-1} = 0$ y $x_0 = 1.0$.

Solución. Recuerde que la raíz real es 0.56714329...

Primera iteración:

$$x_{-1} = 0 \quad f(x_{-1}) = 1.00000$$

$$x_0 = 1 \quad f(x_0) = -0.63212$$

$$x_1 = 1 - \frac{-0.63212(0 - 1)}{1 - (-0.63212)} = 0.61270 \quad \varepsilon_i = 8.0\%$$

Segunda iteración:

$$x_0 = 1 \quad f(x_0) = -0.63212$$

$$x_1 = 0.61270 \quad f(x_1) = -0.07081$$

(Note que ambas aproximaciones se encuentran del mismo lado de la raíz.)

$$x_2 = 0.61270 - \frac{-0.07081(1 - 0.61270)}{-0.63212 - (-0.07081)} = 0.56384 \quad \varepsilon_i = 0.58\%$$

Tercera iteración:

$$x_1 = 0.61270 \quad f(x_1) = -0.07081$$

$$x_2 = 0.56384 \quad f(x_2) = 0.00518$$

$$x_3 = 0.56384 - \frac{0.00518(0.61270 - 0.56384)}{-0.07081 - (-0.00518)} = 0.56717 \quad \varepsilon_i = 0.0048\%$$

6.3.1 Diferencia entre los métodos de la secante y de la falsa posición

Observe la similitud entre los métodos de la secante y de la falsa posición. Por ejemplo, las ecuaciones (6.7) y (5.7) son idénticas en todos los términos. Ambas usan dos valores iniciales para calcular una aproximación de la pendiente de la función que se utiliza para

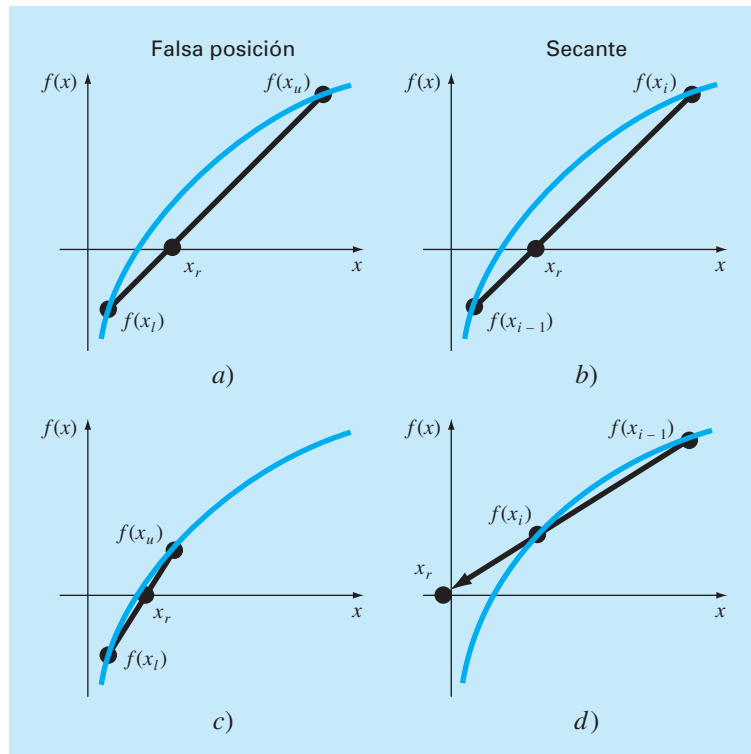
proyectar hacia el eje x una nueva aproximación de la raíz. Sin embargo, existe una diferencia crítica entre ambos métodos. Tal diferencia estriba en la forma en que uno de los valores iniciales se reemplaza por la nueva aproximación. Recuerde que en el método de la falsa posición, la última aproximación de la raíz reemplaza cualquiera de los valores iniciales que dé un valor de la función con el mismo signo que $f(x_r)$. En consecuencia, las dos aproximaciones siempre encierran a la raíz. Por lo tanto, para todos los casos, el método siempre converge, pues la raíz se encuentra dentro del intervalo. En contraste, el método de la secante reemplaza los valores en secuencia estricta: con el nuevo valor x_{i+1} se reemplaza a x_i y x_i reemplaza a x_{i-1} . En consecuencia, algunas veces los dos valores están en el mismo lado de la raíz. En ciertos casos esto puede llevar a divergencias.

EJEMPLO 6.7 Comparación de la convergencia en los métodos de la secante y de la falsa posición

Planteamiento del problema. Utilice los métodos de la secante y de la falsa posición para calcular la raíz de $f(x) = \ln x$. Empiece los cálculos con los valores iniciales $x_l = x_{i-1} = 0.5$ y $x_u = x_i = 5.0$.

FIGURA 6.8

Comparación entre los métodos de la falsa posición y de la secante. Las primeras iteraciones a) y b) de ambos métodos son idénticas. No obstante, en las segundas iteraciones c) y d), los puntos usados son diferentes. En consecuencia, el método de la secante llega a diverger, como se indica en d).



Solución. En el método de la falsa posición, con el uso de la ecuación (5.7) y los criterios del intervalo para el reemplazo de las aproximaciones, se obtienen las siguientes iteraciones:

Iteración	x_l	x_u	x_r
1	0.5	5.0	1.8546
2	0.5	1.8546	1.2163
3	0.5	1.2163	1.0585

Como se observa (figuras 6.8a y c), las aproximaciones van convergiendo a la raíz real que es igual a 1.

En el método de la secante, con el uso de la ecuación (6.7) y el criterio secuencial para el reemplazo de las aproximaciones, se obtiene:

Iteración	x_{i-1}	x_i	x_{i+1}
1	0.5	5.0	1.8546
2	5.0	1.8546	-0.10438

Como se muestra en la figura 6.8d, el método es divergente.

Aunque el método de la secante sea divergente, cuando converge lo hace más rápido que el método de la falsa posición. Por ejemplo, en la figura 6.9 se muestra la superioridad del método de la secante. La inferioridad del método de la falsa posición se debe a que un extremo permanece fijo, para mantener a la raíz dentro del intervalo. Esta propiedad, que es una ventaja porque previene la divergencia, tiene una desventaja en relación con la velocidad de convergencia; esto hace de la diferencia finita estimada una aproximación menos exacta que la derivada.

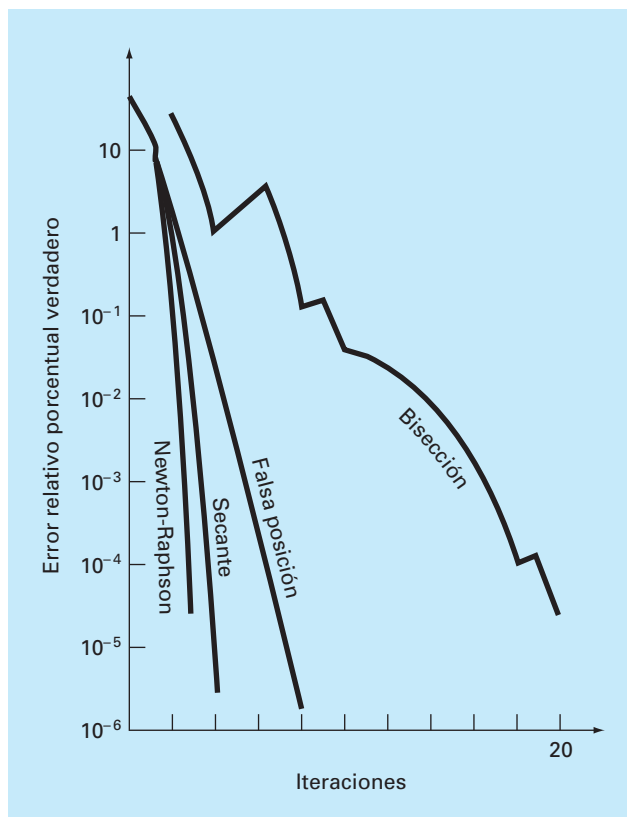
6.3.2 Algoritmo para el método de la secante

Como con los otros métodos abiertos, el algoritmo del método de la secante se obtiene simplemente modificando la figura 6.4, de tal forma que se puedan introducir dos valores iniciales, y usando la ecuación (6.7) se calcule la raíz. Además, las opciones sugeridas en la sección 6.2.3 para el método de Newton-Raphson, también se pueden aplicar para obtener ventajas al programa de la secante.

6.3.3 Método de la secante modificado

En lugar de usar dos valores arbitrarios para aproximar la derivada, un método alternativo considera un cambio fraccionario de la variable independiente para estimar $f'(x)$,

$$f'(x_i) \cong \frac{f(x_i + \delta x_i) - f(x_i)}{\delta x_i}$$

**FIGURA 6.9**

Comparación de los errores relativos porcentuales verdaderos ϵ_i , para los métodos que determinan las raíces de $f(x) = e^{-x} - x$.

donde δ es un pequeño cambio fraccionario. Esta aproximación se sustituye en la ecuación (6.6) que da la siguiente ecuación iterativa:

$$x_{i+1} = x_i - \frac{\delta x_i f(x_i)}{f(x_i + \delta x_i) - f(x_i)} \quad (6.8)$$

EJEMPLO 6.8 Método de la secante modificado

Planteamiento del problema. Con el método de la secante modificado estime la raíz de $f(x) = e^{-x} - x$. Use un valor de 0.01 para δ y comience con $x_0 = 1.0$. Recuerde que la raíz verdadera es 0.56714329...

Solución.

Primera iteración:

$$\begin{aligned}x_0 &= 1 & f(x_0) &= -0.63212 \\x_0 + \delta x_0 &= 1.01 & f(x_0 + \delta x_0) &= -0.64578\end{aligned}$$

$$x_1 = 1 - \frac{0.01(-0.63212)}{-0.64578 - (-0.63212)} = 0.537263 \quad |\varepsilon_t| = 5.3\%$$

Segunda iteración:

$$\begin{aligned}x_0 &= 0.537263 & f(x_0) &= 0.047083 \\x_0 + \delta x_0 &= 0.542635 & f(x_0 + \delta x_0) &= 0.038579\end{aligned}$$

$$x_1 = 0.537263 - \frac{0.005373(0.047083)}{0.038579 - 0.047083} = 0.56701 \quad |\varepsilon_t| = 0.0236\%$$

Tercera iteración:

$$x_0 = 0.56701 \quad f(x_0) = 0.000209$$

$$x_0 + \delta x_0 = 0.567143 \quad f(x_0 + \delta x_0) = -0.00867$$

$$x_1 = 0.56701 - \frac{0.00567(0.000209)}{-0.00867 - 0.000209} = 0.567143 \quad |\varepsilon_t| = 2.365 \times 10^{-5}\%$$

La elección de un valor adecuado para δ no es automática. Si δ es muy pequeño, el método puede no tener éxito por el error de redondeo, causado por la cancelación por resta en el denominador de la ecuación (6.8). Si ésta es muy grande, la técnica puede llegar a ser ineficiente y hasta divergente. No obstante, si se selecciona correctamente, proporciona una adecuada alternativa en los casos donde la evaluación de la derivada se dificulta y el desarrollo de dos valores iniciales es inconveniente.

6.4 RAÍCES MÚLTIPLES

Una *raíz múltiple* corresponde a un punto donde una función es tangencial al eje x . Por ejemplo, una raíz doble resulta de

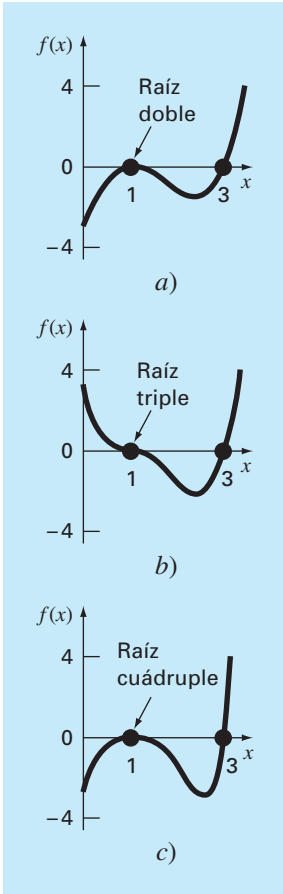
$$f(x) = (x - 3)(x - 1)(x - 1) \quad (6.9)$$

o, multiplicando *términos*, $f(x) = x^3 - 5x^2 + 7x - 3$. La ecuación tiene una *raíz doble* porque un valor de x hace que dos términos de la ecuación (6.9) sean iguales a cero. Gráficamente, esto significa que la curva toca en forma tangencial al eje x en la raíz doble. Observe la figura 6.10a en $x = 1$. Note que la función toca al eje pero no la cruza en la raíz.

Una *raíz triple* corresponde al caso en que un valor de x hace que tres términos en una ecuación sean iguales a cero, como en

$$f(x) = (x - 3)(x - 1)(x - 1)(x - 1)$$

o, multiplicando los términos, $f(x) = x^4 - 6x^3 + 12x^2 - 10x + 3$. Advierta que la representación gráfica (figura 6.10b) indica otra vez que la función es tangente al eje en la raíz, pero que en este caso sí cruza el eje. En general, la multiplicidad impar de raíces cruza

**FIGURA 6.10**

Ejemplos de raíces múltiples que son tangenciales al eje x . Observe que la función no cruza el eje en los casos de raíces múltiples pares a) y c), mientras que con multiplicidad impar sí lo hace en b).

el eje, mientras que la multiplicidad par no lo cruza. Por ejemplo, la raíz cuádruple en la figura 6.10c no cruza el eje.

Las raíces múltiples ofrecen algunas dificultades a muchos de los métodos numéricos expuestos en la parte dos:

1. El hecho de que la función no cambie de signo en raíces múltiples pares impide confiarse de los métodos cerrados, que se analizan en el capítulo 5. Así, en los métodos incluidos en este texto, se está limitando a los abiertos que pueden ser divergentes.
2. Otro posible problema se relaciona con el hecho de que no sólo $f(x)$, sino también $f'(x)$ se aproxima a cero en la raíz. Tales problemas afectan los métodos de Newton-Raphson y de la secante, los cuales contienen derivadas (o su aproximación) en el denominador de sus fórmulas respectivas. Esto provocará una división entre cero cuando la solución converge muy cerca de la raíz. Una forma simple de evitar dichos problemas, que se ha demostrado teóricamente (Ralston y Rabinowitz, 1978), se basa en el hecho de que $f(x)$ siempre alcanzará un valor cero antes que $f'(x)$. Por lo tanto, si se compara $f(x)$ contra cero, dentro del programa, entonces los cálculos se pueden terminar antes de que $f'(x)$ llegue a cero.
3. Es posible demostrar que el método de Newton-Raphson y el método de la secante convergen en forma lineal, en vez de cuadrática, cuando hay raíces múltiples (Ralston y Rabinowitz, 1978). Se han propuesto algunas modificaciones para atenuar este problema. Ralston y Rabinowitz (1978) proponen que se realice un pequeño cambio en la formulación para que se regrese a la convergencia cuadrática, como en

$$x_{i+1} = x_i - m \frac{f(x_i)}{f'(x_i)} \quad (6.9a)$$

donde m es la multiplicidad de la raíz (es decir, $m = 2$ para una raíz doble, $m = 3$ para una raíz triple, etc.). Se trata de una alternativa poco satisfactoria, porque depende del conocimiento de la multiplicidad de la raíz.

Otra alternativa, también sugerida por Ralston y Rabinowitz (1978), consiste en definir una nueva función $u(x)$, que es el cociente de la función original entre su derivada:

$$u(x) = \frac{f(x)}{f'(x)} \quad (6.10)$$

Se puede demostrar que esta función tiene raíces en las mismas posiciones que la función original. Por lo tanto, la ecuación (6.10) se sustituye en la ecuación (6.6) para desarrollar una forma alternativa del método de Newton-Raphson:

$$x_{i+1} = x_i - \frac{u(x_i)}{u'(x_i)} \quad (6.11)$$

Se deriva con respecto a x la ecuación (6.10) para obtener

$$u'(x) = \frac{f'(x)f''(x) - f(x)f'''(x)}{[f'(x)]^2} \quad (6.12)$$

Se sustituyen las ecuaciones (6.10) y (6.12) en la ecuación (6.11) y se simplifica el resultado:

$$x_{i+1} = x_i - \frac{f(x_i)f'(x_i)}{[f'(x)]^2 - f(x_i)f''(x_i)} \quad (6.13)$$

EJEMPLO 6.9 Método de Newton-Raphson modificado para el cálculo de raíces múltiples

Planteamiento del problema. Con los dos métodos, el estándar y el modificado, de Newton-Raphson evalúe la raíz múltiple de la ecuación (6.9), use un valor inicial de $x_0 = 0$.

Solución. La primera derivada de la ecuación (6.9) es $f'(x) = 3x^2 - 10x + 7$, y por lo tanto, el método de Newton-Raphson estándar para este problema es [ecuación (6.6)]

$$x_{i+1} = x_i - \frac{x_i^3 - 5x_i^2 + 7x_i - 3}{3x_i^2 - 10x_i + 7}$$

que se resuelve iterativamente para obtener

i	x_i	$\varepsilon_t(\%)$
0	0	100
1	0.4285714	57
2	0.6857143	31
3	0.8328654	17
4	0.9133290	8.7
5	0.9557833	4.4
6	0.9776551	2.2

Como ya se había anticipado, el método converge en forma lineal hacia el valor verdadero 1.0.

Para el caso del método modificado, la segunda derivada es $f''(x) = 6x - 10$, y en consecuencia la ecuación iterativa será [ecuación (6.13)]

$$x_{i+1} = x_i - \frac{(x_i^3 - 5x_i^2 + 7x_i - 3)(3x_i^2 - 10x_i + 7)}{(3x_i^2 - 10x_i + 7)^2 - (x_i^3 - 5x_i^2 + 7x_i - 3)(6x_i - 10)}$$

que se resuelve para obtener

i	x_i	$\varepsilon_t(\%)$
0	0	100
1	1.105263	11
2	1.003082	0.31
3	1.000002	0.00024

De esta manera, la fórmula modificada converge en forma cuadrática. Se pueden usar ambos métodos para buscar la raíz simple en $x = 3$. Con un valor inicial $x_0 = 4$ se obtienen los siguientes resultados:

<i>i</i>	Estándar	$\varepsilon_t(\%)$	Modificado	$\varepsilon_t(\%)$
0	4	33	4	33
1	3.4	13	2.636364	12
2	3.1	3.3	2.820225	6.0
3	3.008696	0.29	2.961728	1.3
4	3.000075	0.0025	2.998479	0.051
5	3.000000	2×10^{-7}	2.999998	7.7×10^{-5}

De esta forma, deberá notar que, ambos métodos convergen con rapidez, aunque el método estándar es el más eficiente.

En el ejemplo anterior se ilustran los factores de mayor importancia involucrados al elegir el método de Newton-Raphson modificado. Aunque es preferible para raíces múltiples, es menos eficiente y requiere más trabajo computacional que el método estándar para raíces simples.

Se debe notar que hay manera de desarrollar una versión modificada del método de la secante para raíces múltiples, sustituyendo la ecuación (6.10) en la ecuación (6.7). La fórmula resultante es (Ralston y Rabinowitz, 1978)

$$x_{i+1} = x_i - \frac{u(x_i)(x_{i-1} - x_i)}{u(x_{i-1}) - u(x_i)}$$

6.5 SISTEMAS DE ECUACIONES NO LINEALES

Hasta aquí nos hemos ocupado de determinar las raíces de una sola ecuación no lineal. Un problema relacionado con éste consiste en obtener las raíces de un conjunto de ecuaciones simultáneas,

$$\begin{aligned} f_1(x_1, x_2, \dots, x_n) &= 0 \\ f_2(x_1, x_2, \dots, x_n) &= 0 \\ &\vdots \\ f_n(x_1, x_2, \dots, x_n) &= 0 \end{aligned} \quad (6.14)$$

La solución de este sistema consta de un conjunto de valores x_i que simultáneamente hacen que todas las ecuaciones sean iguales a cero.

En la parte tres, presentaremos los métodos, para el caso en que las ecuaciones simultáneas son lineales, es decir, que se puedan expresar en la forma general

$$f(x) = a_1x_1 + a_2x_2 + \dots + a_nx_n - b = 0 \quad (6.15)$$

donde la b y las a son constantes. A las ecuaciones algebraicas y trascendentes que no se pueden expresar de esta forma se les llama *ecuaciones no lineales*. Por ejemplo,

$$x^2 + xy = 10$$

y

$$y + 3xy^2 = 57$$

son dos ecuaciones simultáneas no lineales con dos incógnitas, x y y , las cuales se expresan en la forma de la ecuación (6.14) como

$$u(x, y) = x^2 + xy - 10 = 0 \quad (6.16a)$$

$$v(x, y) = y + 3xy^2 - 57 = 0 \quad (6.16b)$$

Así, la solución serían los valores de x y de y que hacen a las funciones $u(x, y)$ y $v(x, y)$ iguales a cero. La mayoría de los métodos para determinar tales soluciones son extensiones de los métodos abiertos para resolver ecuaciones simples. En esta sección presentaremos dos de ellos: iteración de punto fijo y Newton-Raphson.

6.5.1 Iteración de punto fijo

El método de iteración de punto fijo (sección 6.1) puede modificarse para resolver dos ecuaciones simultáneas no lineales. Este método se ilustra en el siguiente ejemplo.

EJEMPLO 6.10 Iteración de punto fijo para un sistema no lineal

Planteamiento del problema. Con el método de iteración de punto fijo determine las raíces de la ecuación (6.16). Observe que un par correcto de raíces es $x = 2$ y $y = 3$. Inicie el cálculo con el valor inicial $x = 1.5$ y $y = 3.5$.

Solución. En la ecuación (6.16a) se despeja x

$$x_{i+1} = \frac{10 - x_i^2}{y_i} \quad (E6.10.1)$$

y en la ecuación (6.16b) se despeja y

$$y_{i+1} = 57 - 3x_i y_i^2 \quad (E6.10.2)$$

Observe que dejaremos los subíndices en el resto del ejemplo.

Con base en los valores iniciales, la ecuación (E6.10.1) se utiliza para determinar un nuevo valor de x :

$$x = \frac{10 - (1.5)^2}{3.5} = 2.21429$$

Este resultado y el valor inicial de $y = 3.5$ se sustituye en la ecuación (E6.10.2) para determinar un nuevo valor de y :

$$y = 57 - 3(2.21429)(3.5)^2 = -24.37516$$

Así, parece que el método diverge. Este comportamiento es aún más pronunciado en la segunda iteración:

$$x = \frac{10 - (2.21429)^2}{-24.37516} = -0.20910$$

$$y = 57 - 3(-0.20910)(-24.37516)^2 = 429.709$$

En efecto, la aproximación se está descomponiendo.

Ahora repita el cálculo, pero con la ecuación original puesta en una forma diferente. Por ejemplo, un despeje alternativo de la ecuación (6.16a) es

$$x = \sqrt{10 - xy}$$

y de la ecuación (6.16b) es

$$y = \sqrt{\frac{57 - y}{3x}}$$

Ahora los resultados son más satisfactorios:

$$x = \sqrt{10 - 1.5(3.5)} = 2.17945$$

$$y = \sqrt{\frac{57 - 3.5}{3(2.17945)}} = 2.86051$$

$$x = \sqrt{10 - 2.17945(2.86051)} = 1.94053$$

$$y = \sqrt{\frac{57 - 2.86051}{3(1.94053)}} = 3.04955$$

Así, la aproximación converge hacia la solución correcta $x = 2$ y $y = 3$.

El ejemplo anterior ilustra la más seria desventaja de la iteración simple de punto fijo, ésta es que, la convergencia depende de la manera en que se formula la ecuación. Además, aun cuando la convergencia es posible, la divergencia puede ocurrir si los valores iniciales no son suficientemente cercanos a la solución verdadera. Usando un razonamiento similar al del cuadro 6.1, se demuestra que las condiciones suficientes para la convergencia en el caso de dos ecuaciones son

$$\left| \frac{\partial u}{\partial x} \right| + \left| \frac{\partial v}{\partial x} \right| < 1$$

y

$$\left| \frac{\partial u}{\partial y} \right| + \left| \frac{\partial v}{\partial y} \right| < 1$$

Estos criterios son tan restringidos que el método de punto fijo tiene una utilidad limitada para resolver sistemas no lineales. Sin embargo, como se describirá más adelante en el libro, será muy útil para resolver sistemas de ecuaciones lineales.

6.5.2 Newton-Raphson

Recuerde que el método de Newton-Raphson se utilizó empleando la derivada (al evaluar, es la pendiente de la recta tangente) de una función, para calcular su intersección con el

eje de la variable independiente; esto es, la raíz (figura 6.5). Dicho cálculo se basó en la expansión de la serie de Taylor de primer orden (recuerde el cuadro 6.2),

$$f(x_{i+1}) = f(x_i) + (x_{i+1} - x_i) f'(x_i) \quad (6.17)$$

donde x_i es el valor inicial de la raíz y x_{i+1} es el valor en el cual la recta tangente interseca el eje x . En esta intersección, $f(x_{i+1})$ es, por definición, igual a cero y la ecuación (6.17) se reordena para tener

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)} \quad (6.18)$$

que es la forma del método de Newton-Raphson para una sola ecuación.

La forma para múltiples ecuaciones se obtiene en forma idéntica. Sin embargo, se debe usar una serie de Taylor de múltiples variables para tomar en cuenta el hecho de que más de una variable independiente contribuye a la determinación de la raíz. En el caso de dos variables, una serie de Taylor de primer orden se escribe [recuerde la ecuación (4.26)] para cada ecuación no lineal como

$$u_{i+1} = u_i + (x_{i+1} - x_i) \frac{\partial u_i}{\partial x} + (y_{i+1} - y_i) \frac{\partial u_i}{\partial y} \quad (6.19a)$$

y

$$v_{i+1} = v_i + (x_{i+1} - x_i) \frac{\partial v_i}{\partial x} + (y_{i+1} - y_i) \frac{\partial v_i}{\partial y} \quad (6.19b)$$

De la misma manera como en la versión para una sola ecuación, la raíz aproximada corresponde a los valores de x y y , donde u_{i+1} y v_{i+1} son iguales a cero. En tal situación, se reordena la ecuación (6.19) como:

$$\frac{\partial u_i}{\partial x} x_{i+1} + \frac{\partial u_i}{\partial y} y_{i+1} = -u_i + x_i \frac{\partial u_i}{\partial x} + y_i \frac{\partial u_i}{\partial y} \quad (6.20a)$$

$$\frac{\partial v_i}{\partial x} x_{i+1} + \frac{\partial v_i}{\partial y} y_{i+1} = -v_i + x_i \frac{\partial v_i}{\partial x} + y_i \frac{\partial v_i}{\partial y} \quad (6.20b)$$

Debido a que se conocen todos los valores con subíndice i (corresponden al último valor estimado), las únicas incógnitas son x_{i+1} y y_{i+1} . Entonces, la ecuación (6.20) es un conjunto de dos ecuaciones lineales con dos incógnitas [compare con la ecuación (6.15)]. En consecuencia, se pueden usar manipulaciones algebraicas (por ejemplo, la regla de Cramer) para resolverlo:

$$x_{i+1} = x_i - \frac{u_i \frac{\partial v_i}{\partial y} - v_i \frac{\partial u_i}{\partial y}}{\frac{\partial u_i}{\partial x} \frac{\partial v_i}{\partial y} - \frac{\partial u_i}{\partial y} \frac{\partial v_i}{\partial x}} \quad (6.21a)$$

$$y_{i+1} = y_i - \frac{v_i \frac{\partial u_i}{\partial x} - u_i \frac{\partial v_i}{\partial x}}{\frac{\partial u_i}{\partial x} \frac{\partial v_i}{\partial y} - \frac{\partial u_i}{\partial y} \frac{\partial v_i}{\partial x}} \quad (6.21b)$$

El denominador de cada una de esas ecuaciones se conoce formalmente como el determinante *Jacobiano* del sistema.

La ecuación (6.21) es la versión para dos ecuaciones del método de Newton-Raphson. Como en el siguiente ejemplo, se puede emplear en forma iterativa para determinar las raíces de dos ecuaciones simultáneas.

EJEMPLO 6.11 Newton-Raphson para un sistema no lineal

Planteamiento del problema. Con el método de Newton-Raphson para múltiples ecuaciones determine las raíces de la ecuación (6.16). Observe que un par correcto de raíces es $x = 2$ y $y = 3$. Use como valores iniciales $x = 1.5$ y $y = 3.5$.

Solución. Primero calcule las derivadas parciales y evalúelas con los valores iniciales de x y y :

$$\begin{aligned} \frac{\partial u_0}{\partial x} &= 2x + y = 2(1.5) + 3.5 = 6.5 & \frac{\partial u_0}{\partial y} &= x = 1.5 \\ \frac{\partial v_0}{\partial x} &= 3y^2 = 3(3.5)^2 = 36.75 & \frac{\partial v_0}{\partial y} &= 1 + 6xy = 1 + 6(1.5)(3.5) = 32.5 \end{aligned}$$

Así, el determinante jacobiano para la primera iteración es

$$6.5(32.5) - 1.5(36.75) = 156.125$$

Los valores de las funciones se evalúan con los valores iniciales como

$$u_0 = (1.5)^2 + 1.5(3.5) - 10 = -2.5$$

$$v_0 = 3.5 + 3(1.5)(3.5)^2 - 57 = 1.625$$

Estos valores se sustituyen en la ecuación (6.21):

$$x = 1.5 - \frac{-2.5(32.5) - 1.625(1.5)}{156.125} = 2.03603$$

$$y = 3.5 - \frac{1.625(6.5) - (-2.5)(36.75)}{156.125} = 2.84388$$

Así, los resultados están convergiendo a los valores verdaderos $x = 2$ y $y = 3$. Los cálculos se repiten hasta que se obtenga una precisión aceptable.

Como con el método de iteración de punto fijo, la aproximación de Newton-Raphson puede diverger si los valores iniciales no están lo suficientemente cercanos a la raíz

verdadera. Mientras que para el caso de una sola ecuación los métodos gráficos son útiles para obtener un buen valor inicial, ningún procedimiento tan simple está disponible para el caso de múltiples ecuaciones. Aunque existen algunos métodos avanzados para obtener una primer aproximación aceptable, los valores iniciales a menudo deben obtenerse mediante prueba y error, con el conocimiento del sistema físico que se está modelando.

El método de Newton-Raphson para dos ecuaciones puede generalizarse para resolver n ecuaciones simultáneas. Debido a que el camino más eficiente para esto implica el álgebra matricial y la solución de ecuaciones lineales simultáneas, se pospondrá su estudio para la parte tres.

PROBLEMAS

6.1 Utilice la iteración simple de punto fijo para localizar la raíz de

$$f(x) = 2 \sin(\sqrt{x}) - x$$

Haga una elección inicial de $x_0 = 0.5$ e itere hasta que $\varepsilon_a \leq 0.001\%$. Compruebe que el proceso converge en forma lineal según se describió en el recuadro 6.1.

6.2 Determine la raíz real más grande de

$$f(x) = 2x^3 - 11.7x^2 + 17.7x - 5$$

- En forma gráfica.
- Con el método de iteración simple de punto fijo (tres iteraciones, $x_0 = 3$). Nota: asegúrese de haber desarrollado una solución que converja a la raíz.
- Con el método de Newton-Raphson (tres iteraciones, $x_0 = 3$, $\delta = 0.001$).
- Con el método de la secante (tres iteraciones $x_{-1} = 3$, $x_0 = 4$).
- Con el método de la secante modificado (tres iteraciones, $x_0 = 3$, $\delta = 0.01$). Calcule el porcentaje aproximado de errores relativos para sus soluciones.

6.3 Utilice los métodos de a) iteración de punto fijo, y b) Newton-Raphson, para determinar una raíz de $f(x) = -x^2 + 1.8x + 2.5$ con el uso de $x_0 = 5$. Haga el cálculo hasta que ε_a sea menor que $\varepsilon_s = 0.05\%$. Asimismo, realice una comprobación del error de su respuesta final.

6.4 Determine las raíces reales de $f(x) = -1 + 5.5x - 4x^2 + 0.5x^3$: a) en forma gráfica, y b) con el método de Newton-Raphson dentro de $\varepsilon_s = 0.01\%$.

6.5 Emplee el método de Newton-Raphson para determinar una raíz real de $f(x) = -1 + 5.5x - 4x^2 + 0.5x^3$ con el uso de elección

iniciales de a) 4.52, y b) 4.54. Estudie y use métodos gráficos y analíticos para explicar cualquier peculiaridad en sus resultados.

6.6 Determine la raíz real más pequeña de $f(x) = -12 - 21x + 18x^2 - 2.4x^3$: a) en forma gráfica, y b) con el empleo del método de la secante para un valor de ε_s que corresponda a tres cifras significativas.

6.7 Localice la primera raíz positiva de

$$f(x) = \sin x + \cos(1 + x^2) - 1$$

donde x está en radianes. Para localizar la raíz, use cuatro iteraciones del método de la secante con valores iniciales de a) $x_{i-1} = 1.0$ y $x_i = 3.0$; y b) $x_{i-1} = 1.5$ y $x_i = 2.5$, y c) $x_{i-1} = 1.5$ y $x_i = 2.25$.

6.8 Determine la raíz real de $x^{3.5} = 80$, con el método de la secante modificado dentro de $\varepsilon_s = 0.1\%$, con el uso de una elección inicial de $x_0 = 3.5$ y $\delta = 0.01$.

6.9 Determine la raíz real más grande de $f(x) = 0.95x^3 - 5.9x^2 + 10.9x - 6$:

- En forma gráfica.
- Con el uso del método de Newton-Raphson (tres iteraciones, $x_i = 3.5$).
- Con el método de la secante (tres iteraciones, $x_{i-1} = 2.5$ y $x_i = 3.5$).
- Por medio del método de la secante modificado (tres iteraciones, $x_i = 3.5$, $\delta = 0.01$).

6.10 Determine la menor raíz positiva de $f(x) = 8 \sin(x)e^{-x} - 1$:

- En forma gráfica.
- Con el uso del método de Newton-Raphson (tres iteraciones, $x_i = 0.3$).

- c) Con el método de la secante (tres iteraciones, $x_{i-1} = 0.5$ y $x_i = 0.3$).
- d) Por medio del método de la secante modificado (cinco iteraciones $x_i = 0.3$, $\delta = 0.01$).

6.11 La función $x^3 + 2x^2 - 4x + 8$ tiene una raíz doble en $x = 2$. Emplee a) el método estándar de Newton-Raphson [ec. (6.6)], b) el método de Newton-Raphson modificado [ec. (6.9a)], y c) el método de Newton-Raphson modificado [ec. (6.13)] para resolver para la raíz en $x = 2$. Compare y analice la tasa de convergencia con un valor inicial $x_0 = 1.2$.

6.12 Determine las raíces de las siguientes ecuaciones no lineales simultáneas, por medio de los métodos de a) iteración de punto fijo, y b) Newton-Raphson:

$$y = -x^2 + x + 0.75$$

$$y + 5xy = x^2$$

Utilice valores iniciales de $x = y = 1.2$, y analice los resultados.

6.13 Encuentre las raíces de las ecuaciones simultáneas que siguen:

$$(x - 4)^2 + (y - 4)^2 = 5$$

$$x^2 + y^2 = 16$$

Use un enfoque gráfico para obtener los valores iniciales. Encuentre estimaciones refinadas con el método de Newton-Raphson para dos ecuaciones, que se describe en la sección 6.5.2.

6.14 Repita el problema 6.13, excepto que

$$y = x^2 + 1$$

$$y = 2 \cos x$$

6.15 El balance de masa de un contaminante en un lago bien mezclado se expresa así:

$$V \frac{dc}{dt} = W - Qc - kV\sqrt{c}$$

Dados los valores de parámetros $V = 1 \times 10^6 \text{ m}^3$, $Q = 1 \times 10^5 \text{ m}^3/\text{año}$ y $W = 1 \times 10^6 \text{ g/año}$, y $k = 0.25 \text{ m}^{0.5}/\text{año}$, use el método de la secante modificado para resolver para la concentración de estado estable. Emplee un valor inicial $c = 4 \text{ g/m}^3$ y $\delta = 0.5$. Realice tres iteraciones y determine el error relativo porcentual después de la tercera iteración.

6.16 Para el problema 6.15, la raíz puede localizarse con iteración de punto fijo como

$$c = \left(\frac{W - Qc}{kV} \right)^2$$

o bien como

$$c = \frac{W - kV\sqrt{c}}{Q}$$

De las que solo una convergerá para valores iniciales de $2 < c < 6$. Seleccione la que sea correcta y demuestre por qué siempre lo será.

6.17 Desarrolle un programa amigable para el usuario para el método de Newton-Raphson, con base en la figura 6.4 y la sección 6.2.3. Pruébelo por medio de repetir el cálculo del ejemplo 6.3.

6.18 Desarrolle un programa amigable para el usuario para el método de la secante, con base en la figura 6.4 y la sección 6.3.2. Pruébelo con la repetición de los cálculos del ejemplo 6.6.

6.19 Haga un programa amigable para el usuario para el método de la secante modificado, con base en la figura 6.4 y la sección 6.3.2. Pruébelo con la repetición del cálculo del ejemplo 6.8.

6.20 Desarrolle un programa amigable para el usuario para el método de Newton-Raphson para dos ecuaciones, con base en la sección 6.5. Pruébelo con la solución del ejemplo 6.10.

6.21 Use el programa que desarrolló en el problema 6.20 para resolver los problemas 6.12 y 6.13, con una tolerancia de $\epsilon_s = 0.01\%$.

6.22 El antiguo método de *dividir y promediar*, para obtener una aproximación de la raíz cuadrada de cualquier número positivo, a , se formula del modo siguiente:

$$x = \frac{x + a/x}{2}$$

Demuestre que éste es equivalente al algoritmo de Newton-Raphson.

6.23 a) Aplique el método de Newton-Raphson a la función $f(x) = \tanh(x^2 - 9)$ para evaluar su raíz real conocida en $x = 3$. Use un valor inicial de $x_0 = 3.2$ y haga un mínimo de cuatro iteraciones. b) ¿Converge el método a su raíz real? Bosqueja la gráfica con los resultados para cada iteración que obtenga.

6.24 El polinomio $f(x) = 0.0074x^4 - 0.284x^3 + 3.355x^2 - 12.183x + 5$ tiene una raíz real entre 15 y 20. Aplique el método de Newton-Raphson a dicha función con valor inicial $x_0 = 16.15$. Explique sus resultados.

6.25 Emplee el método de la secante con la función del círculo $(x + 1)^2 + (y - 2)^2 = 16$, a fin de encontrar una raíz real positiva. Haga que el valor inicial sea $x_i = 3$ y $x_{i-1} = 0.5$. Aproxímese a la solución del primer y cuarto cuadrantes. Cuando resuelva para

$f(x)$ en el cuarto cuadrante, asegúrese de tomar el valor negativo de la raíz cuadrada. ¿Por qué diverge la solución?

6.26 Suponga el lector que está diseñando un tanque esférico (véase la figura P6.26) de almacenamiento de agua para un poblado pequeño de un país en desarrollo. El volumen del líquido que puede contener se calcula con

$$V = \pi h^2 \frac{[3R - h]}{3}$$

donde V = volumen [pie³], h = profundidad del agua en el tanque [pies], y R = radio del tanque [pies].

Si $R = 3$ m, ¿a qué profundidad debe llenarse el tanque de modo que contenga 30 m³? Haga tres iteraciones del método de Newton-Raphson para determinar la respuesta. Encuentre el error relativo aproximado después de cada iteración. Observe que el valor inicial de R convergerá siempre.

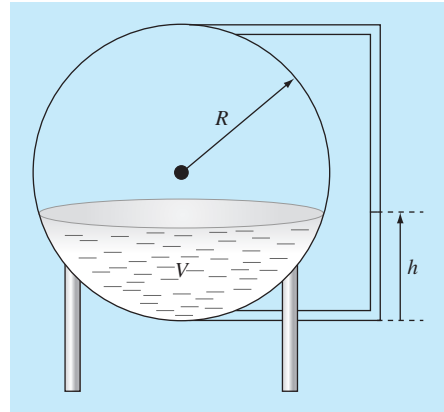


Figura P6.26